重 庆 邮 电 大 学

CHONGQING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 面向端侧深度学习推理的高能效异构多核任务调度算法研究

专业学位类别	电子信息
学　　号	
作者姓名	
指导教师	
学　　院	

学校代码	10617	UDC	004.2
分 类 号	TP301	密级	公开

学 位 论 文

面向端侧深度学习推理的高能效异构多核任务调度算法研究

指导教师 _____

申请学位级别 **硕士** 专业学位类别 **电子信息**

专业学位领域 **新一代电子信息技术（含量子技术等）**

答辩委员会主席 _____ 论文答辩日期 **2026 年 5 月 17 日**

学位授予单位和日期 **重庆邮电大学 2026 年 6 月**

Research on Energy-Efficient Task Scheduling Algorithms for Heterogeneous Multicore Platforms for Edge Deep Learning Inference

A Master Thesis Submitted to
Chongqing University of Posts and Telecommunications

Discipline	**Electronic and Information Engineering**
Student ID	
Author	
Supervisor	
School	

重庆邮电大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文中不包含其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在论文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

重庆邮电大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于 ：

□**公开论文**

□**涉密论文**，保密____年，过保密期后适用本授权书。

（请在以上方框内选择打“√”）

作者签名：　　导师签名：

日期：　　年　　月　　日

摘 要

随着深度学习应用不断向端侧扩展，集成中央处理单元（Central Processing Unit，CPU）、图形处理单元（Graphics Processing Unit，GPU）和神经网络处理单元（Neural Processing Unit，NPU）的异构多核系统已成为智能终端中的重要计算平台。在任务依赖关系、处理单元执行约束和应用截止时间要求都满足的情况下，降低系统长期运行的期望总能耗，已经成为端侧智能系统设计中的重要问题。针对现有研究尚未将 NPU 统一纳入异构多核平台建模，以及在多场景条件下高能效联合调度研究仍不充分，本文围绕面向深度学习应用的高能效异构多核任务调度问题展开研究。

针对上述问题，本文对深度学习应用任务建模，并引入场景化的方法来描述系统在不同运行状态下的任务组合情况，面向 CPU–GPU–NPU 异构端侧平台，综合考虑任务分配、频率选择和任务执行顺序之间的关系，在满足截止时间要求的前提下，建立了以系统总能耗最小为目标的多场景联合优化模型，并将其形式化为紧凑混合整数线性规划（Mixed-Integer Linear Programming，MILP）问题。进一步地，本文提出基于延迟约束生成的 MILP 精确求解方法，通过求解主问题、检测违例窗口和迭代增广约束，得到多场景条件下任务分配、执行顺序和频率配置的全局优化结果。实验结果表明，在复杂的多场景配置下，本文提出的算法相比其他优化算法，系统能耗可进一步降低约 12% ~ 34%。

针对提出的优化算法在大规模任务图下求解时间较长的问题，本文进一步提出面向多场景联合优化问题的分解式启发调度算法。该算法按照初始跨场景统一任务分配、超时场景任务再分配和场景内降频与执行顺序优化的流程，对原问题进行分层处理，在保证可行性的前提下兼顾求解效率和节能效果。实验结果表明，相比其他优化算法系统能耗可进一步降低约 13% ~ 22%。

关键词：异构多核平台，深度学习推理，任务调度，能效优化

ABSTRACT

With the continuous expansion of deep learning applications toward edge devices, heterogeneous multicore systems integrating a Central Processing Unit (CPU), a Graphics Processing Unit (GPU), and a Neural Processing Unit (NPU) have become important computing platforms in intelligent terminals. Under the constraints of task dependencies, processing-unit execution restrictions, and application deadline requirements, reducing the long-term expected total energy consumption of the system has become an important problem in the design of edge intelligent systems. Since existing studies have not yet incorporated the NPU into a unified modeling framework for heterogeneous multicore platforms, and research on energy-efficient joint scheduling under multi-scenario conditions remains insufficient, this thesis focuses on the high-energy-efficiency task scheduling problem for deep learning applications on heterogeneous multicore systems.

To address the above problem, this thesis models deep learning application tasks and introduces a scenario-based method to characterize task combinations under different system operating states. For CPU–GPU–NPU heterogeneous edge platforms, a multi-scenario joint optimization model is established with the objective of minimizing the total system energy consumption while satisfying deadline constraints. The model comprehensively considers the relationships among task allocation, frequency selection, and task execution order, and is further formulated as a compact Mixed-Integer Linear Programming (MILP) problem. Furthermore, an exact MILP solution method based on lazy constraint generation is proposed. By solving the master problem, detecting violated windows, and iteratively augmenting constraints, the method obtains globally optimized results for task allocation, execution order, and frequency configuration under multi-scenario conditions. Experimental results show that, under complex multi-scenario settings, the proposed algorithm can further reduce system energy consumption by approximately 12% ~ 34% compared with other optimization algorithms.

To address the problem of long solving time for the proposed optimization algorithm on large-scale task graphs, this thesis further proposes a decomposed heuristic scheduling algorithm for the multi-scenario joint optimization problem. The algorithm hierarchically handles the original problem through the procedure of initial cross-scenario unified task allocation, task reallocation for deadline-violating scenarios, and in-scenario frequency scaling and execution-order optimization, thereby balancing solving efficiency and energy-

saving performance while ensuring feasibility. Experimental results show that, compared with other optimization algorithms, the proposed method can further reduce system energy consumption by approximately 13% ~ 22%.

Keywords： Heterogeneous Multicore Platform, Deep Learning Inference, Task Scheduling, Energy Efficiency Optimization

目 录

摘 要 ····· I
ABSTRACT ····· II
图目录 ····· VII
缩略词表 ····· VIII
第 1 章 绪论 ····· 1
1.1 研究背景及意义 ····· 1
1.2 国内外研究现状 ····· 3
1.3 论文研究的主要内容 ····· 5
1.4 论文组织结构 ····· 5
第 2 章 理论基础 ····· 7
2.1 本章引言 ····· 7
2.2 系统模型 ····· 7
2.2.1 实时系统模型 ····· 7
2.2.2 包含 NPU 的异构多核处理器 ····· 8
2.3 任务模型 ····· 10
2.3.1 经典周期任务 ····· 10
2.3.2 有依赖关系的图任务模型 ····· 11
2.3.3 面向深度学习应用的任务图表示 ····· 12
2.4 多核系统能耗管理技术 ····· 14
2.4.1 功耗模型 ····· 14
2.4.2 能耗管理及优化技术 ····· 14
2.5 任务调度算法 ····· 16
2.5.1 异构最早完成时间算法 ····· 16
2.5.2 元启发式遗传算法 ····· 17
2.5.3 基于数学规划的全局优化方法 ····· 17
2.6 本章小结 ····· 18
第 3 章 面向深度学习应用的高能效任务调度联合优化方法 ····· 19
3.1 本章引言 ····· 19
3.2 系统模型 ····· 19
3.2.1 任务与场景化建模 ····· 19
3.2.2 处理器模型 ····· 20

3.3 功耗模型 ····· 21
3.3.1 功耗分解与状态模型 ····· 21
3.3.2 CPU 功耗模型 ····· 21
3.3.3 GPU 功耗模型 ····· 22
3.3.4 NPU 功耗模型 ····· 23
3.3.5 统一能耗表达与离散参数化 ····· 23
3.4 问题定义 ····· 24
3.5 问题分析与形式化 ····· 25
3.5.1 任务分配约束 ····· 25
3.5.2 截止时间约束 ····· 26
3.5.3 处理单元执行约束 ····· 26
3.5.4 目标函数与优化问题定义 ····· 28
3.6 LCG-MILP 求解流程 ····· 29
3.6.1 LCG-MILP 的引入 ····· 29
3.6.2 基于延迟约束生成的求解流程 ····· 30
3.7 仿真结果与分析 ····· 33
3.7.1 仿真参数设置与实验设计 ····· 35
3.7.2 仿真结果分析 ····· 37
3.8 本章小结 ····· 42
第 4 章 面向多场景联合优化问题的分解式启发调度算法 ····· 44
4.1 本章引言 ····· 44
4.2 系统模型与任务定义 ····· 44
4.2.1 任务与场景模型 ····· 44
4.2.2 处理器模型 ····· 44
4.3 问题分析 ····· 45
4.4 分解式启发算法流程 ····· 48
4.4.1 初始跨场景统一任务分配 ····· 49
4.4.2 超时场景任务再分配 ····· 51
4.4.3 场景内降频与执行顺序优化流程 ····· 54
4.5 仿真结果与分析 ····· 57
4.5.1 仿真参数设置与实验设计 ····· 57
4.5.2 实验结果分析 ····· 59
4.6 本章小结 ····· 64

第 5 章 总结与展望 ····· 66
5.1 主要工作与创新点 ····· 66
5.2 研究展望 ····· 66
参考文献 ····· 68
作者简介 ····· 73
1. 攻读学位期间的研究成果 ····· 73
1.1 申请（授权）专利 ····· 73
1.2 参与的科研项目及获奖 ····· 73
致 谢 ····· 74

图目录

图 2-1 包含 NPU 在内的端侧异构多核 SoC …… 9
图 2-2 从原生算子到 Stage 级任务图的建模过程 …… 13
图 2-3 DPM 与 DVFS 工作机制对比 …… 15
图 3-1 异构处理器抢占机制不同 …… 27
图 3-2 引入窗口约束 …… 27
图 3-3 LCG-MILP 算法框图 …… 29
图 3-4 MILP 算法流程图 …… 31
图 3-5 不同配置下的能耗表现 …… 37
图 3-6 异构处理单元频率与功耗的关系 …… 38
图 3-7 不同配置下不同算法的能耗对比 …… 39
图 3-8 不同截止时间系数下各算法性能对比 …… 40
图 3-9 不同任务规模下各算法的可调度率比较 …… 41
图 3-10 不同算法求解时间随任务规模的变化 …… 42
图 4-1 DHA 算法框图 …… 48
图 4-2 初始跨场景统一任务分配流程图 …… 50
图 4-3 超时场景任务再分配流程图 …… 52
图 4-4 场景内降频与执行顺序优化流程图 …… 55
图 4-5 不同场景配置下 DHA 与 LCG-MILP 算法的能耗对比 …… 59
图 4-6 不同算法能耗对比 …… 61
图 4-7 不同截止时间系数下各算法性能对比 …… 62
图 4-8 不同任务规模下各算法的可调度率比较 …… 63
图 4-9 不同算法求解时间对比 …… 64

缩略词表

英文缩写	英文全称	中文全称
DNN	Deep Neural Network	深度神经网络
CPU	Central Processing Unit	中央处理单元
GPU	Graphics Processing Unit	图形处理单元
DSP	Digital Signal Processor	数字信号处理器
NPU	Neural Processing Unit	神经网络处理单元
SoC	System on Chip	片上系统
CNN	Convolutional Neural Network	卷积神经网络
DAG	Directed Acyclic Graph	有向无环图
DVFS	Dynamic Voltage and Frequency Scaling	动态电压频率调整
AI	Artificial Intelligence	人工智能
MPSoC	Multiprocessor System on Chip	多处理器片上系统
ILP	Integer Linear Programming	整数线性规划
MILP	Mixed-Integer Linear Programming	混合整数线性规划
QoS	Quality of Service	服务质量
SMP	Symmetric Multiprocessing	共享内存多核处理器结构
CMOS	Complementary Metal-Oxide-Semiconductor	CMOS 集成电路工艺
NP-hard	Nondeterministic Polynomial-time hard	NP-困难

第 1 章 绪论

1.1 研究背景及意义

近年来，深度学习已经广泛用于目标检测、语义分割、语音交互和多模态理解等任务，并持续提升系统的感知和决策能力。随着应用需求变化，深度学习也逐步从云端训练和推理扩展到端侧执行。这样做可以缩短响应时间，减少对网络传输的依赖，也能增强数据本地处理和离线运行能力[1]。和云端集中推理相比，端侧部署通常可以减少数据回传，降低带宽占用，也能减小网络波动对时延的影响。在工业视觉、智能巡检、预测性维护和边缘监测这类更依赖本地处理的场景中，端侧部署还能提高系统独立运行的能力[1-3]。

不过，端侧部署也面临不少限制。当系统需要同时支持多路推理任务、保持持续在线运行并满足严格响应截止时间，系统优化不再仅仅关注吞吐率提升或平均能耗降低，而需要在多种复杂情况下进行综合考虑。更关键的是，系统能否在满足端到端截止时间约束的前提下，以较低能耗持续提供稳定的推理服务[4-5]。不过，端侧部署面临的限制并不只来自计算能力不足，还来自内存容量、资源抢占和异构专用单元协同等多方面约束。近年来的研究已经表明，在边缘设备上，深度神经网络（Deep Neural Network, DNN）推理性能瓶颈往往并不单纯由算力决定。FlexNN 针对内存受限边缘设备指出，DNN 推理过程中的模型切分、数据加载和计算安排需要统一规划。否则，内存碎片化和动态调整开销会明显降低推理效率。[6]。

与此同时，随着移动边缘设备多模型并发推理需求的增长，图形处理单元（Graphics Processing Unit, GPU）的资源竞争问题和实时性压力进一步加剧。Pantheon 研究了移动边缘 GPU 上多 DNN 推理的可抢占执行问题，说明移动 GPU 上已有的优先级机制可以结合软件调度设计，用于区分实时任务和非实时任务，从而改善多模型共存时的响应能力[7]。从硬件体系结构角度看，端侧人工智能（Artificial Intelligence, AI）平台也在持续从中央处理单元（Central Processing Unit, CPU）或 CPU–GPU 结构，演化为集成数字信号处理器（Digital Signal Processor, DSP）、神经网络处理单元（Neural Processing Unit, NPU）等专用处理单元的异构片上系统（System on Chip, SoC）。Mandheling 表明，DSP 不仅可以支持端侧 DNN 训练，还可以借助 CPU 与 DSP 的协同调度以及混合精度计算，改善时间和能耗表现[8]，在端侧大模型推理中，NPU 已经成为提升性能和能效比的关键资源[9]。

专用加速器研究表明，深度学习任务的执行效率受数据流组织、片上存储利用和任务映射方式影响很大。Eyeriss 通过优化数据流、片上存储层次和数据移动，降低了卷积神经网络推理的系统总能耗[10]。Cambricon-X 针对稀疏神经网络设计

了专用架构，提高了稀疏模型的执行效率[11]。MAESTRO 进一步指出，DNN 的性能和能耗不是只由硬件决定，还和网络层特征、映射方式以及硬件组织有关[12]。

传统的单核处理器以及同构多核处理器由于其固有的架构限制和计算能力瓶颈，已经难以满足现代应用场景对处理器性能的需求[13]。现在的端侧 SoC 通常采用 CPU、GPU 和 NPU 协同工作的异构架构，而不是过去以同构多核为主的形式。原因在于，继续提高通用核频率，或者继续增加同构核心数量，已经很难在既定功耗预算下持续换来有效的计算收益。受功耗墙和暗硅效应影响，体系结构逐步转向通用控制、并行计算和专用加速相结合的设计方式[14]。基于此，异构多核平台已在工业控制系统、自动驾驶系统、通信系统以及视频图像处理系统等领域得到广泛应用[15]。在这类平台中，CPU 更适合处理控制流复杂、并发度较低的任务，GPU 更适合处理大规模并行计算，NPU 则依靠面向张量计算的数据通路和片上存储优化，在深度学习主干算子上通常能用更少能耗完成计算[16-17]。

但异构架构也让系统调度更复杂。同一个推理阶段分配到不同处理单元后，完成时间、功耗、资源占用情况和数据布局要求都可能不同。尤其是 NPU 往往还受算子支持范围、量化格式、编译部署流程和片上存储容量等条件限制。这样一来，资源分配和调度就不能只看哪个处理单元算得更快，还要考虑任务能否部署、代价是否合适，以及整体安排是否协调[16, 18]。

除了平台本身的异构性，深度学习推理任务的结构也会增加调度难度。推理过程本身具有前后依赖关系，适合表示成带端到端时限约束的有向无环图（Directed Acyclic Graph, DAG）任务图。当平台中同时存在 CPU、GPU 和 NPU 时，图中不同阶段映射到不同处理单元后，完成时间和资源占用都会变化，这会直接增加关键执行路径判定、资源争用处理和截止时间分析的复杂度[18-20]。而且，推理任务在实际运行中也不是始终不变。输入复杂度会变化，分辨率和帧率会切换，应用会启动和停止，并发情况也会变化。这样一来，系统在长期运行中往往会呈现出几种相对稳定的运行模式。如何在这些模式之间调整部署方式和运行配置，使系统在长期运行中保持较低能耗，同时在运行压力较大的场景下仍满足截止时间约束，已经成为端侧异构推理中的一个重要问题[18, 21]。

另一个要提到的是动态电压频率调整（Dynamic Voltage and Frequency Scaling, DVFS）。它是端侧 SoC 中最常见的能耗控制方法之一。降低频率和电压可以减少动态功耗，但也会延长执行时间。这样一来，任务分配、调度顺序和频率配置就会互相影响。平台中的异构处理单元越多，这种影响越明显，问题也越难求解[5, 22]。

基于这些背景，本文关注的问题是，在 CPU、GPU 和 NPU 构成的异构端侧平台上，面对具有依赖关系的深度学习推理任务和多种运行模式，如何在满足依赖约束、资源约束和端到端截止时间约束的前提下，联合确定任务分配、频率配置和

执行顺序，并尽量降低系统长期运行时的能耗[23]。围绕这一问题开展研究，既有理论意义，也有工程意义。从理论上看，需要把异构平台表示、图结构任务描述、频率相关能耗建模以及多种运行模式下的资源协调纳入同一个优化框架，建立更符合端侧 AI SoC 特点的联合建模方法。从工程上看，这项研究可以为实际部署提供可解释、可比较的配置依据。在中小规模实例上，可以给出精确优化结果作为参考。在更大规模任务图上，也可以提供能够扩展的近似求解方法，为端侧深度学习系统的低功耗稳定运行提供支持[5, 18, 21]。

1.2 国内外研究现状

围绕本文研究的问题，现有工作大体可以分成三个方向。第一类是传统异构多核平台上的能耗感知映射与调度。第二类是面向端侧深度学习推理的资源控制和实时调度。第三类是把 NPU 或运行场景变化纳入考虑的异构推理协同优化。

第一类研究主要面向通用异构多核平台或异构多处理器片上系统（Multiprocessor System on Chip, MPSoC）的能耗优化调度。相关研究通常将异构处理器集合、任务分配、任务开始时间和频率配置作为核心决策变量，并基于整数线性规划（Integer Linear Programming, ILP）、混合整数线性规划（Mixed-Integer Linear Programming, MILP）或其他数学规划方法，在给定截止时间、资源容量和能耗模型的条件下求解全局最优解，或者给出可证明的最优界[24-27]。此外，部分研究将 DAG 任务依赖关系和 DVFS 机制共同纳入模型，用于分析或优化能耗与截止时间之间的关系。上述方法具有较为清晰的问题定义、目标函数和约束条件，因此能够为能耗优化调度研究提供重要参考。然而，现有研究面对的对象主要是通用任务图或传统异构多核平台，尚未充分考虑 NPU 参与下的异构协同计算特征，也较少将端侧深度学习推理中常见的算子级或阶段级任务直接纳入模型。因此，将上述方法直接用于 CPU、GPU 和 NPU 协同推理场景时，仍然存在明显的建模缺口[18, 20]。

第二类研究更直接面向端侧深度学习推理本身。该方向更关注系统实现和运行时控制，重点是降低推理过程中的资源消耗和能耗。DeepX 通过软硬件协同加快移动端深度学习执行，减少了推理过程中的时间和资源消耗[28]。NestDNN 通过构建不同容量的模型集合，并结合运行时资源状态进行选择，在多应用并发条件下协调资源、精度和运行代价之间的关系[29]。DeepEye 则从本地同时执行多个视觉模型的角度出发，改善了资源使用情况和设备续航[30]。上述研究推动了端侧推理系统的发展。研究重点已经不再只是模型能否在端侧运行，而是如何在资源受限条件下持续运行。但相关工作大多更关注吞吐率、资源适应能力或者模型选择问题，对于在截止时间约束下联合优化任务映射、调度和 DVFS，并以总能耗最小为目标

的研究还不够充分。换言之，现有研究已经说明端侧推理需要面向能耗的运行时管理，但尚未系统解决本文关注的联合优化问题。

第三类研究开始更直接地处理实时性、异构资源协同和专用加速单元调度等问题。比如，面向多 DNN 或流式推理的研究，把截止期、空闲时间和分阶段并行执行等因素纳入考虑，用来提高截止期满足率和系统吞吐率[31-32]。还有一些面向多加速器或多租户系统的研究，开始考虑 CPU、GPU 和其他加速器之间的在线资源协调，以满足服务质量（Quality of Service, QoS）要求或提高资源使用水平[19, 33]。进一步地，部分显式考虑 NPU 的协同调度研究表明，NPU 的加入虽然使端侧推理能够在单位能耗下完成更多计算，但也引入了新的决策问题。比如，哪些任务适合放到 NPU 上执行，哪些任务需要退回 GPU 或 CPU，实时约束下又该怎样协调这些处理单元。上述问题均需要新的调度方法加以处理[34]。相关研究表明，CPU、GPU 和 NPU 协同调度已经成为端侧推理中的一个重要方向。但现有方法大多偏向启发式方法、在线控制或者局部决策，通常没有用统一的数学规划形式给出全局能耗最优解。

如果把问题放到更大的背景里看，端侧计算平台本身也一直在演进。早期平台多是同构多核对称共享内存多处理器结构（Symmetric Multiprocessing, SMP）。之后逐渐发展为 CPU 和 GPU 组成的传统异构架构。现在又进一步演化为集成 NPU 的边缘 AI SoC。和这个变化对应，系统处理的计算形式也从以标量控制为主，逐步扩展到标量、向量和张量计算并存。这里真正重要的，不是某一种具体芯片的形态，而是调度对象和约束条件已经变了。NPU 被纳入 SoC 之后，系统优化问题不再只是把任务分配到若干近似同构的核心上，而是要在计算特点不同、可执行算子范围不同、频率域不同、单位能耗下计算能力也不同的处理单元之间，联合确定任务映射、执行顺序和频率配置。这正是本文研究问题产生的直接背景[14, 16-17]。

除异构硬件本身外，面向具体运行场景的离线优化也为本文问题提供了重要的方法参考。已有研究表明，针对输入变化带来的运行状态变化和多应用模式切换，可以采用面向场景的离线优化，再配合运行时切换管理，在较低在线调度代价下改善截止时间满足情况和能耗水平[18, 21]。不过，现有场景化研究中，一部分更偏向通用 SoC 应用映射，另一部分虽然开始面向深度学习推理，但通常输出的是一组可供选择的配置方案，再由运行时根据场景进行选择，较少把 NPU 约束和离散频率选择直接纳入模型，并进一步把跨场景共享部署、场景内频率配置和执行时序一起放进统一的精确能耗优化框架中[18, 21]。

综合来看，现有研究已经分别在几个方向上取得进展。传统异构调度研究证明了用数学规划刻画能耗和截止时间约束联合优化是可行的。端侧深度学习推理研究说明了面向资源状态的调节和运行时控制对系统运行和续航很重要。NPU 协

同与场景化研究则说明，端侧 AI SoC 已经不适合继续用单一 CPU 或 CPU 和 GPU 的视角来处理。不过，现有工作之间仍存在明显不足。一方面，精确优化方法往往缺少对 NPU 约束、端侧推理执行过程以及多运行模式切换过程的统一描述。另一方面，NPU 场景化方法又多采用启发式方法，只关注局部决策，较少在统一模型下同时考虑任务分配、执行顺序和频率配置，来降低系统总体能耗。

1.3 论文研究的主要内容

本文研究 CPU–GPU–NPU 异构端侧平台上的深度学习推理任务，关注端到端截止时间约束下的低能耗任务分配、执行顺序安排和频率配置问题，主要工作如下：

（1）建立面向端侧异构推理的统一模型。本文从平台、任务和能耗三个方面进行建模。在平台方面，把 CPU、GPU 和 NPU 放到同一个异构资源集合中描述。在任务方面，针对深度学习推理过程中的依赖关系，构建适合任务分配和调度求解的图结构任务表示。在能耗方面，建立与频率相关的功耗和执行时间表达，为后续能耗计算和优化目标构造提供统一的参数基础。

（2）建立联合优化问题，并给出精确求解流程。在统一模型基础上，本文进一步研究，在不同运行条件和不同应用组合下，怎样联合确定任务分配、频率配置和执行顺序，使系统在满足任务前后依赖关系以及截止时间要求的前提下，尽可能降低整体运行能耗。

（3）面向大规模任务图设计可扩展的启发式求解方法。考虑到精确方法在任务规模扩大后往往难以求解，本文进一步针对大规模图结构推理任务设计启发式能耗优化算法。在保持前文平台模型、任务模型设定一致的前提下，提高复杂实例下的可计算性和实际应用价值。

1.4 论文组织结构

全文共分为五章，内容安排如下。

第 1 章为绪论。本章介绍研究背景和研究意义，梳理相关研究现状，说明本文问题的来源及研究价值，并概括论文的主要内容和整体安排。

第 2 章介绍本文研究所涉及的理论基础；本章围绕端侧异构多核系统、深度学习推理任务表示以及能耗与频率调节等内容，梳理后文分析和求解所需要的基本概念与相关方法。

第 3 章定义面向场景的深度学习应用的联合优化问题，并构建相应的精确优化模型。并给出精确求解流程以及仿真实验及结果分析。

第 4 章针对第 3 章方法在大规模任务图下难以直接求解的问题，进一步设计可扩展的启发式能耗优化方法。本章在与前文设定保持一致的基础上，提高大规模实例下的求解能力。

第 5 章对全文工作进行总结，并说明当前工作的不足，对后续研究进行展望。

第 2 章 理论基础

2.1 本章引言

本章主要介绍异构多核平台任务调度研究中的相关概念，以及本文用于对比的典型调度算法，为后续章节的问题分析、方法设计和结果验证提供基础。

本章内容安排如下。第 2.2 节介绍实时系统和异构处理器的基本模型。第 2.3 节介绍任务模型。第 2.4 节讨论功耗模型、能耗模型以及 DVFS 的影响。第 2.5 节介绍多核系统中的能耗管理方法。第 2.6 节归纳典型调度算法。第 2.7 节对本章进行总结。

2.2 系统模型

2.2.1 实时系统模型

实时系统关注的，不只是计算结果是否正确，还要看结果能否在规定时间内给出。也就是说，实时系统的正确性既取决于结果本身，也取决于是否满足时间要求，例如截止时间[35-37]。早期实时系统主要用于过程控制、航空航天和工业自动化等场景，这些场景对时间可预测性要求很高。随着计算平台和应用不断发展，实时系统逐步形成了以任务模型、可调度性分析和调度算法为核心的理论体系[35-36]。

随着端侧智能快速发展，深度学习推理开始进入越来越多有时间要求的应用过程，例如在线感知、交互响应、检测和决策等[1]。这类任务通常具有周期触发或高频触发的特点，同时还受到能耗和散热条件的限制[4]。因此，面向端侧深度学习的系统设计，往往需要同时考虑截止时间要求和能耗限制，并在此基础上进行资源分配和调度安排[5, 18]。近几年，面向异构处理器的实时推理研究也说明，这一问题正受到越来越多关注[19, 33]。在实际应用中，实时系统又通常按时间要求分为三类，分别是硬实时、软实时和准实时[36-37]。

（1）硬实时系统：硬实时系统要求每一次任务都在截止时间之前完成，任何一次超时都可能带来不可接受的后果[35-36]。因此，硬实时系统强调在最不利情况下仍能满足时间要求，也就是在最坏执行路径和最强干扰条件下，系统仍然能够验证满足截止时间[36]。在经典周期任务模型中，任务通常记为 $\tau_i = (C_i, T_i, D_i)$。其中，C_i 表示最坏执行时间，T_i 表示周期，D_i 表示相对截止时间[35-36]，本文采用硬实时系统作为后续建模与分析基础，假定各推理任务均需在给定截止时间内完成，并据此开展任务分配、调度与能耗优化研究。

（2）软实时系统：软实时系统同样有截止时间要求，但允许偶发的超时[36]。在

这种情况下，任务如果晚于截止时间完成，通常意味着结果已经不够及时，系统表现会下降，但一般不会造成严重后果[37]。因此，软实时系统更关注整体上的时间表现，例如平均响应时间、超时程度，以及系统在运行条件变化时能否保持稳定[36]。在端侧深度学习推理中，很多非安全关键任务更接近这一类。一方面，这些任务通常能够容忍少量超时。另一方面，推理时间可能会随着输入特征、并发情况以及编译和运行过程中的优化变化而波动，这使得严格建立最坏执行时间上界不仅计算开销较高，也容易过于保守[1, 4]。

（3）准实时系统：准实时系统介于硬实时和软实时之间，更接近许多流式感知和推理任务的实际需求。它允许少量受控制的超时，但对超时出现的方式有要求。常见情况是，一次任务如果错过截止时间，结果在逻辑上已经失效，可以直接丢弃；但从整体上看，系统只允许少量这样的情况出现，而且不能无控制地连续积累[38-39]。为描述这种允许但受限制的超时，研究中提出了更细的窗口约束模型。例如，(m, k)-firm 约束要求任意连续 k 个 job 中，至少有 m 个按时完成，这样不仅限制了超时次数，也限制了超时在时间上的分布[40]。weakly-hard 框架则通过描述窗口内按时完成和超时的模式，用更一般的方式表示这类允许超时但必须受控的需求[39]。在以帧为单位或近似周期性触发的深度学习推理场景中，准实时建模较为常见，系统通常可以容忍少量偶发的丢帧或跳帧，但难以接受连续多帧超时，否则会导致处理过程出现明显中断[32, 34]。因此，后续调度设计不仅需要保证任务在滑动窗口内的时限可满足性，还需要兼顾能耗预算约束下系统的长期稳定运行[4-5, 18]。

2.2.2 包含 NPU 的异构多核处理器

多核处理器是在一块单独的芯片上集成了多个处理器核心[41]，每个核心都可以独立执行指令，处理数据，类似于多个单核处理器并行工作。处理器核间通过集成总线进行通信，不仅能够传递数据，还能共同处理并行线程计算。根据芯片上集成的核心架构类型异同，多核处理器可以分为同构多核和异构多核两类[42]。随着工艺不断演进，单芯片可集成的晶体管数量持续增加。早期提高计算能力，主要依赖提升单核频率或增加单核复杂度。但这种方式越来越受到功耗和散热条件的限制，多核处理器因此成为提高并行处理能力的重要方向[14]。早期单芯片多处理器研究已经表明，在一块芯片上集成多个相同处理核心、共享片上资源，并利用线程级并行提高计算能力是可行的[43-44]。这类同构多核处理器由结构相同、计算特点和能耗特点接近的核心组成[45]，所以，这类任务更容易采用统一的执行方式和调度方法。不过，在功耗墙约束下，仅靠增加同构核心数量，已经难以继续提高单位功耗下的计算能力。尤其是暗硅现象表明，随着芯片集成规模不断扩大，受功耗密

度限制，能够同时开启并保持高效运行的电路比例会逐渐受限。这也推动处理器体系结构从单一的通用计算，逐步转向通用计算、并行计算和专用加速结合的方向[14]。

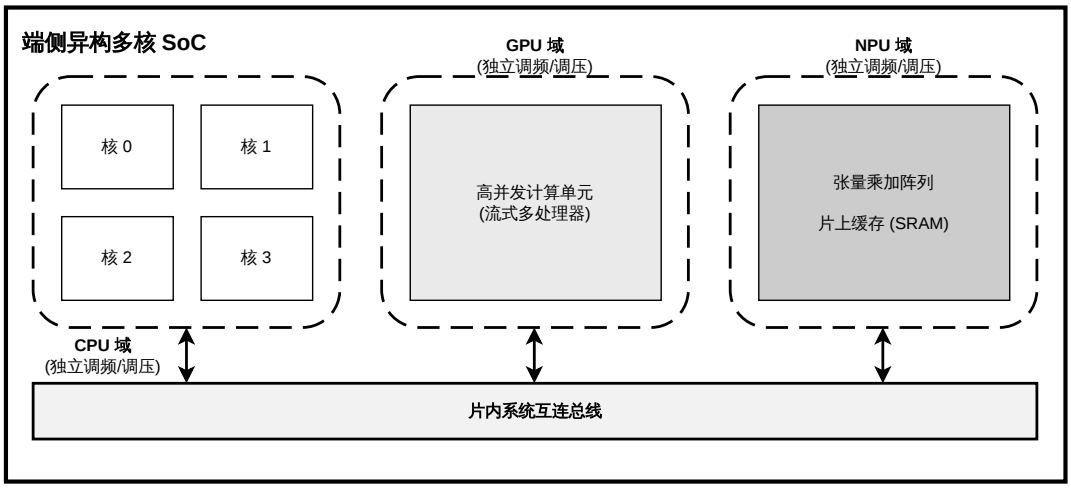


图 2-1 包含 NPU 在内的端侧异构多核 SoC

Fig. 2-1 Edge heterogeneous multi-core SoC including an NPU

异构多核或多处理器系统是在单芯片上集成不同结构、不同数据通路的计算单元，再根据任务特点选择更合适的处理单元执行，以提高计算资源的使用效果，并减轻功耗约束带来的限制[14]。在端侧 AI 推理场景中，这类平台通常还会进一步集成面向张量计算或卷积计算的专用加速单元，也就是常说的 NPU，从而形成 CPU、GPU 和 NPU 协同工作的结构[16-17]。如图 2-1 所示，典型的端侧异构多核 SoC 通常包括 CPU 域、GPU 域和 NPU 域，并通过片上互连总线进行数据交互。CPU 域由多个通用处理核心组成，主要负责系统控制和通用计算任务。GPU 域包含适合高并发执行的并行计算单元，适合处理计算模式固定的并行计算的大规模任务。NPU 域通常由张量乘加阵列及其配套的片上缓存组成，主要用于支持深度学习推理中的张量计算。与此同时，这三个计算域通常具有各自独立的调频和调压能力，这为运行过程中协调计算速度和能耗提供了硬件条件。

从功能特点来看，不同类型的计算单元各有适合的任务类型。CPU 具有较强的通用处理能力，更适合处理控制关系复杂、分支较多或并发程度较低的任务，也常用于承担系统管理、任务协调和数据预处理等工作。这类任务通常对执行流程控制要求较高，计算过程也不完全规则，因此更适合由 CPU 完成。GPU 则更适合处理大规模数据并行计算，常用于计算模式较为固定、可并行程度较高的算子计算。在这类任务中，大量计算操作可以同时展开，因此 GPU 能较好发挥并行处理优势。NPU 主要面向深度学习中的张量计算进行专门设计，通常具有针对乘加运算的数

据通路和片上存储结构，因此在卷积、矩阵运算等主干计算中往往能够以更低的能耗完成任务。也正因为如此，NPU 的引入不仅提高了端侧平台对深度学习推理任务的支持能力，也使系统在满足截止时间要求的同时，拥有了更大的能耗调节空间[4, 16]。

但这种架构也使任务分配和调度问题复杂化。同一个任务在不同计算单元上执行时，完成时间、功耗和共享资源占用情况可能差别很大[5, 18]。同时，NPU 还受到算子支持范围、量化与编译部署条件，以及张量格式和内存布局要求的限制，这使任务分配并不是任意可选的，而是受到较强约束[16, 18]。因此，在包含 NPU 的异构多核平台上研究深度学习推理的能耗优化问题时，需要把不同计算单元的执行差别、NPU 的任务分配限制，以及实时任务的截止时间要求和能耗目标统一考虑，作为后续任务表示、功耗分析和 DVFS 建模的基础[5, 18, 21]。

2.3 任务模型

任务模型用于描述计算任务的到达方式、时间要求和内部结构，是实时系统分析和调度研究的基础。从已有研究来看，任务模型逐步从独立的周期任务和非周期任务，扩展到更细化的任务到达模型，再发展到有依赖关系的图任务模型。

2.3.1 经典周期任务

经典实时调度理论通常把相互独立的任务集作为基本研究对象。在最常见的周期任务模型中，任务 τ_i 可表示为

$$\tau_i = (C_i, T_i, D_i) \tag{2-1}$$

式中，C_i 表示任务的最坏情况执行时间；T_i 表示任务周期；D_i 表示相对截止时间。

在隐式截止时间条件下，通常有 $D_i = T_i$。若任务 τ_i 在时刻 $r_{i,k}$ 释放第 k 次任务作业，则该任务作业可记为

$$J_{i,k} = (r_{i,k}, C_i, d_{i,k}) \qquad d_{i,k} = r_{i,k} + D_i \tag{2-2}$$

周期任务模型形式简单，便于分析，为固定优先级调度、最早截止时间优先调度以及利用率分析提供了基础[35]。例如，对于单处理器上的独立隐式截止时间周期任务集，总利用率常写为

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \tag{2-3}$$

这一表达直接反映了任务集对处理器计算能力的平均需求，也是经典可调度性分

析的出发点[35]。

当任务到达不再严格周期，而是由外部事件触发时，研究中进一步引入了最小到达间隔任务模型[46]。这类任务仍可写为

$$\tau_i = (C_i, T_i^{\min}, D_i) \tag{2-4}$$

式中，$T_i^{\min}$ 表示相邻两次任务作业到达之间的最小时间间隔；与严格周期模型相比，这种表示更适合描述传感器触发、由用户操作触发和边缘设备上到达时间不固定的输入[46]。

2.3.2 有依赖关系的图任务模型

为了描述一个任务内部多个计算部分之间的前后关系，研究中常用图结构任务模型，其中最常见的是有向无环图模型[47-49]。一个 DAG 任务通常表示为

$$G = (V, E) \tag{2-5}$$

式中，$V = \{v_1, v_2, \ldots, v_m\}$ 表示节点集合；每个节点对应一个可以单独安排执行的计算部分；$E \subseteq V \times V$ 表示有向边集合；若 $(v_i, v_j) \in E$，则表示节点 v_j 只有在节点 v_i 完成后才能开始执行。

对于 DAG 中的每个节点 $v \in V$，通常赋予一个执行计算开销 $c(v)$。据此，图任务的总计算量可写为

$$Q(G) = \sum_{v \in V} c(v) \tag{2-6}$$

而关键执行路径计算开销可定义为

$$L(G) = \max_{\pi \in \mathcal{P}(G)} \sum_{v \in \pi} c(v) \tag{2-7}$$

式中，$\mathcal{P}(G)$ 表示从源节点到汇节点的所有有向路径集合；$Q(G)$ 表示完成整个图任务所需的计算总量；$L(G)$ 则表示在前后关系约束下，沿最长依赖路径完成任务所需的最小时间下界。

与独立任务模型相比，DAG 任务模型有两个明显优点。一方面，它可以直接表示串行、分支和汇聚等关系，因此更适合描述复杂任务的内部组成。另一方面，它可以保留互不依赖节点之间的并行执行可能，为多核平台和异构平台上的调度提供基础。因此，DAG 已成为并行实时任务、工作流调度和异构计算任务建模中最常用的表示方式之一[49]。

2.3.3 面向深度学习应用的任务图表示

深度学习应用，尤其是神经网络推理任务，通常可以表示成计算图[50]。从组成上看，神经网络由卷积、归一化、激活、池化、矩阵乘法、注意力和残差连接等计算部分构成，这些部分通过张量数据传递连接起来，形成从输入到输出的有向计算过程。因此，可将神经网络推理图表示为

$$G_{\mathrm{nn}} = (V_{\mathrm{nn}}, E_{\mathrm{nn}}) \tag{2-8}$$

式中，V_{nn} 表示网络中的层或算子节点集合；E_{nn} 表示由张量传递关系构成的数据依赖边集合。

若节点 $v \in V_{\mathrm{nn}}$ 对应一个算子或网络层，则可为其关联执行计算开销、访存需求和中间张量规模等属性。例如，可将节点属性写为

$$\phi(v) = \big(c(v),\, m(v),\, q(v)\big) \tag{2-9}$$

式中，$c(v)$ 表示计算开销；$m(v)$ 表示中间数据访问或存储需求；$q(v)$ 表示与输入尺寸、通道数或批大小相关的结构参数。

采用这种表示后，神经网络就不再被看作一个整体任务，而是被表示为由多个存在前后关系的计算节点组成的图任务[50]。

需要说明的是，深度学习应用中的任务图并不总是直接对应训练框架中的原始层级表示。在实际部署过程中，推理引擎和深度学习编译器通常会对计算图进行优化，例如常量折叠、死节点消除、布局变换和算子融合等[51-52]。这些处理主要是为了减少中间数据搬移，降低内核启动开销，并改善实际执行过程中的资源使用情况[51-52]。因此，部署阶段得到的执行图，通常可以看作是在原始计算图基础上重新组织后的结果[51-52]。

从形式上看，若原始计算图为 $G_{\mathrm{nn}} = (V_{\mathrm{nn}}, E_{\mathrm{nn}})$，并对节点集合作一个划分

$$\Pi = \{S_1, S_2, \ldots, S_K\} \qquad \bigcup_{k=1}^{K} S_k = V_{\mathrm{nn}} \quad S_i \cap S_j = \varnothing \ (i \neq j) \tag{2-10}$$

则可得到一个层次更高的子图表示

$$G_{\Pi} = (V_{\Pi}, E_{\Pi}) \tag{2-11}$$

式中，$V_{\Pi} = \{S_1, S_2, \ldots, S_K\}$，每个 S_k 表示由若干原始算子组成的子图或执行阶段。

若存在 $u \in S_i$ 与 $v \in S_j$ 使得 $(u, v) \in E_{\mathrm{nn}}$ 且 $i \neq j$，则在 G_{Π} 中建立边 $(S_i, S_j) \in E_{\Pi}$。这种从原始算子图到执行图的转化，反映了部署过程中对神经网络计算图进

行重组和融合的结果[51-52]。

因此，从任务表示的角度看，深度学习应用通常有三种常见形式。第一种是把一次完整推理看作一个整体任务。第二种是把层或算子作为节点，直接表示原始计算图。第三种是在图优化或算子融合之后得到的子图表示。前两种分别对应整体层面的任务表示和原始网络结构的直接表示，第三种则更接近实际部署时的执行方式[50-52]。无论采用哪一种表示形式，其共同点都在于用图结构表示推理过程中的数据依赖关系、可并行关系和结构限制。

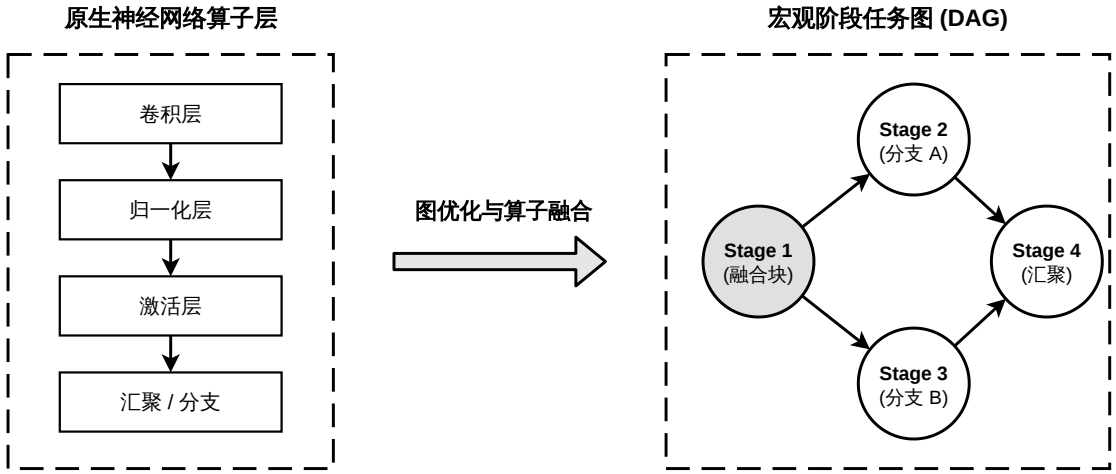


图 2-2 从原生算子到 Stage 级任务图的建模过程

Fig. 2-2 Modeling process from primitive operators to a stage-level task graph

图 2-2 给出了本文采用的任务表示方法。原始神经网络通常由卷积层、批归一化层、激活层、池化层和残差连接等多种网络层逐层组成，这些网络层共同形成特征提取、非线性变换以及分支汇合等计算过程。如果直接把这些原生层作为调度对象，虽然能够完整保留网络结构，但也会使节点数量过多，前后关系变得复杂，从而增加后续求解难度。基于这一点，本文在保持关键前后关系不变的前提下，结合编译期图优化和算子融合，将卷积、批归一化和激活等可以连续执行、且数据局部性较强的网络层合并为一个计算阶段，而将残差、汇合等相对独立的部分保留为单独阶段，据此形成后续优化所使用的 Stage 级任务图[51-52]。这样处理以后，一方面可以减少调度对象的数量，缩小优化问题规模；另一方面也使任务表示更接近实际推理引擎中的执行方式[51-52]。

这里的 Stage 是指，在保持关键前后关系不变的前提下，将若干数据联系紧密、能够连续执行或可以在编译阶段合并的细粒度算子组织成一个计算阶段。和直接按原生算子建模相比，阶段级表示在保留任务结构的同时，采用了更大的调度单位，更适合在异构平台上进行联合优化。

2.4 多核系统能耗管理技术

2.4.1 功耗模型

功耗模型用于描述处理器在执行任务时的能量消耗规律，是后续能耗建模与优化分析的基础。从数字电路角度看，现代 CMOS 集成电路工艺（Complementary Metal-Oxide-Semiconductor, CMOS）处理器的功耗通常可分为动态功耗和静态功耗两部分[53]。其中，动态功耗主要来源于晶体管翻转时对负载电容的充放电过程，静态功耗则主要来源于漏电流。在经典近似下，动态功耗可表示为

$$P_{\mathrm{dyn}} \propto \alpha C V^2 f \tag{2-12}$$

式中，α 表示平均开关活动因子；C 表示等效开关电容；V 表示供电电压；f 表示时钟频率。

静态功耗通常与漏电流和供电电压相关，可表示为

$$P_{\mathrm{sta}} \propto I_{\mathrm{leak}} V \tag{2-13}$$

因此，处理器总功耗可写为

$$P = P_{\mathrm{dyn}} + P_{\mathrm{sta}} \tag{2-14}$$

从能量角度看，功率表示单位时间内的能量消耗速率，而能量等于功率对时间的积分。在简化分析中，若处理器在一段时间内功率近似稳定，则能量可表示为

$$E = P \cdot T \tag{2-15}$$

式中，P 表示平均功率；T 表示执行时间。

对于不同类型的处理器，虽然产生功耗的方式相同，但其主要影响因素并不完全相同。CPU 的功耗通常与核心频率、电压以及指令多少相关；GPU 除核心频率和电压外，还受到存储子系统频率、线程并发度和访存行为的影响；NPU 或其他神经网络加速器除了受频率和电压控制外，还与数据搬移、片上缓冲区访问、阵列利用率以及算子类型等因素有关。

2.4.2 能耗管理及优化技术

动态电源管理（Dynamic Power Management, DPM）是面向处理器空闲阶段的典型能耗优化技术。随着半导体工艺进入深亚微米阶段，器件漏电流不断增加，静态功耗在芯片总功耗中的占比持续上升，在部分低负载场景下甚至可能超过动态功耗。DPM 的基本思想是利用系统运行过程中的负载波动，在处理单元出现空闲

间隙时，将其从活动状态切换到低功耗状态，以减少空闲期间的能量消耗。常见的低功耗状态包括挂起（Suspend）、休眠（Sleep）和深度休眠（Deep Sleep），其实现方式通常涉及时钟门控（Clock Gating）或电源门控（Power Gating）。

尽管 DPM 能够有效降低空闲阶段的功耗，但其实际应用受到状态切换开销的限制。处理器从低功耗状态恢复到工作状态时，通常需要重新稳定时钟和电压，并完成必要的上下文恢复，因此会带来额外的唤醒时延（Wake-up Latency）和唤醒能耗（Wake-up Energy）。如果空闲区间不足以覆盖这一状态切换开销，则频繁进入和退出低功耗状态反而可能增加系统总能耗。因此，DPM 更适用于存在较长可利用空闲区间的场景。

与 DPM 侧重利用空闲时间节能不同，动态电压频率调整（Dynamic Voltage and Frequency Scaling, DVFS）[54] 主要面向任务执行阶段，通过降低处理器运行频率和供电电压来减少动态功耗，是现代异构 SoC 中实现性能与能耗平衡的重要技术。根据 CMOS 电路功耗规律，动态功耗与供电电压的平方以及工作频率近似成正比。当处理器运行频率降低时，通常可以同步降低供电电压，从而进一步减少动态能耗。因此，DVFS 本质上是一种以执行时间换取能耗下降的优化方式[22]。电压和频率是影响处理器功耗的两个关键因素。这也是动态电压频率调节技术广泛用于节能控制的重要原因[55]。DVFS 的基本思想是在满足性能需求的前提下，适当降低处理器频率，并同步降低供电电压，从而减少动态功耗，并在一定条件下降低总能耗。相关研究表明，DVFS 已广泛应用于通用处理器、图形处理器以及边缘智能设备中的 AI 加速器，是当前最常用的运行时能耗管理机制之一。[56-58]

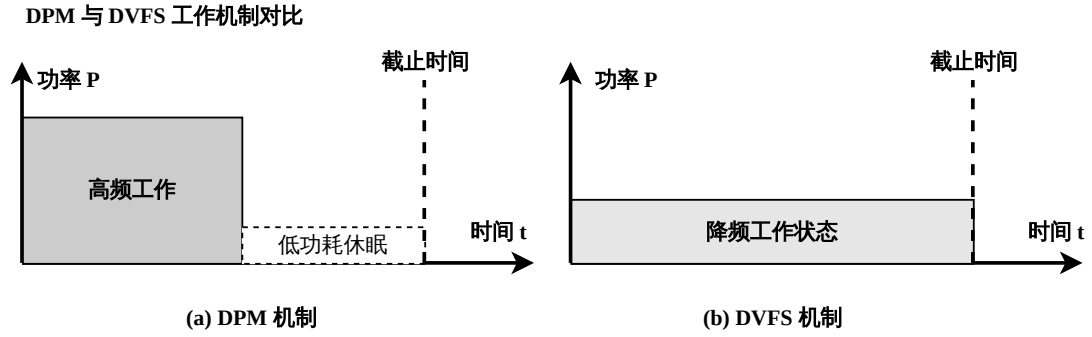


图 2-3 DPM 与 DVFS 工作机制对比

Fig. 2-3 Comparison of DPM and DVFS mechanisms

在异构多核系统中，DVFS 的应用通常依赖于多个独立的电压频率岛（Voltage-Frequency Islands, VFI）。例如，高性能 CPU 簇、高能效 CPU 簇以及 GPU 往往具有各自独立的调频调压域。调度算法可根据任务的截止时间要求和执行时间特性，评估任务可利用的时间余量，并选择满足时序要求的最低运行性能点（Operating

Performance Point, OPP），从而在保证任务按时完成的前提下降低能量开销。

图 2-3 直观展示了 DPM 与 DVFS 的工作机制差异。其中，DPM 通过在任务执行间隙将处理器切换到低功耗状态来降低空闲功耗，适用于存在可利用空闲区间的场景；而 DVFS 则是在任务执行过程中主动降低处理器运行频率和电压，使其以较低功率完成任务。本文在后续研究中采用 DVFS 作为主要能耗管理机制，并在满足任务截止时间约束的前提下，通过联合优化处理单元频率选择、任务映射与执行顺序来降低系统总能耗。

2.5 任务调度算法

面向异构多核系统的任务调度与映射问题通常属于 NP 困难问题（Nondeterministic Polynomial-time hard, NP-hard）问题。为了在可接受的时间内获得高质量解，学术界和工业界提出了多种求解策略。在深度学习推理场景下，常见方法主要包括基于列表的启发式算法、元启发式进化算法以及近年来发展的基于深度强化学习的调度方法。

2.5.1 异构最早完成时间算法

异构最早完成时间算法（Heterogeneous Earliest Finish Time, HEFT）是经典的静态列表调度方法之一，其基本思想是通过任务优先级排序与处理器选择相结合，逐步构造可行调度结果[59]。

HEFT 的基本流程可分为两个阶段。在任务排序阶段，算法根据任务的平均计算开销及其后继路径计算开销，递归计算每个任务 v_i 的上行秩（upward rank）：

$$\mathrm{rank}_u(v_i) = w_i + \max_{v_j \in \mathrm{succ}(v_i)} \mathrm{rank}_u(v_j) \tag{2-16}$$

式中，w_i 表示任务 v_i 在所有处理器上的平均执行时间；$\mathrm{succ}(v_i)$ 表示任务 v_i 的后继任务集合。

在处理器选择阶段，算法按照上行秩从高到低依次调度任务，并将当前任务分配到最早完成该任务的处理器上。对于任务 v_i 和处理器 p_j，其最早完成时间定义为

$$\mathrm{EFT}(v_i, p_j) = \max\left\{\mathrm{Avail}(p_j), \max_{v_k \in \mathrm{pred}(v_i)} \mathrm{AFT}(v_k)\right\} + w_{i,j} \tag{2-17}$$

式中，$\mathrm{Avail}(p_j)$ 表示处理器 p_j 的当前可用时间；$\mathrm{pred}(v_i)$ 表示任务 v_i 的前驱任务集合；$\mathrm{AFT}(v_k)$ 表示前驱任务 v_k 的实际完成时间；$w_{i,j}$ 表示任务 v_i 在处理器 p_j 上的执行时间。

HEFT 的优点在于计算复杂度较低、实现简单，能够快速生成满足依赖关系

的可行调度结果，因此适合作为异构调度问题中的启发式基线方法。然而，经典 HEFT 主要以最早完成时间作为处理器选择准则，没有考虑全局能耗优化、离散 DVFS 配置以及严格截止时间约束。基于这点，本文在后续实验中以经典 HEFT 为基础构建能耗感知扩展版本 EA-HEFT，作为列表调度类对比方法。

2.5.2 元启发式遗传算法

为避免贪心调度方法容易陷入局部最优，遗传算法（Genetic Algorithm, GA）等元启发式方法被广泛用于有截止时间约束与能耗联合优化问题。在本文的相关工作对比中，重点基于进化搜索的遗传算法方法[18]。该方法将任务分配与频率选择编码为染色体，通过种群迭代搜索获得较优调度结果，属于典型的离线全局搜索型启发式方法。

具体而言，设任务图包含 N 个节点，则染色体可表示为长度为 N 的编码序列，其中每个基因位对应一个二元组 (p, f)，分别表示任务分配的处理单元及其运行频率。算法从初始种群出发，通过选择、交叉和变异等操作不断生成新个体，并利用适应度函数评估个体优劣。为同时考虑时延与能耗目标，适应度函数通常采用加权求和形式：

$$F(x) = \alpha \frac{T(x)}{D} + \beta \frac{E(x)}{E_0} + \gamma P(x) \tag{2-18}$$

式中，$T(x)$ 和 $E(x)$ 分别表示个体 x 对应调度方案的总时延与总能耗；D 表示截止时间基准，E_0 表示参考能耗预算；$P(x)$ 表示违反时限约束时的惩罚项。

遗传算法是一类典型的元启发式搜索方法，常用于任务映射、处理器选择和频率配置等离散优化问题。其基本流程包括染色体编码、种群初始化、适应度评价、选择、交叉和变异等步骤。对于能耗感知异构调度问题，染色体通常可用于表示任务到处理单元的映射关系，或进一步包含频率选择信息；适应度函数则可由能耗、时延和约束违反惩罚项共同构成。本文后续实验参考该类方法构建 EA-GA 对比算法。

2.5.3 基于数学规划的全局优化方法

不同于主要依靠预设规则或搜索策略的启发式方法，基于混合整数线性规划（Mixed-Integer Linear Programming, MILP）的全局优化方法是一类典型的精确优化方法[60]。其基本思想是将调度问题中的离散决策和连续决策统一表示为数学规划模型，其中，任务分配、频率选择和执行顺序等决策可用整数或 0–1 变量描述，而开始时间、完成时间和能耗等量则可用连续变量表示。在此基础上，将优化目标表示为线性目标函数，并将任务依赖、处理单元执行约束、截止时间约束等条件统一

写成线性等式或不等式，最终借助数学规划求解器获得满足约束条件的最优解。

对于本文所研究的 CPU/GPU/NPU 异构平台，MILP 方法的优势在于能够将任务依赖关系、处理单元执行约束、截止时间约束以及频率相关能耗统一纳入同一优化框架，并在统一目标下对任务分配、频率选择和执行时序进行联合求解。这样不仅能够避免分阶段决策可能带来的局部最优问题，还能够在满足硬约束的前提下，以系统期望总能耗最小化为目标，对端侧异构推理任务调度问题进行准确描述。因此，MILP 不仅是一种建模工具，也为后续全局优化方法的设计提供了理论基础。

2.6 本章小结

本章围绕本文后续研究所需的系统建模与相关理论基础展开介绍。在系统模型方面，本章介绍了实时系统的基本概念，并结合本文研究对象，给出了包含 CPU、GPU 和 NPU 的端侧异构多核处理平台。围绕该平台，进一步说明了不同类型处理单元的结构特点、执行能力及其在异构多核系统中的作用，为后续面向端侧异构平台的任务建模与调度优化提供了系统基础。在任务模型方面，本章从经典周期任务模型和有依赖关系的图任务模型出发，进一步引出面向深度学习应用的 Stage 级任务图表示方法。在多核系统能耗管理技术方面，本章介绍了处理器功耗的基本组成及其与电压、频率之间的关系，并对 DPM 与 DVFS 两类典型能耗管理技术进行了分析。结合本文研究目标，明确后续将采用 DVFS 作为主要能耗控制手段，在满足任务截止时间约束的前提下降低系统运行能耗。在任务调度算法方面，本章对异构最早完成时间算法、元启发式遗传算法以及基于数学规划的全局优化方法进行了介绍，并分析了不同类型方法在异构任务调度问题中的适用特点。上述内容为后续章节中联合优化模型的建立、求解方法的设计以及对比算法的选取提供了理论依据。

第 3 章 面向深度学习应用的高能效任务调度联合优化方法

3.1 本章引言

针对现有异构多核平台研究尚未将 NPU 处理单元统一纳入建模框架，以及面向端侧异构平台中多应用推理任务在不同运行场景下缺乏联合能效优化算法的问题，面向深度学习应用，针对现有研究尚未将 NPU 处理单元统一纳入异构多核平台建模，以及多场景运行条件下高能效联合调度方法不足的问题，提出了一种面向 CPU-GPU-NPU 异构平台的高能效任务调度优化算法。

本章结构如下：第 3.2 节建立系统模型，包括任务与场景建模、处理器模型和统一功耗模型；第 3.3 节给出问题定义，明确优化目标及其约束条件；第 3.4 节对联合优化问题进行分析并给出形式化表达；第 3.5 节介绍基于延迟约束生成的混合整数线性规划方法 LCG-MILP 求解流程；第 3.6 节给出仿真结果与分析。

3.2 系统模型

本章研究针对通用异构平台，提出一种面向 CPU-GPU-NPU 异构平台的高能效任务调度优化算法，算法使用的平台模型包括任务模型、处理器模型以及功耗模型，本小节将对采用的模型逐一介绍。

3.2.1 任务与场景化建模

采用第二章介绍的 Stage 级 DAG 作为后续研究的基础任务模型。设端侧系统中深度学习应用集合为 $\mathcal{A} = \{1, 2, \ldots, A\}$。对于每个应用 $a \in \mathcal{A}$，其单次推理作业采用 Stage 级 DAG 表示为

$$G_a = (V_a, E_a) \tag{3-1}$$

式中，V_a 表示应用 a 所包含的 Stage 集合；E_a 表示 Stage 之间的前驱后继依赖关系。

在实际端侧系统中，推理运行状态并不是始终保持不变。应用开启与关闭、输入复杂度变化、分辨率或帧率切换等因素，都会使系统在运行过程中呈现出不同的运行状态。若仅针对单一固定运行状态建立优化模型，则难以反映系统长期运行中的实际情况。为此，进一步引入场景化建模方法，用有限个典型场景描述系统在不同运行状态下的任务组合。

$$\Theta^{(s)} = \left(A^{(s)}, \{D_a^{(s)}\}, \{\tau_{j,p}(f)\}\right) \qquad s \in \mathcal{S} \tag{3-2}$$

式中，$A^{(s)}$ 表示场景 s 下的激活应用集合；$D_a^{(s)}$ 表示应用 a 在场景 s 下的相对截止时间；$\tau_{j,p}(f)$ 表示 Stage j 在处理单元 p、频点 f 下的执行时间；此外，场景 s 还可关联权重 π_s，用于表示该场景的出现概率或时间占比，从而支撑后续期望能耗最小化目标的要求。

需要说明的是，本文中的场景表示端侧系统在长期运行过程中可能出现的有限种稳定运行状态，不同场景主要由激活应用集合 $A^{(s)}$ 进行区分，即不同场景下同时运行的深度学习应用数量和组合不同。因此，场景变化会进一步导致任务集合 $J^{(s)}$、依赖边集合 $E^{(s)}$、处理单元竞争关系以及系统能耗发生变化。此外，不同场景也可以通过应用截止时间 $D_a^{(s)}$ 的变化表示不同运行压力。

对于场景 s 中任一激活应用 $a \in A^{(s)}$，其推理任务仍沿用上述 Stage 级 DAG 模型。记场景 s 中所有激活应用对应的 Stage 实例集合为 $J^{(s)}$，依赖边集合为 $E^{(s)}$，其中

$$J^{(s)} = \{(a,i) \mid a \in A^{(s)},\ i \in V_a\} \qquad E^{(s)} = \bigcup_{a \in A^{(s)}} E_a \tag{3-3}$$

式中，$J^{(s)}$ 表示场景 s 中需要参与联合映射与调度的 Stage；$E^{(s)}$ 表示这些 Stage 之间继承自各应用内部任务图的依赖关系。

3.2.2 处理器模型

本章使用的处理器模型基于以下几个重要前提：

（1）端侧异构平台由处理单元集合 $\mathcal{P}$ 构成，可按类型划分为 CPU、GPU 和 NPU 三类，即

$$\mathcal{P} = \mathcal{P}^{\mathrm{cpu}} \cup \mathcal{P}^{\mathrm{gpu}} \cup \mathcal{P}^{\mathrm{npu}} \tag{3-4}$$

在执行机制上，CPU 支持任务抢占，GPU 和 NPU 则采用非抢占执行方式。

（2）在频率配置方面，每个处理单元 p 对应一个离散频点集合

$$\mathcal{F}_p = \{f_{p,1}, f_{p,2}, \ldots, f_{p,|\mathcal{F}_p|}\} \tag{3-5}$$

在任一场景下，每个处理单元仅能从其离散频点集合 $\mathcal{F}_p$ 中选择一个频点执行任务。

（3）由于不同类型处理单元的执行方式不同，在后续建模中对其资源约束进行统一描述。其中，CPU 的资源约束主要体现为并行执行能力，GPU 和 NPU 的资源约束主要体现为独占使用要求。

3.3 功耗模型

本节面向 CPU–GPU–NPU 异构平台建立统一的功耗模型，用于后续能耗计算与优化目标构造。具体而言，本节从频率、电压、功耗与执行时间之间的关系出发，描述异构处理单元的功耗特性，并给出任务计算能耗与系统总能耗的统一表达。需要说明的是，相关频率表达仅用于描述频率变化对功耗和执行时间的影响，而后续优化仍基于离散频点集合 $\mathcal{F}_p$ 展开[18, 22, 24]。

3.3.1 功耗分解与状态模型

对任意处理单元 $p \in \mathcal{P}$，其瞬时功耗通常可分解为静态功耗和动态功耗，即

$$P(t) = P^{sta}(t) + P^{dyn}(t) \tag{3-6}$$

式中，$P(t)$ 表示处理单元在时刻 t 的总功耗，$P^{sta}(t)$ 和 $P^{dyn}(t)$ 分别表示静态功耗和动态功耗。

在 DVFS 作用下，电压和频率的变化会同时影响这两部分功耗，因此工程上通常通过离线测量或性能计数器回归，获得不同频点下的功耗参数[4, 22]。

为支撑后续优化，进一步采用工作态（active）和空闲态（idle）对处理单元功耗进行统一描述：

$$P(t) = \begin{cases} P^{act}(f), & \text{当处理单元处于工作态时,} \\ P^{idl}(f), & \text{当处理单元处于空闲态时,} \end{cases} \qquad f \in \mathcal{F}_p \tag{3-7}$$

式中，$P^{act}(f)$ 和 $P^{idl}(f)$ 分别表示处理单元在频点 f 下的工作态功耗和空闲态功耗；$\mathcal{F}_p$ 表示处理单元 p 的离散频点集合。

3.3.2 CPU 功耗模型

对于 CPU 类处理单元 $p \in \mathcal{P}_{\mathrm{CPU}}$，其动态功耗通常与电压、频率和开关活动强度有关。根据经典 CMOS 功耗模型，动态功耗可近似表示为

$$P_{p,\mathrm{dyn}}(f) \approx \alpha_p C_p V_p^2(f) f, \tag{3-8}$$

式中，$\alpha_p C_p$ 表示处理单元 p 的等效开关电容参数，$V_p(f)$ 表示频率 f 对应的工作电压。

在实际 DVFS 系统中，电压与频率之间通常存在平台相关的非线性关系。为

便于描述，可采用如下经验形式近似表示：

$$V_p(f) \approx \nu_{p,0} + \nu_{p,1} f, \tag{3-9}$$

式中，$\nu_{p,0}$ 和 $\nu_{p,1}$ 为与具体处理单元和平台相关的拟合参数。

需要说明的是，将上述电压–频率关系代入 CMOS 动态功耗模型后，功耗关于频率的表达通常不是单一幂函数形式；同时，实际处理器功耗还受到漏电功耗、温度、负载特征和平台电源管理策略等因素影响。因此，本文不直接将物理功耗模型展开用于后续优化，而是采用基于离散频点测量或回归得到的参数化功耗模型。对于 CPU 类处理单元，其在频率 f 下的参考工作态功耗表示为

$$P_{p,\mathrm{act}}(f) = P_{p,\mathrm{idl}}(f) + a_{p,\mathrm{CPU}} f^{\gamma_{p,\mathrm{CPU}}}, \tag{3-10}$$

式中，$P_{p,\mathrm{idl}}(f)$ 表示处理单元 p 在频点 f 下的空闲态功耗，$a_{p,\mathrm{CPU}}$ 和 $\gamma_{p,\mathrm{CPU}}$ 为由离线测量或回归得到的 CPU 功耗拟合参数。该表达用于刻画 CPU 在不同 DVFS 频点下由空闲态到参考工作态的功耗增量。

3.3.3 GPU 功耗模型

对 GPU 类处理单元 $p \in \mathcal{P}_{\mathrm{GPU}}$，其功耗不仅与频率和电压有关，还受到 kernel 并行度、线程占用率、访存强度以及内存带宽压力等因素影响。在深度学习推理场景中，不同 Stage 对 GPU 计算单元和存储子系统的利用程度存在差异，从而导致实际执行功耗随任务负载变化而波动。因此，近年来不少工作从测量与回归角度建立 DVFS 下的 GPU 时延/功耗关系，用于能耗优化与鲁棒 DVFS 配置[4]。

在本文的系统级建模中，为保持后续优化模型的可解性，并与 CPU、NPU 等处理单元的离散频点建模方式保持一致，本文首先将 GPU 在频点 f 下的参考工作态功耗表示为处理单元级参数：

$$P_{p,\mathrm{act}}(f) = P_{p,\mathrm{idl}}(f) + a_{p,\mathrm{GPU}} f^{\gamma_{p,\mathrm{GPU}}}, \tag{3-11}$$

式中，$P_{p,\mathrm{idl}}(f)$ 表示 GPU 处理单元 p 在频点 f 下的空闲态功耗，$a_{p,\mathrm{GPU}}$ 和 $\gamma_{p,\mathrm{GPU}}$ 为由平台测量或回归得到的 GPU 功耗拟合参数。

为描述不同 Stage 在 GPU 上执行时的负载差异，本文在统一能耗模型中引入等效利用率 $\rho_{j,p}$。当 Stage j 在处理单元 p 上以频点 f 执行时，其平均执行功耗表示为

$$\bar{P}_{j,p}(f) = P_{p,\mathrm{idl}}(f) + \rho_{j,p} \left(P_{p,\mathrm{act}}(f) - P_{p,\mathrm{idl}}(f) \right), \tag{3-12}$$

式中，$\rho_{j,p} \in [0,1]$ 表示 Stage j 在 GPU 处理单元 p 上执行时的等效利用率。

3.3.4 NPU 功耗模型

与 CPU 和 GPU 相比，NPU 面向张量与卷积等规则计算构建专用数据通路，其功耗除受频率和电压影响外，还与当前运行状态的活动强度相关。已有 AI 加速器功耗建模工作通常从动态功耗和静态功耗分解出发，同时考虑频率 f 与对应电压 $V(f)$ 的影响[61]。本文沿用这一建模思路，并采用参数化形式描述 NPU 功耗。

在该近似下，NPU 的动态功耗可写为

$$P^{dyn}(f,\eta) = \alpha\eta f V^2(f) \tag{3-13}$$

式中，α 为动态功耗系数；η 表示当前 Stage 在 NPU 上执行时的等效利用率；$V(f)$ 表示频率 f 对应的工作电压。

考虑到 NPU 在保持当前工作点时仍存在片上存储、控制逻辑和时钟网络等基线开销，其静态功耗可表示为

$$P^{sta}(f) = \beta f V^2(f) + \theta V(f) \tag{3-14}$$

式中，β 和 θ 为与平台相关的常数。

于是，NPU 的工作态功耗可进一步表示为

$$P^{act}(f,\eta) = (\beta + \alpha\eta) f V^2(f) + \theta V(f) \tag{3-15}$$

相应地，当 NPU 的空闲功耗可近似写为

$$P^{idl}(f) = P^{sta}(f) \tag{3-16}$$

3.3.5 统一能耗表达与离散参数化

前述 CPU、GPU 与 NPU 功耗模型描述了频率变化对功耗和执行时间的影响。为便于后续联合优化建模，本文基于各处理单元的离散频点集合 $\mathcal{F}_p$，对执行时间、空闲功耗、工作功耗及能耗参数进行统一表示，从而将频率选择纳入数学规划模型。

对任意处理单元 $p \in \mathcal{P}$ 及频点 $f \in \mathcal{F}_p$，分别用 $P_p^{idl}(f)$ 和 $P_p^{act}(f)$ 表示其空闲功耗和工作功耗，上述功耗参数均为离散频点集合上的已知输入。进一步地，对任意 Stage j，记其在处理单元 p 上的等效利用率为 $\rho_{j,p} \in [0,1]$，则其在配置 (p,f) 下的工作功耗可表示为

$$\bar{P}_{j,p}(f) = P_p^{idl}(f) + \rho_{j,p}\left(P_p^{act}(f) - P_p^{idl}(f)\right) \tag{3-17}$$

若 Stage j 在处理单元 p 上以频点 $f \in \mathcal{F}_p$ 执行，且其执行时间为 $\tau_{j,p}(f)$，则对应的计算能耗为

$$E_{j,p}(f) = \bar{P}_{j,p}(f)\,\tau_{j,p}(f) \tag{3-18}$$

在系统级分析中，若处理单元 p 在分析窗口 $[0, H]$ 内选择稳态频点 f_p，并记 $u_p(t) \in [0, \kappa_p]$ 为时刻 t 的占用量，则其瞬时功率可写为

$$P_p(t) = \kappa_p P_p^{idl}(f_p) + \Big(P_p^{act}(f_p) - P_p^{idl}(f_p)\Big)u_p(t) \tag{3-19}$$

对分析窗口积分，可得处理单元 p 的总能耗为

$$E_p = \kappa_p P_p^{idl}(f_p)H + \Big(P_p^{act}(f_p) - P_p^{idl}(f_p)\Big)B_p \tag{3-20}$$

其中

$$B_p \triangleq \int_0^H u_p(t)\,dt \tag{3-21}$$

表示处理单元 p 在分析窗口内的累计执行时间。

因此，系统总能耗既可以表示为各处理单元在分析窗口内能耗之和，也可以等价表示为各 Stage 计算能耗与各处理单元空闲能耗之和。若记系统总能耗为 E_{sys}，则有

$$E_{\mathrm{sys}} = \sum_j E_j + \sum_{p\in\mathcal{P}} E_{p,\mathrm{idl}} \tag{3-22}$$

3.4 问题定义

基于前文所建立的系统模型、场景化任务表示以及频率相关能耗模型，本文研究多场景端侧系统中的联合能效优化问题。

在 CPU–GPU–NPU 异构平台中，不同处理单元在计算能力和执行方式上存在差异，任务分配结果、频率选择和调度顺序共同影响任务执行时间与系统总能耗。同时，该问题还受到任务依赖关系、处理单元执行约束以及应用截止时间要求的共同限制。因此，本文将该问题定义为：在多场景条件下，联合确定任务分配结果、各场景下的频率选择以及任务执行时序，在满足各项约束的前提下，使系统期望总能耗最小。

具体而言，该优化问题需要满足以下条件：

（1）确定跨场景共享的任务分配结果，即为每个 Stage 选择且仅选择一个可行处理单元；

（2）确定各场景下的频率选择结果，即为每个处理单元在每个场景中选择唯一的工作频点；

（3）确定各场景下的任务执行时序，即给出各任务的开始时刻与完成时刻；

（4）任务执行过程需满足 DAG 的依赖关系；

（5）任务执行结果需满足各应用在对应场景下的相对截止时间要求；

（6）在满足上述约束条件的前提下，使系统在多场景分布下的期望总能耗最小。

3.5 问题分析与形式化

第3.4节已经给出了优化问题的目标及其需要满足的系统约束。由于任务分配、频率选择和调度顺序之间具有较强关联，本节按照任务分配约束、截止时间约束、处理单元执行约束和目标函数与优化问题定义的顺序，引入相应的求解变量，并将系统需求转化为数学约束。

3.5.1 任务分配约束

（1）任务分配约束：每个应用中的 Stage 在所有场景中都固定分配到同一个处理单元上。为此，引入二元变量 $x_{a,i,p} \in \{0,1\}$，当 $x_{a,i,p} = 1$ 时，表示应用 a 中的 Stage i 分配到处理单元 p 上执行，否则为 0。由于每个 Stage 只能分配到其可行处理单元集合 $P_{a,i}$ 中的一个处理单元上，因此有

$$\sum_{p \in P_{a,i}} x_{a,i,p} = 1, \qquad \forall a \in \mathcal{A},\ \forall i \in V_a \tag{3-23}$$

（2）频率选择约束：为适应不同场景下的运行需求变化，系统允许各处理单元在不同场景中选择不同的运行频点。为此，引入二元变量 $z_{p,f}^{(s)} \in \{0,1\}$，当 $z_{p,f}^{(s)} = 1$ 时，表示处理单元 p 在场景 s 下选择频点 f 运行，否则为 0。由于每个处理单元在任一场景下只能选择一个频点，因此有

$$\sum_{f \in \mathcal{F}_p} z_{p,f}^{(s)} = 1, \qquad \forall s \in \mathcal{S},\ \forall p \in \mathcal{P} \tag{3-24}$$

（3）联合配置约束：Stage 的执行时间和能耗同时受到任务分配结果以及处理单元频率选择的影响。为此，引入二元变量 $w_{j,p,f}^{(s)} \in \{0,1\}$，当 $w_{j,p,f}^{(s)} = 1$ 时，表示 Stage j 在场景 s 下由处理单元 p 以频点 f 执行。该变量用于同时描述任务分配结果与场景内频率选择，并满足

$$\sum_{f \in \mathcal{F}_p} w_{j,p,f}^{(s)} = x_{j,p}, \qquad \forall s \in \mathcal{S},\ \forall j \in J^{(s)},\ \forall p \in P_j \tag{3-25}$$

公式 (3-25) 表示：若 Stage j 分配到处理单元 p，则其在任一场景下只能对应一个

执行频点；

$$w_{j,p,f}^{(s)} \le z_{p,f}^{(s)}, \qquad \forall s \in \mathcal{S},\ \forall j \in J^{(s)},\ \forall p \in P_j,\ \forall f \in \mathcal{F}_p \tag{3-26}$$

公式 (3-26) 表示：Stage j 的执行频点与处理单元 p 在该场景下选择的频点一致。

3.5.2 截止时间约束

在确定任务分配和频率选择后，还需要进一步安排各任务的执行时间，以保证任务之间的先后顺序满足依赖关系，并满足应用的截止时间要求。

为此，引入连续变量 $S_j^{(s)} \ge 0$ 和 $F_j^{(s)} \ge 0$，分别表示场景 s 下 Stage j 的开始时刻和完成时刻。本文以场景 s 的分析窗口起点作为统一时间参考点，记为 0，因此所有 Stage 的开始时刻、完成时刻以及应用的相对截止时间 $D_a^{(s)}$ 均相对于该时间参考点定义。

（1）执行时间约束：Stage 的完成时刻等于其开始时刻与执行时长之和。由于 Stage 的执行时长由其分配的处理单元及所选频点共同决定，因此有

$$F_j^{(s)} = S_j^{(s)} + \sum_{p \in P_j} \sum_{f \in \mathcal{F}_p} w_{j,p,f}^{(s)}\, \tau_{j,p}(f), \qquad \forall s \in \mathcal{S},\ \forall j \in J^{(s)} \tag{3-27}$$

（2）依赖关系约束：对于存在依赖关系的两个 Stage，后继 Stage 只能在前驱 Stage 完成后开始执行，即

$$S_k^{(s)} \ge F_j^{(s)}, \qquad \forall s \in \mathcal{S},\ \forall (j,k) \in E^{(s)} \tag{3-28}$$

（3）截止时间约束：对于应用 a 在场景 s 下的出口 Stage，其完成时刻不能超过该应用在场景 s 下的相对截止时间。记应用 a 在场景 s 下对应的出口 Stage 实例集合为 $X_a^{(s)} \subseteq J^{(s)}$，则有

$$F_j^{(s)} \le D_a^{(s)}, \qquad \forall s \in \mathcal{S},\ \forall a \in A^{(s)},\ \forall j \in X_a^{(s)} \tag{3-29}$$

3.5.3 处理单元执行约束

在一般 MILP 调度建模中，这类约束通常通过任务对之间的先后关系来表示。具体做法是：对于任意可能分配到同一处理单元上的两个任务，引入比较变量或排序变量，显式决定二者的执行先后，并利用互斥约束保证它们在时间上不重叠。

然而，本文所研究的异构平台并不完全符合这一建模前提。如图 3-1 所示，GPU 与 NPU 更接近非抢占式处理资源，同一时刻通常只允许一个任务连续执行；而 CPU 则具有并发执行能力，允许多个任务同时运行，并且支持任务抢占。因此，不同类

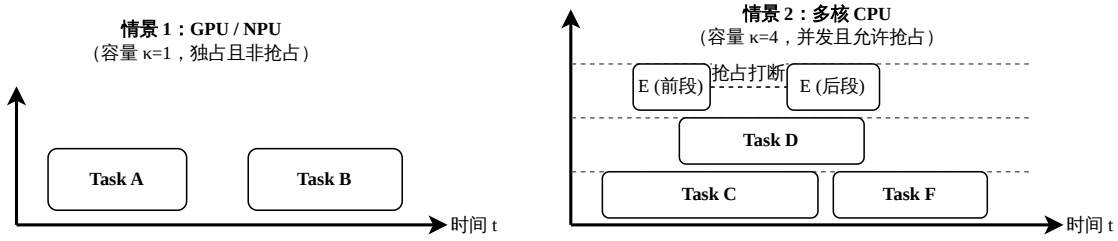


图 3-1 异构处理器抢占机制不同

Fig. 3-1 Different preemption mechanisms of heterogeneous processors

型处理单元在执行方式上存在差异：GPU 和 NPU 适合用互斥关系描述，CPU 则更适合用并发容量约束描述。如果仍采用基于任务对排序的建模方法，一方面需要为大量任务对引入额外变量和约束，模型规模会随任务数量迅速增长；另一方面，这种方法更适合描述两两不重叠，不好描述 CPU 上多任务并发执行时的容量限制。

基于上述分析，本文不再从任务对之间的依赖关系出发描述处理单元执行约束，而是采用时间窗口约束统一描述不同处理单元上的执行需求。其基本思想如图 3-2 所示：对于任意观察窗口 $\omega = [t_1, t_2]$，分配给处理单元 p 的累计执行需求不能超过该处理单元在该窗口内能够提供的最大执行能力，即 $\kappa_p|\omega|$。其中，κ_p 表示处理单元 p 的执行能力，$|\omega|$ 表示窗口长度。这样，GPU 和 NPU 的独占执行可以看作 $\kappa_p = 1$ 的特例，而 CPU 的并发执行能力也可以在同一框架下统一表示。

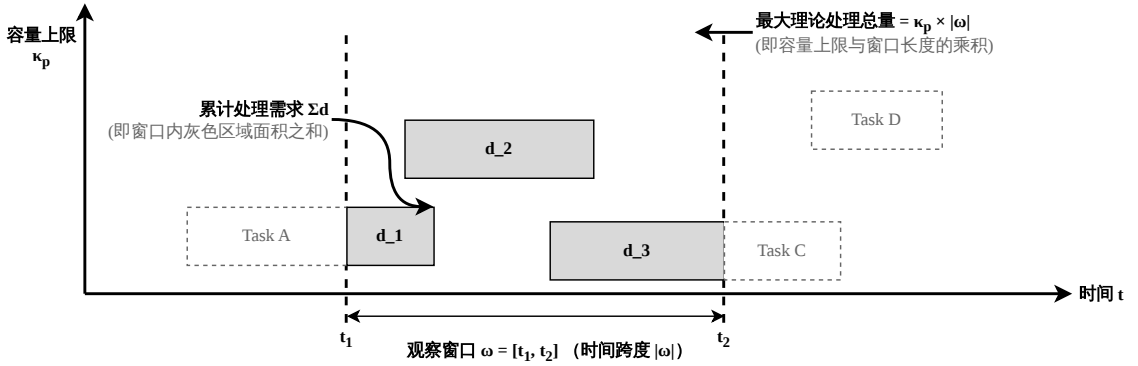


图 3-2 引入窗口约束

Fig. 3-2 Introduction of Window Constraints

进一步地，本文仅考虑由任务开始时刻与完成时刻确定的候选观察窗口，记场景 s 下的窗口集合为

$$W^{(s)} = \left\{ [S_u^{(s)}, F_v^{(s)}] \mid u, v \in J^{(s)},\ S_u^{(s)} < F_v^{(s)} \right\} \tag{3-30}$$

对于任意窗口 $\omega = [t_1, t_2] \in W^{(s)}$，引入连续变量 $d_{\omega,j,p}^{(s)}$，表示场景 s 下 Stage j

在窗口 ω 内对处理单元 p 的执行需求。若 Stage j 未分配到处理单元 p，或其执行区间与窗口 ω 不重叠，则该值为 0；否则，该值由 Stage 的执行区间与窗口的重叠长度确定。

于是，处理单元执行约束可写为

$$\sum_{j\in J^{(s)}} d^{(s)}_{\omega,j,p} \le \kappa_p|\omega|, \qquad \forall s\in\mathcal{S},\ \forall p\in\mathcal{P},\ \forall \omega\in W^{(s)} \tag{3-31}$$

公式 (3-31) 表示：在任一观察窗口内，分配给处理单元 p 的总执行需求不能超过该处理单元在该窗口内的最大执行能力。

3.5.4 目标函数与优化问题定义

在满足任务分配、截止时间和处理单元执行约束的前提下，本文进一步以系统总能耗最小为优化目标。

对于场景 s，定义处理单元 p 在频点 f 下由任务执行产生的累计执行时间为

$$B^{(s)}_{p,f} = \sum_{j\in J^{(s)}} w^{(s)}_{j,p,f}\,\rho_{j,p}\,\tau_{j,p}(f) \tag{3-32}$$

据此，场景 s 下的系统总能耗可写为

$$E^{(s)} = \sum_{p\in\mathcal{P}}\sum_{f\in\mathcal{F}_p} z^{(s)}_{p,f}\,\kappa_p\,P^{idl}_p(f)\,H^{(s)} + \sum_{p\in\mathcal{P}}\sum_{f\in\mathcal{F}_p}\Big(P^{act}_p(f)-P^{idl}_p(f)\Big)B^{(s)}_{p,f} \tag{3-33}$$

其中，第一项表示场景窗口 $H^{(s)}$ 内处理单元的空闲能耗，第二项表示任务运行产生的活动能耗。

进一步结合各场景的概率权重 π_s，其中 $\pi_s\ge 0$ 且 $\sum_{s\in\mathcal{S}}\pi_s=1$，则联合优化目标为最小化系统在所有场景下的期望总能耗，即

$$\min \sum_{s\in\mathcal{S}} \pi_s E^{(s)} \tag{3-34}$$

综合前文给出的任务分配约束、频率选择约束、截止时间约束以及处理单元执行约束，面向 CPU–GPU–NPU 异构平台的多场景联合能效优化问题可形式化为如下紧凑混合整数线性规划模型：

$$\textbf{P1:}\quad \min_{x,z,w,S,F} \sum_{s\in\mathcal{S}} \pi_s E^{(s)} \tag{3-35}$$

$$\text{s.t.}\quad (x,z,w,S,F)\in \Omega_{\text{map}}\cap\Omega_{\text{freq}}\cap\Omega_{\text{time}}\cap\Omega_{\text{exec}} \tag{3-36}$$

$$x_{a,i,p},\ z_{p,f}^{(s)},\ w_{j,p,f}^{(s)} \in \{0,1\} \qquad S_j^{(s)},\ F_j^{(s)} \in \mathbb{R}_+ \tag{3-37}$$

式中，Ω_{map}、Ω_{freq}、Ω_{time} 和 Ω_{exec} 分别表示前文定义的任务分配约束、频率选择与联合配置约束、时序与截止时间约束以及处理单元执行约束。

3.6 LCG-MILP 求解流程

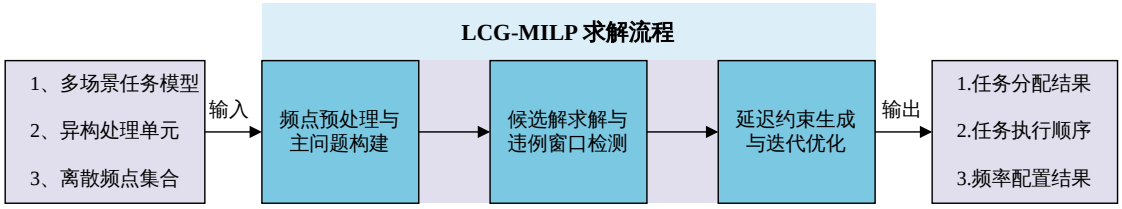


图 3-3 LCG-MILP 算法框图

Fig. 3-3 Block diagram of the LCG-MILP algorithm

如图 3-3 所示，LCG-MILP 的求解流程包括三个阶段。首先，依据多场景任务模型、异构处理单元和离散频点集合等输入信息，完成频点预处理与主问题构建。其次，求解当前主问题，获得候选解对应的任务分配、执行顺序和频率配置结果，并基于该候选解检查是否存在违反处理单元执行约束的窗口。最后，若存在违例窗口，则生成相应的延迟约束并将其加入主问题，继续迭代求解；若不存在违例窗口，则说明当前解已满足全部约束条件，算法终止并输出最终优化结果。

前文已经建立了多场景联合能效优化模型 **P1**。该模型以系统期望总能耗最小为目标，并同时满足任务依赖关系、截止时间要求和处理单元执行约束。

由于处理单元执行约束采用时间窗口形式描述，直接展开会引入大量辅助变量和线性化约束，从而导致模型规模迅速增大，影响求解效率。为此，本文采用延迟约束生成机制，对 **P1** 进行求解。

其基本思路是：先求解主问题，再检查当前候选解是否违反处理单元执行约束；若存在违例窗口，则将相应约束加入主问题并重新求解；如此反复迭代，直至得到满足全部约束的优化解。

本节将进一步介绍基于 LCG-MILP 的具体求解流程。

3.6.1 LCG-MILP 的引入

直接求解模型 **P1** 的主要困难在于处理单元执行约束。该约束不是仅对每个处理单元设置一个总量上限，而是要求在任意候选观察窗口内，分配给该处理单元的总执行需求都不能超过其最大执行能力。为描述这一约束，模型中不仅需要引入与窗口相关的变量 $d_{\omega,j,p}^{(s)}$，还需要加入其与 (w,S,F) 之间的线性化关系。因此，真

正导致模型规模增长的并不是基础变量 x、z、w、S 和 F，而是候选窗口集合 $W^{(s)}$ 所对应的大量辅助变量和约束。

如果在建模初始阶段就对每个场景、每个处理单元和每个候选窗口加入完整的处理单元执行约束，那么模型规模会随着 $|\mathcal{S}|$、$|\mathcal{P}|$ 和 $|W^{(s)}|$ 的增大迅速增长。随着 Stage 数量增加，由开始时刻和完成时刻形成出的候选窗口也会明显增多，模型中将出现大量暂时不起作用的窗口约束。这样虽然能够得到完整模型，但也会显著增加构模、LP 松弛和分支定界的求解开销。

基于此，本文不在初始主问题中一次性加入全部窗口约束，而是采用延迟约束生成机制。主问题先保留任务分配、频率选择、执行时间、依赖关系和截止时间等核心约束，并只关注当前已激活的窗口集合 Ω。在得到候选解后，再检查是否存在违反处理单元执行约束的窗口；若存在未激活窗口 $\omega \notin \Omega$ 不满足约束，则将该窗口对应的约束及其线性化关系加入主问题，再继续求解。通过这种方式，可以在保证模型完整性的同时控制求解规模。

3.6.2 基于延迟约束生成的求解流程

按照上述思路，LCG-MILP 的求解过程可分为初始化、主问题求解、违例检测和约束增广四个环节，整体流程如图3-4 所示。首先，读入场景集合、处理单元集合、离散频点集合，以及前文已经确定的执行时间参数、功耗参数和截止时间参数。随后，对各处理单元的离散频点集合进行预处理，删除被统一支配的频点，得到精简后的候选频点集。在此基础上，初始化激活窗口集合 $\Omega = \varnothing$，并建立当前主问题 $P1(\Omega)$。此时，主问题保留目标函数和基础约束，但尚不加入未激活窗口对应的处理单元执行约束及其线性化关系。

在给定当前窗口集合 Ω 后，调用 MILP 求解器求解主问题 $P1(\Omega)$。若主问题无解，则说明即使不考虑尚未加入的窗口约束，现有任务分配、频率选择、依赖关系、执行时间和截止时间条件也无法同时满足，此时可直接判定原问题不可行。若主问题存在可行解，则得到一个候选解，其中包括跨场景共享的任务分配结果、各场景下的频率选择结果，以及对应的开始时刻和完成时刻。

得到候选解后，需要进一步检查其是否满足完整模型中的处理单元执行约束。具体而言，对每个场景 s 和每个处理单元 p，根据当前候选解中的时序变量形成相应的候选窗口集合 $W^{(s)}(\hat{S}, \hat{F})$，并计算各窗口上的执行需求是否超过上限。若对所有被检查窗口均满足

$$\sum_{j \in J^{(s)}} d^{(s)}_{\omega,j,p} \le \kappa_p |\omega|, \tag{3-38}$$

则说明当前候选解已经满足完整模型中的处理单元执行约束，此时迭代结束，当前

解即为所求结果。若存在违反上述约束的窗口，则说明当前候选解尚未满足完整模型，需要继续向主问题中加入新的窗口约束。

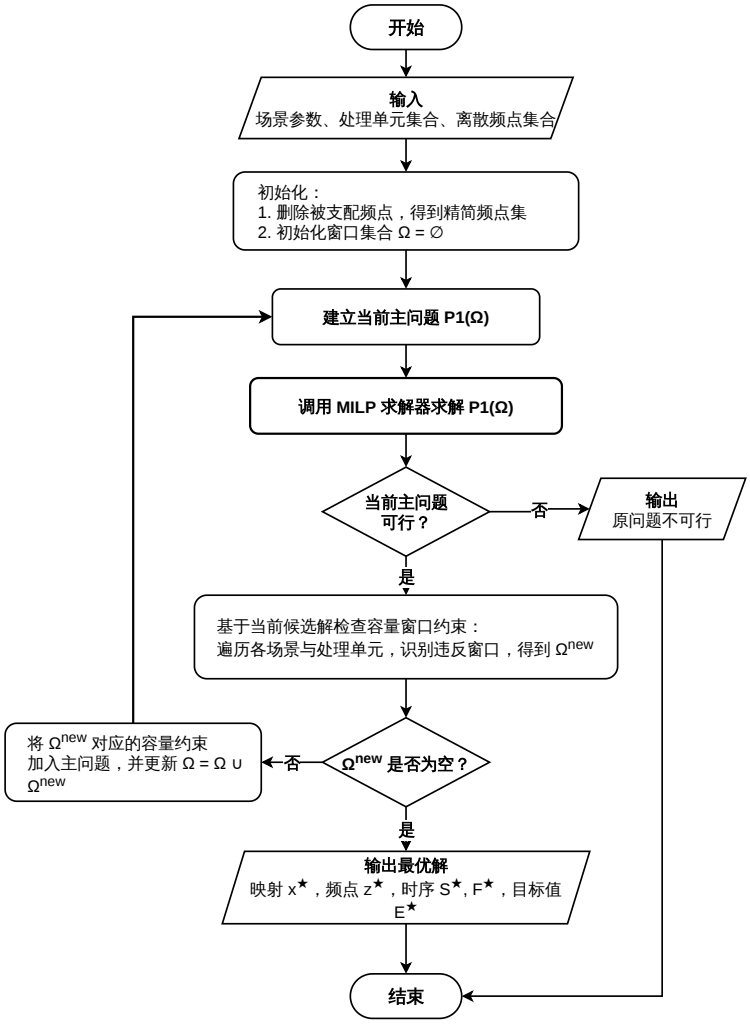


图 3-4 MILP 算法流程图

Fig. 3-4 Flowchart of the MILP algorithm

约束增广过程围绕违例窗口展开。对于每个场景 s 和每个处理单元 p，若检测到多个违例窗口，则从中选取违反程度最大的窗口加入新增窗口集合 $\boldsymbol{\Omega}^{\mathrm{new}}$。这样可以避免一次性加入过多约束，在每轮增广中都集中处理最主要的违例窗口。完成检测后，若 $\boldsymbol{\Omega}^{\mathrm{new}} \neq \varnothing$，则将其中各窗口对应的处理单元执行约束及线性化约束族 $L^{(s)}_{\omega,j,p}(d,w,S,F)$ 一并加入主问题，同时更新

$$\boldsymbol{\Omega} \leftarrow \boldsymbol{\Omega} \cup \boldsymbol{\Omega}^{\mathrm{new}}. \tag{3-39}$$

随后重新求解增广后的主问题，并重复上述过程。随着迭代推进，集合 $\boldsymbol{\Omega}$ 不断扩展，主问题逐步逼近完整模型；当某一轮检测中不再出现新的违例窗口时，算法终

止。

基于以上过程，LCG-MILP 的具体步骤可归纳为算法 3-1。其核心思想是在 MILP 主问题的基础上，对处理单元执行约束按需加入，从而在保证模型完整性的同时控制求解规模。

算法 3-1　基于延迟约束生成的离线 LCG-MILP 求解流程

输入：场景参数元组 $\{\Theta^s\}_{s\in\mathcal{S}}$；处理单元集合 $\mathcal{P}$；离散频点集合 $\{\mathcal{F}_p\}_{p\in\mathcal{P}}$。

输出：最优任务分配结果 $x^\star$；各场景最优频率选择结果 $\{z^{s\star}\}$；各场景最优时序 $\{S^{s\star}, F^{s\star}\}$；期望总能耗最优值 $E^\star$。

1: 对每个处理单元 p 删除被统一支配的频点，得到精简后的频点集合 $\widetilde{\mathcal{F}}_p$；

2: 初始化激活窗口集合 $\Omega \leftarrow \varnothing$；

3: 基于 $\widetilde{\mathcal{F}}_p$ 建立主问题 **P1**(Ω)：保留目标函数（3-33）及第 3.5 节中的基础约束，仅加入 Ω 中对应的处理单元执行约束；

4: **repeat**

5:　调用 MILP 求解器求解当前主问题 **P1**(Ω)；

6:　**if** 当前主问题无解 **then** 返回原问题不可行；

7:　提取当前候选解 $(\hat{x}, \hat{z}, \hat{w}, \hat{S}, \hat{F})$；

8:　初始化新增窗口集合 $\Omega^{\text{new}} \leftarrow \varnothing$；

9:　**for** 每个场景 $s \in \mathcal{S}$ 与每个处理单元 $p \in \mathcal{P}$ **do**

10:　　根据 $(\hat{S}, \hat{F})$ 形成候选窗口集合 $W^{(s)}(\hat{S}, \hat{F})$；

11:　　计算各窗口上的违反量 $\Delta^{(s)}_{\omega,p}$；

12:　　**if** 存在 $\Delta^{(s)}_{\omega,p} > 0$ 的窗口 **then**

13:　　　取违反程度最大的窗口 $\omega^\star_{s,p}$，并加入 Ω^{new}；

14:　　**end if**

15:　**end for**

16:　**if** $\Omega^{\text{new}} \neq \varnothing$ **then**

17:　　将 Ω^{new} 中各窗口对应的处理单元执行约束及线性化约束族 $L^{(s)}_{\omega,j,p}(d, w, S, F)$ 加入主问题；

18:　　更新 $\Omega \leftarrow \Omega \cup \Omega^{\text{new}}$；

19:　**end if**

20: **until** $\Omega^{\text{new}} = \varnothing$

21: 返回当前候选解对应的 $x^\star, z^{s\star}, S^{s\star}, F^{s\star}$ 及 $E^\star$。

图 3-4 与算法 3-1 给出了该求解流程的整体结构。其中，主问题求解用于在当

前激活窗口集合下产生候选解，违例检测用于识别尚未加入模型的违例窗口，约束增广则将相应窗口约束补充回主问题。通过这一过程，前文以紧凑形式给出的模型 **P1** 得以逐步扩展并完成求解。其核心思想是：不在初始阶段展开全部窗口约束，而是围绕当前候选解按需加入必要约束，从而在保证模型完整性的同时控制求解规模。

3.7 仿真结果与分析

为评估本章所提出 LCG-MILP 求解方法的有效性，本节基于典型端侧异构平台开展多组对比仿真实验。

考虑到现有经典异构任务调度算法大多以执行时间优化为主要目标，较少同时考虑 NPU 处理单元约束和 DVFS 机制，本文在经典算法基础上构建能耗感知的异构最早完成时间算法（Energy-Aware HEFT，简称 EA-HEFT）和能耗感知遗传算法（Energy-Aware GA，简称 EA-GA）作为对比基准。同时，为验证 DVFS 在降低能耗方面的效果，本文引入未采用 DVFS 的映射优化方法（Mapping-Only）作为求解对照组。

EA-HEFT 以经典 HEFT 算法为基础[59]，原始 HEFT 通过 upward rank 对 DAG 节点进行优先级排序，并在处理器选择阶段选择使当前任务最早完成的处理器。本文保留其任务排序方式，但在处理器选择阶段引入能耗目标和频率选择机制。具体而言，对于待调度 Stage j，EA-HEFT 枚举其可执行处理单元 $p \in P_j$ 以及对应离散频点 $f \in \mathcal{F}_p$，计算该配置下的最早完成时间和执行能耗：

$$EFT(j,p,f) = EST(j,p) + \tau_{j,p}(f) \tag{3-40}$$

$$E_{j,p}(f) = \bar{P}_{j,p}(f)\tau_{j,p}(f) \tag{3-41}$$

其中，$EST(j,p)$ 表示 Stage j 在处理单元 p 上满足前驱依赖和处理单元可用时间后的最早开始时刻，$\tau_{j,p}(f)$ 表示其在处理单元 p、频点 f 下的执行时间，$\bar{P}_{j,p}(f)$ 表示对应的工作功耗。在满足依赖关系、处理单元执行约束和应用截止时间约束的候选配置中，EA-HEFT 优先选择执行能耗最低的 (p,f)；若多个候选配置能耗相同或接近，则选择完成时间更早者。

EA-GA 参考能耗感知场景化映射方法构建[18]。该方法采用遗传算法搜索多个深度学习应用在 CPU/GPU/DLA 异构平台上的映射组合，并在染色体评价过程中结合场景可调度性检查、频率选择和能耗估计。本文在该思想基础上进行适配，将染色体编码为 Stage 的处理单元与频点联合配置。设染色体为 $X = [g_1, g_2, \ldots, g_{N_{\text{stage}}}]$，其中基因 $g_j = (p_j, f_j)$ 表示 Stage j 被分配到处理单元 p_j，并以频点 f_j 执行。染色

体初始化、交叉和变异过程中均需满足 $p_j \in P_j$，即 Stage 只能被分配到其可执行处理单元集合中。对于染色体 X，若其违反 Stage 可执行集合、处理单元执行约束或截止时间约束，则赋予较大惩罚值；若其满足全部约束，则以多场景期望总能耗作为适应度值：

$$Fitness(X) = \sum_{s \in \mathcal{S}} \pi_s E^{(s)}(X) \tag{3-42}$$

需要说明的是，本文所有对比算法均纳入与 LCG-MILP 相同的 NPU 约束。

本节仿真实验均在 MATLAB 平台上完成系统级建模与开发。针对混合整数线性规划问题，底层调用 YALMIP 建模工具，并结合 CPLEX 或 Gurobi 求解器进行求解。实验所用硬件平台为 Windows 11 专业版操作系统，配置 32GB DDR4 内存和 Intel Core i5-12490 处理器（6 核 12 线程）。

为量化评估各类方法在端侧深度学习推理场景下的表现，本节采用以下三个评价指标：

（1）本文采用归一化能耗作为评价指标。设算法 m 在任务规模 N、场景数 $|\mathcal{S}|$ 和资源配置 R 下得到的系统总能耗为 $ETE_m(N;|\mathcal{S}|,R)$。为便于比较不同算法的相对能耗水平，本文将各算法得到的系统总能耗统一除以当前实验配置下的最大能耗，记为 $ETE_{\max}(N;|\mathcal{S}|,R)$。因此，算法 m 的归一化能耗定义为

$$\tilde{E}_m(N;|\mathcal{S}|,R) = \frac{ETE_m(N;|\mathcal{S}|,R)}{ETE_{\max}(N;|\mathcal{S}|,R)}.$$

$\tilde{E}_m \in (0,1]$。其值越小，说明对应算法在当前实验点下的能耗表现越好。

（2）可调度率（Schedulability Ratio, SR）：该指标用于衡量算法在给定任务规模、场景数量和资源配置条件下生成可行调度结果的能力。设算法 α 在任务规模 N、场景数 $|\mathcal{S}|$ 和资源配置 R 下，共测试 M 组随机生成实例，其中成功得到满足截止时间约束和处理单元执行约束的可行解的实例数为 $M_\alpha^{\text{feas}}(N;|\mathcal{S}|,R)$，则其可调度率定义为

$$SR_\alpha(N;|\mathcal{S}|,R) = \frac{M_\alpha^{\text{feas}}(N;|\mathcal{S}|,R)}{M}.$$

$SR_\alpha \in [0,1]$。其值越大，说明对应方法在当前任务规模和资源条件下越容易得到可行调度结果。

（3）相对求解时间（Solving Time, ST）：该指标记录算法从读入场景参数和任务图，到输出最终任务分配结果与频率选择结果所需的实际运行时间（Wall-clock Time，单位：s）。该指标主要用于衡量不同方法的求解效率，以及在任务规模增大时的可扩展性。

3.7.1 仿真参数设置与实验设计

本节仿真实验面向 CPU–GPU–NPU 异构平台开展，实验对象为多场景条件下的 Stage 级 DAG 推理任务。表 3-2 给出了本节采用的仿真参数与控制变量设置。

在任务生成方面，本文首先生成应用级 DAG 任务图，再将其转换为可调度的 Stage 级依赖图。后续实验中，任务规模采用场景内全部应用经过转换后的总 Stage 数量进行表示。

在平台设置方面，所有处理单元均支持独立的离散 DVFS 调节，并统一采用

$$\mathcal{F}_p = \{0.6, 0.7, 0.8, 0.9, 1.0\} f_p^{\max}.$$

在截止时间设置方面，本文通过参考截止时间系数构造不同松紧程度的应用截止时间[59, 62]。由于本文实验中的 Stage 级 DAG 任务由随机方式生成，缺乏真实的端到端截止时间，若直接采用固定绝对截止时间，则不同任务规模和不同资源配置下的时间约束强度难以保持一致。与此同时，若采用本文所提算法自身的调度结果来生成基准截止时间，则会使实验输入依赖于被评价算法本身。因此，本文采用独立的经典列表调度方法生成参考完成时间，并在此基础上构造应用截止时间。

具体而言，对于场景 s 中的应用 a，令所有处理单元均运行于最高频点 $f_p^{\max}$，并采用最早完成时间优先的列表调度方法生成参考调度方案。记该参考调度中应用 a 的出口 Stage 完成时间为 $C_{a,\mathrm{ref}}^{(s)}$，则将其作为应用 a 在场景 s 下的基准完成时间，并定义基准截止时间为

$$D_{a,0}^{(s)} = C_{a,\mathrm{ref}}^{(s)}.$$

在此基础上，引入截止时间系数 β，并将实际截止时间设置为

$$D_a^{(s)}(\beta) = \beta D_{a,0}^{(s)}.$$

其中，β 用于控制截止时间约束的松紧程度。已有的研究通常通过参考完成时间或估计完成时间乘以截止时间系数的方式生成不同约束强度的截止时间，用于评估调度算法在不同剩余时间条件下的性能表现[62]。若某一调度结果中任一应用出口 Stage 的完成时间超过对应的 $D_a^{(s)}(\beta)$，则该实例被判定为不可调度。本文通过改变 β，进一步分析不同算法在严格截止期和宽松截止期下的可调度能力与能耗优化能力。

除表 3-2 所列核心变量外，Stage 在最高频点下的基准执行时间和活动系数均采用随机方式生成，并在各组实验中保持相同的统计分布。为减弱随机样本带来的偶然性，每个实验点均独立生成 50 组实例并取平均值；除特别说明外，各场景权重统一取 $\pi_s = 1/|\mathcal{S}|$。

本文基础实验中统一设置 $\pi_s = 1/|\mathcal{S}|$，表示在缺乏实际运行日志或状态转移概率时，假设系统在各场景中的驻留时间相同，并以场景平均能耗作为优化目标。若实际系统能够获得各场景的出现概率或驻留时间，则可直接将 π_s 设置为非均匀权重，并代入目标函数 $\sum_{s\in\mathcal{S}} \pi_s E^{(s)}$ 进行优化。

表 3-2 仿真参数与控制变量设置

Table 3-2 Simulation parameters and control variable settings

参数名	参数取值范围或设定
基准资源配置	$(\lvert\mathcal{P}^{\mathrm{cpu}}\rvert, \lvert\mathcal{P}^{\mathrm{gpu}}\rvert, \lvert\mathcal{P}^{\mathrm{npu}}\rvert) = (4,1,1)$
异构资源配置	$\{(2,1,1),(4,1,1),(4,2,1),(4,2,2)\}$
场景内总 Stage 数量 N_{stage}	小规模：10 ~ 40；中等规模：50 ~ 100
场景数量 $\lvert\mathcal{S}\rvert$	$\{1,2,3,4,5,6,7,8\}$
离散频点集合 $\mathcal{F}_p$	$\{0.6,0.7,0.8,0.9,1.0\} f_p^{\max}$
截止时间系数 β	$\{1.0,1.1,1.2,1.3,1.4\}$

实验一：本实验用于评估不同场景数量和异构资源配置条件下各算法的能耗表现。评价指标采用归一化能耗。实验分别从场景数量和异构资源配置两个方面进行对比，其中，场景数量取 $\{1,2,3,4,5,6,7,8\}$，异构资源配置取 $\{(2,1,1),(4,1,1),(4,2,1),(4,2,2)\}$。

实验二：本实验用于评估不同任务规模条件下各算法的能耗表现。评价指标采用归一化能耗。实验通过改变场景内 Stage 级任务规模，比较各算法在任务规模逐渐增大时的能耗变化趋势。为便于比较不同算法在各任务规模下的相对表现，将各算法得到的系统总能耗结果进行归一化处理。场景内 Stage 级任务规模取 $\{5,10,15,20,25,30,35,40,45,50\}$，场景数量固定为 4，异构资源配置固定为 $(4,1,1)$。

实验三：本实验用于评估截止时间约束松紧程度对各算法能耗表现和可调度性能的影响。评价指标采用归一化能耗和可调度率。实验通过改变截止时间系数 β，比较不同算法在严格截止期和宽松截止期下获得可行调度结果的能力，以及在截止时间逐渐放宽时利用剩余时间进行降频节能的能力。截止时间系数取 $\{1.0,1.1,1.2,1.3,1.4\}$，实验中场景内 Stage 级任务规模固定为 30，场景数量固定为 4，异构资源配置固定为 $(4,1,1)$。

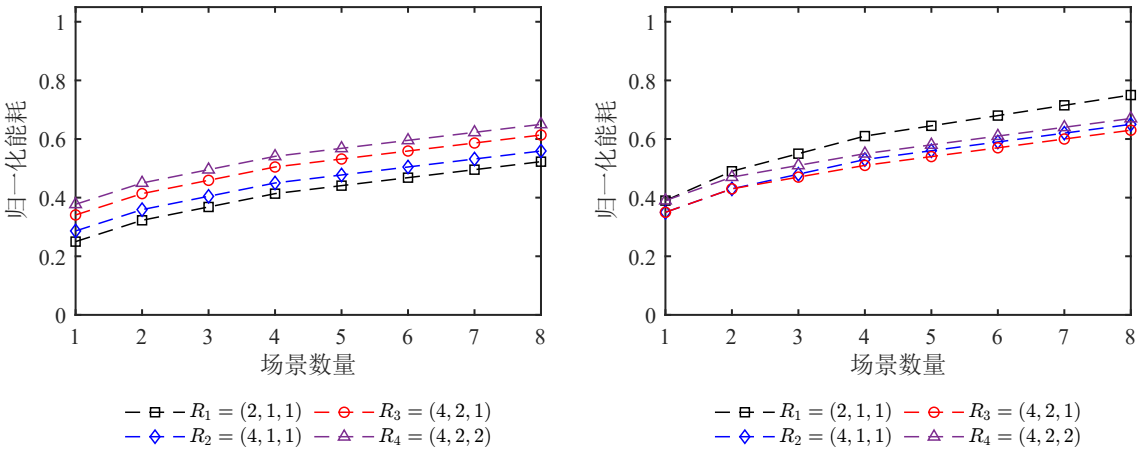
实验四：本实验用于评估不同异构资源配置条件下，各算法的可调度性能随任务规模变化的趋势。评价指标采用可调度率。实验通过改变场景内 Stage 级任务规模，比较不同资源配置下各算法获得可行调度结果的能力。场景内 Stage 级任务规

模取 $\{5, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$，异构资源配置取 $\{(2,1,1), (4,1,1), (4,2,1), (4,2,2)\}$，场景数量固定为 4。

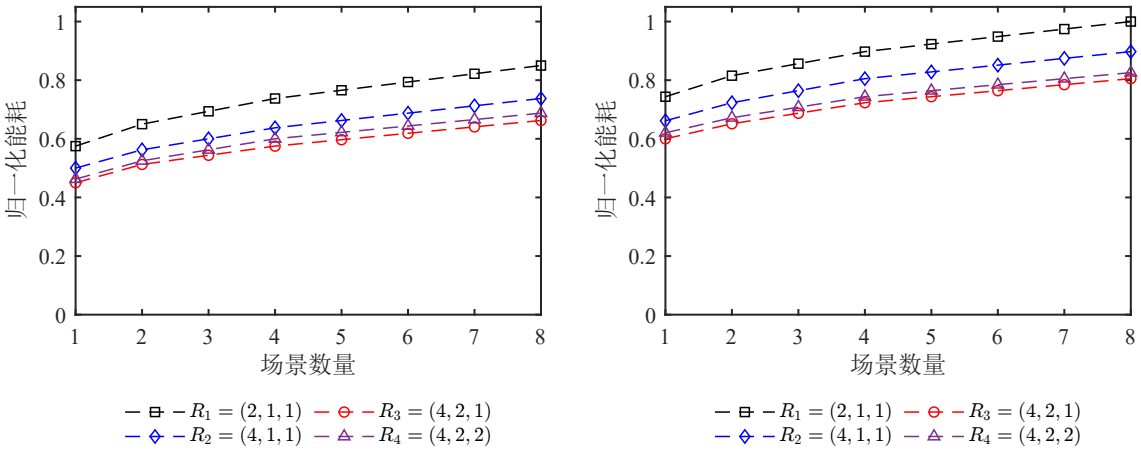
实验五：本实验用于评估不同算法的求解时间。评价指标采用求解时间 ST。实验通过改变场景内 Stage 级任务规模，比较各算法在任务规模逐渐增大时的求解时间变化。场景内 Stage 级任务规模取 $\{10, 15, 20, 25, 30, 35, 40\}$，场景数量固定为 4，异构资源配置固定为 $(4,1,1)$。

3.7.2 仿真结果分析

（1）实验一的结果如图 3-5 所示。四幅子图分别对应 Stage 级任务规模为 5、15、30 和 50 时，不同场景数量与异构资源配置下系统归一化能耗的变化情况；取值越大，表示对应配置下系统能耗越高。



(a) 任务规模 Stage=5　　(b) 任务规模 Stage=15



(c) 任务规模 Stage=30　　(d) 任务规模 Stage=50

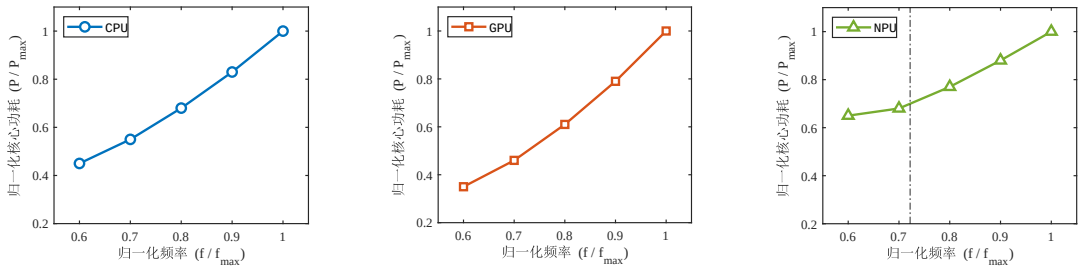
图 3-5 不同配置下的能耗表现

Fig. 3-5 Energy consumption performance under different configurations

从整体趋势来看，在任务规模固定时，随着场景数量由 1 增加到 8，各资源配置下的归一化能耗均呈上升趋势，且任务规模越大，上升幅度越明显。这说明当系统需要同时适配的运行场景增多时，跨场景共享的任务分配、场景内频率选择以及执行顺序之间的关系越复杂，可用于降低运行频率和调整任务执行顺序的时间空间被进一步压缩，从而使系统整体能耗水平持续上升。这一现象在 Stage=30 和 Stage=50 时尤为明显，其曲线增长幅度明显大于 Stage=5 和 Stage=15。

从不同任务规模下各资源配置的相对表现来看，异构资源扩展带来的能效收益跟任务规模数量有很大的关系。当 Stage=5 时，四条曲线整体上满足 $R_1 < R_2 < R_3 < R_4$，即资源配置越丰富，归一化能耗反而越高。这表明在任务规模较小的情况下，现有任务量不足以充分发挥额外 GPU 和 NPU 资源的并行与专用加速优势，反而会引入更多的运行开销，因此单纯扩展异构资源并不能带来节能效果。随着任务规模增大到 Stage=15，各配置之间的差距明显缩小，且从大多数场景数量下的结果来看，$R_3 = (4, 2, 1)$ 的能耗最低，$R_2 = (4, 1, 1)$ 次之，$R_4 = (4, 2, 2)$ 与 $R_1 = (2, 1, 1)$ 相对较高，说明此时增加一定数量的异构资源已经能够改善系统能耗表现，但资源继续扩充后的效果仍不稳定。

进一步在 Stage=30 和 Stage=50 条件下，整体上表现为 R_3 最优、R_4 次之、R_2 再次、R_1 最差。该结果说明，当任务规模进一步增大后，适度增加异构资源能够更有效地分担任务执行压力，并为频率选择和任务调度提供更大空间，从而降低系统总能耗；但当资源规模继续扩展到 R_4 时，额外增加的 NPU 并未带来相匹配的节能增益，说明异构资源配置存在一定的最优区间。



(a) CPU 频率功耗之间的关系　(b) GPU 频率功耗之间的关系　(c) NPU 频率功耗之间的关系

图 3-6 异构处理单元频率与功耗的关系

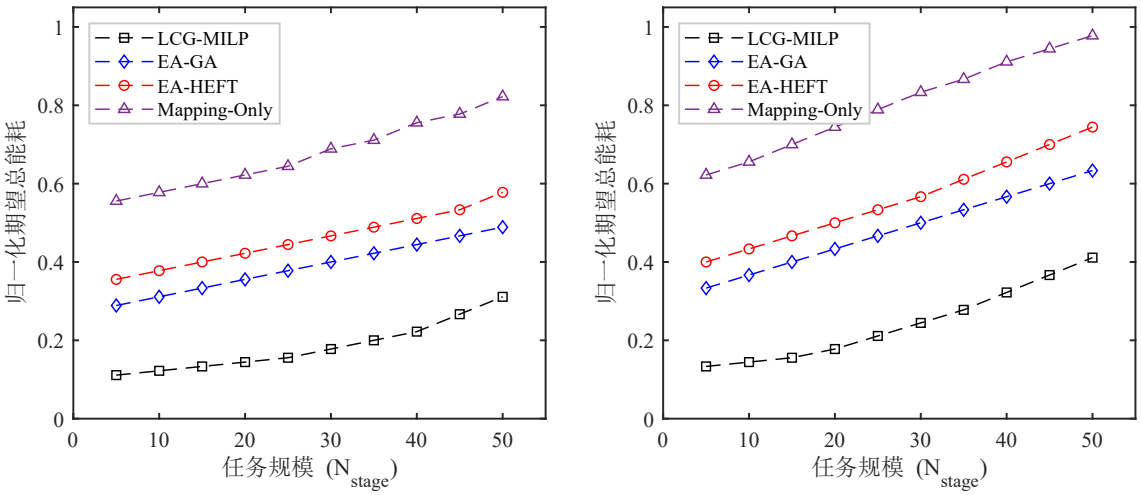
Fig. 3-6 Relationship between frequency and power consumption of heterogeneous processing units

上述现象可以结合图 3-6 所示的频率–功耗关系进行解释。可以看到，CPU、GPU 和 NPU 的归一化功耗都会随归一化频率升高而增加，但三类处理单元的变化趋势并不相同。CPU 的功耗随频率上升持续增加；GPU 在低频段的归一化功耗较低，但在中高频区也会明显上升；NPU 则在低频段已经具有较高的功耗基线，并

在约 1300 MHz 附近出现明显拐点[61]，此后功耗增长进一步加快。由此可见，异构资源扩展能否带来节能收益，并不只取决于资源数量本身，更取决于新增资源能否被有效利用，并进一步转化为更低的运行频点。

对于 Stage=5 这类小规模任务，系统整体计算需求较低，新增 GPU 和 NPU 的并行能力难以充分发挥，随着异构资源增加并未缩短执行时间。因此，新增资源带来的额外功耗无法被频率下降带来的节能收益抵消，最终表现为资源配置越高，系统总能耗反而越高。

当任务规模增大到 Stage=15 及以上时，更多 Stage 可以分散到不同类型的处理单元上执行，系统获得了更多的空闲时间，从而能够在满足约束的前提下选择更低的运行频点，节能效果开始显现。特别是从 R_2 到 R_3 的变化可以看到，增加 GPU 资源后，系统能耗进一步下降，这与图 3-6 中 GPU 在低中频区较优的功耗特性是一致的。相比之下，从 R_3 到 R_4 后，继续增加一个 NPU 并未带来明显改善，且在多数情况下略高于 R_3。这说明当新增 NPU 资源不能转化为足够的并行执行收益时，其较高的低频功耗基线以及拐点后的快速功耗增长，会限制进一步的节能效果。



(a) $|S|=4, R=(4,2,1)$ 配置下的能耗对比　　(b) $|S|=8, R=(4,2,2)$ 配置下的能耗对比

图 3-7 不同配置下不同算法的能耗对比

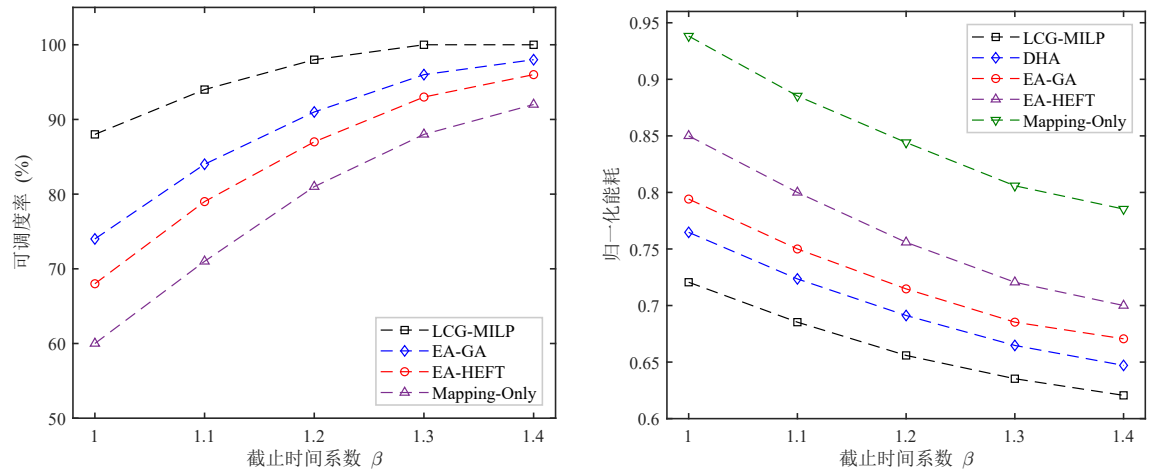
Fig. 3-7 Comparison of energy consumption among different algorithms under different configurations

（2）实验二的结果如图 3-7 所示。整体来看，无论是在 $(|\mathcal{S}|=4,\ R=(4,2,1))$ 还是在 $(|\mathcal{S}|=8,\ R=(4,2,2))$ 的设定下，随着 Stage 级任务规模由 5 增加至 50，四种方法的归一化能耗均呈持续上升趋势。

进一步观察左图 $(|\mathcal{S}|=4,\ R=(4,2,1))$ 可以发现，LCG-MILP 的归一化能耗始终处于最低水平，表明其在任务规模扩大时仍能较好地降低能耗。在小规模任务下，EA-GA、EA-HEFT 和 Mapping-Only 的归一化能耗分别比 LCG-MILP 高约

12%、21% 和 40%；而在最大任务规模附近，这一差距进一步扩大到约 18%、26% 和 43%。这说明在中等复杂度的场景与资源配置下，虽然近似方法同样能够得到可行调度结果，但随着任务规模增大，其能耗增长速度明显快于 LCG-MILP，说明仅启发式算法或局部搜索，难以在全局范围内同时兼顾任务分配、执行顺序和离散频点选择。

右图给出了更复杂设定 ($|\mathcal{S}| = 8,\ R = (4,2,2)$) 下的结果。可以看到，在场景数量和异构资源复杂度进一步提高后，各方法的归一化能耗整体进一步上升，且近似方法与 LCG-MILP 之间的差距更加明显。在该设定下，小规模任务时，EA-GA、EA-HEFT 和 Mapping-Only 相比 LCG-MILP 分别高出约 15%、27% 和 49%；当任务规模增大到 $N_{\text{stage}} = 50$ 时，这一差距进一步扩大到约 22%、34% 和 57%。



(a) 不同截止时间系数下各算法的可调度率比较 (b) 不同截止时间系数下各算法的归一化能耗比较

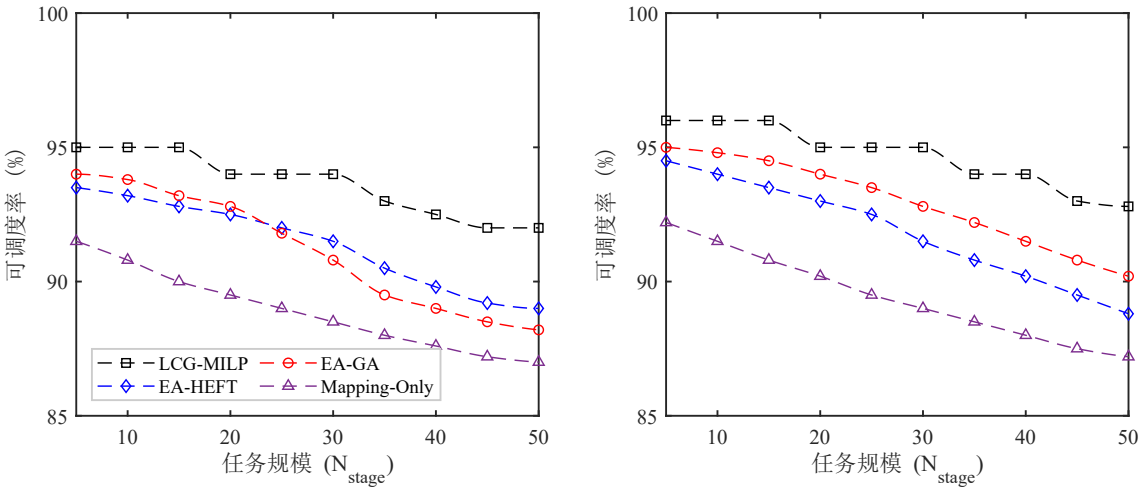
图 3-8 不同截止时间系数下各算法性能对比

Fig. 3-8 Performance comparison of different algorithms under different deadline factors

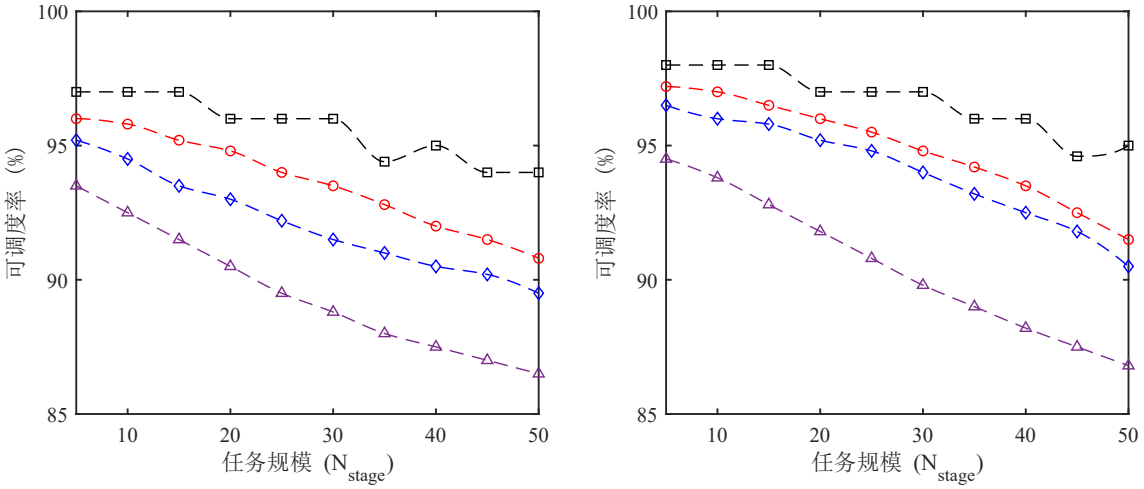
（3）实验三结果如图 3-8 所示。随着截止时间系数 β 增大，应用截止时间约束逐渐放宽，系统可利用的剩余时间增加，因此各算法的可调度率整体提高，归一化能耗整体下降。图 3-8(a) 表明，当 β 从 1.0 增加到 1.4 时，任务依赖、处理单元竞争与截止时间约束之间的冲突减弱，各算法更容易得到可行调度方案。

不同算法之间仍存在明显差距。LCG-MILP 在所有 β 取值下均取得最高可调度率。当 $\beta = 1.0$ 时，其可调度率约为 88%；当 β 增加到 1.3 后，可调度率达到 100%，说明其在严格截止期下仍具有较强的可行解搜索能力。EA-GA 和 EA-HEFT 的可调度率也随 β 增大而提升，但始终低于 LCG-MILP。Mapping-Only 表现最弱，在 $\beta = 1.0$ 时可调度率约为 60%，即使 $\beta = 1.4$ 也仅约为 92%，说明仅进行任务映射难以充分协调资源竞争和截止时间约束。

图 3-8(b) 表明，截止时间放宽后，各算法均可利用更多剩余时间进行降频，从而降低能耗。其中，LCG-MILP 在所有 β 下保持最低归一化能耗，由约 0.70 降至约 0.60。EA-GA 和 EA-HEFT 能耗也有所下降，但整体高于 LCG-MILP。Mapping-Only 能耗始终最高，由约 0.91 降至约 0.76，说明其对剩余时间的利用能力相对有限。



(a) $|S| = 4, R = (2, 1, 1)$　　(b) $|S| = 4, R = (4, 1, 1)$



(c) $|S| = 4, R = (4, 2, 1)$　　(d) $|S| = 4, R = (4, 2, 2)$

图 3-9 不同任务规模下各算法的可调度率比较

Fig. 3-9 Comparison of schedulability among different algorithms under different task scales

（4）实验四的结果如图 3-9 所示。在不同异构资源配置下，随着任务规模不断增大，各算法的可调度率整体均呈下降趋势。这说明随着场景内 Stage 数量增加，处理单元占用、执行顺序和截止时间约束之间的冲突增加，从而更难获得满足全部约束的可行调度结果。同时，不同资源配置下曲线的整体位置存在明显差异：当异构资源由较紧张的 $(2, 1, 1)$ 逐步扩展到较充足的 $(4, 2, 2)$ 时，各算法的可调度率曲线整体上移，且下降速度有所减缓，说明增加 CPU、GPU 和 NPU 资源数量能够

有效扩大系统的可行调度范围。

进一步比较各算法可见，LCG-MILP 在不同任务规模和异构资源配置下均保持最高可调度率，说明其通过联合优化任务分配、执行顺序和频率选择，能够更有效地满足截止时间约束。EA-HEFT 和 EA-GA 受启发式或局部搜索策略影响，在任务规模增大后可调度率下降更明显。Mapping-Only 始终最低，表明仅进行任务映射难以应对复杂资源竞争和时序约束。

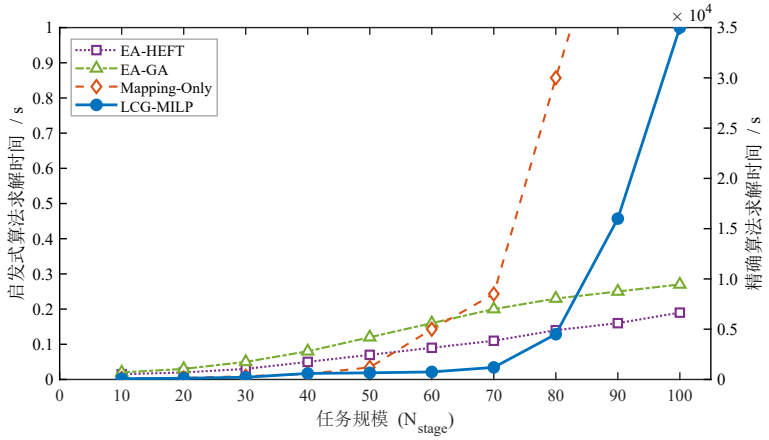


图 3-10 不同算法求解时间随任务规模的变化

Fig. 3-10 Evolution of solving time of different algorithms with task size

（5）实验五的结果如图 3-10 所示。本实验主要考察不同算法的求解时间随任务规模增大的变化趋势，评价指标为求解时间。由于图中采用对数坐标，因此分析重点在于比较各类算法随任务规模增大时的增长速度，以及求解时间开始快速上升的位置。总体来看，随着任务规模逐步扩大，各算法的求解时间均呈上升趋势，但增长速度存在明显差异。

从图中可以看出，EA-HEFT 始终保持最低的求解时间，且增长最为平缓，这与其启发式列表调度方法的特点一致。EA-GA 的求解时间整体高于 EA-HEFT，但仍明显低于两类精确方法，EA-GA 的求解时间整体高于 EA-HEFT，说明种群搜索虽然会带来多轮迭代计算开销，但相较于精确求解仍具有较高的求解效率。

与此同时，LCG-MILP 的求解时间随着任务规模增大明显上升，尤其在大规模区间增长更快。这说明虽然本文提出的方法能够获得高质量解，但其求解时间在任务规模继续扩大时仍然较长，从而其在大规模实例中的应用很难实现。

3.8 本章小结

本章面向端侧 CPU–GPU–NPU 异构平台，研究了多场景条件下的联合能效优化问题，目标是在满足应用截止时间约束和处理单元执行约束的前提下，使系统总能耗最小。围绕这一问题，建立了任务、平台、频率与能耗的统一模型，分析了多

场景共享任务分配、场景内频率选择与执行顺序之间的关联关系，并在此基础上构建了相应的混合整数线性规划模型。

针对处理单元执行约束在时间窗口展开后容易导致模型规模迅速增长的问题，本章进一步引入延迟约束生成机制，构建了基于 LCG 的精确求解流程。该方法不在初始阶段一次性加入全部窗口约束，而是先求解主问题，再对候选解进行违例检测，并将被违反的关键窗口约束按需加入模型中迭代求解，从而在保证精确求解能力的同时有效控制模型规模。基于这一思路，实现了 LCG-MILP 求解方法，并结合 YALMIP 与 CPLEX 完成模型求解。

最后，通过多组对比仿真实验验证了所提方法的有效性。实验结果表明，LCG-MILP 在不同任务规模和场景配置下均能获得最低能耗，并能够更有效地发挥异构处理单元的能效优势。在较复杂配置 ($|\mathcal{S}| = 4,\ \boldsymbol{R} = (4,2,1)$) 和 ($|\mathcal{S}| = 8,\ \boldsymbol{R} = (4,2,2)$) 下，随着任务规模增加，EA-GA 相对于 LCG-MILP 的额外能耗约增加到 $12\% \sim 22\%$，EA-HEFT 的额外能耗约增加到 $21\% \sim 34\%$。这说明，仅依赖启发式任务分配或局部搜索，难以在多场景条件下同时协调任务分配、执行顺序和离散频点选择，而频率联合优化对降低系统总能耗具有重要作用。

然而，随着任务规模和场景数量进一步增大，精确方法的求解时间仍会明显增加，难以满足大规模实例下的快速求解需求。因此，第 4 章将在本章精确建模与求解框架的基础上，进一步研究面向大规模实例的高效启发式调度方法。

第 4 章 面向多场景联合优化问题的分解式启发调度算法

4.1 本章引言

针对大规模多场景条件下端侧异构平台联合能效优化问题精确求解代价较高、难以满足快速求解需求的情况，本章进一步研究计算效率更高的近似求解方法。该问题需要在多场景条件下同时确定跨场景共享的任务分配结果、各场景下的频率选择以及任务执行顺序，并满足任务依赖关系、处理单元执行约束和应用截止时间要求。

基于此，本章提出一种分解式启发算法，记为 DHA（Decomposed Heuristic Algorithm）。该方法不再直接对原始联合优化问题进行整体精确求解，而是按照任务分配与场景内优化之间的关系，将求解过程划分为多个阶段，以降低大规模实例下的求解复杂度。

4.2 系统模型与任务定义

本章沿用第 3 章任务与场景模型、处理器模型、功耗模型、离散频点设定，不再重复定义，下面只介绍补充内容。

4.2.1 任务与场景模型

考虑到本章需要讨论跨场景统一任务分配，这里仅补充两个后文直接使用的记号：

$$\mathcal{J} = \bigcup_{s \in \mathcal{S}} \mathcal{J}_s \qquad \mathcal{S}_j = \{ s \in \mathcal{S} \mid j \in \mathcal{J}_s \} \tag{4-1}$$

式中，$\mathcal{J}$ 表示全部场景上的全局 Stage 集合；$\mathcal{S}_j$ 表示包含 Stage j 的场景集合；前者用于定义跨场景共享的统一任务分配结果，后者用于在不同场景之间汇总同一 Stage 的局部代价信息。

4.2.2 处理器模型

本节只保留一个与场景内列表调度直接相关的辅助记号。对任一处理单元 p，根据其并行处理能力 κ_p，定义并行处理能力单元集合

$$\mathcal{R}_p = \{ r_{p,1}, r_{p,2}, \ldots, r_{p,\kappa_p} \} \tag{4-2}$$

式中，CPU 对应多个可并行使用的处理能力单元；GPU 和 NPU 由于 $\kappa_p = 1$，只对应一个处理能力单元；该集合只用于场景内构造可行执行顺序，不改变第 3 章中定

义的处理单元级映射关系。

当统一任务分配结果给定后，记

$$p(j) = \arg\max_{p \in \mathcal{P}_j} x_{j,p} \qquad r_j^{(s)} \in \mathcal{R}_{p(j)} \tag{4-3}$$

式中，$p(j)$ 表示 Stage j 的处理单元分配位置；$r_j^{(s)}$ 表示其在场景 s 的列表调度过程中占用的具体并行处理能力单元；后者仅是算法执行过程中的辅助量，不属于跨场景共享决策。

4.3 问题分析

由于统一任务分配结果需要在所有场景之间保持一致，而场景内的频率配置和执行顺序只在各自场景中起作用，所以第 3 章中的联合优化问题在本章可以自然分成两部分，一部分是统一任务分配，另一部分是场景内优化。记统一任务分配结果为 x，则长期期望目标写为

$$\min_{x} \sum_{s \in \mathcal{S}} \pi_s \, \phi^{(s)}(x) \tag{4-4}$$

其中

$$\phi^{(s)}(x) = \min_{z,S,F} E^{(s)}(x, z, S, F) \tag{4-5}$$

这里，$\phi^{(s)}(x)$ 表示在统一任务分配结果 x 已经确定后，场景 s 通过频率配置和执行顺序安排所能达到的最小能耗。由于式 (4-5) 只对应单个给定场景，所以其中的配置变量采用局部写法。

先对每个场景在最高频率下做一次预调度，并据此得到 Stage 在不同处理单元上的局部时间代价 $T_j^{(s)}(p)$ 和局部能耗代价 $E_j^{(s)}(p)$。在此基础上，定义局部综合代价为

$$Q_j^{(s)}(p) = \alpha \, T_j^{(s)}(p) + (1 - \alpha) \, E_j^{(s)}(p) \qquad 0 \le \alpha \le 1 \tag{4-6}$$

式中，参数 α 用来平衡局部时间代价和局部能耗代价。

不过，只看局部综合代价，还不能体现不同场景对统一任务分配结果的影响差别，所以进一步定义场景时间紧迫程度

$$\chi^{(s)} = \frac{\hat{M}^{(s)}}{\min_{a \in A^{(s)}} D_a^{(s)}} \tag{4-7}$$

式中，$\hat{M}^{(s)}$ 表示场景 s 在预调度后得到的整体完成时间；$\chi^{(s)}$ 越大，说明这个场景越接近截止时间约束，因此在确定统一任务分配结果时，需要更多考虑它的影响。

基于局部综合代价和场景时间紧张程度，对每个全局 Stage j 及其候选处理单元 $p \in \mathcal{P}_j$，定义加权差值为

$$R_j(p) = \sum_{s \in \mathcal{S}} \pi_s \chi^{(s)} \left(Q_j^{(s)}(p) - \min_{q \in \mathcal{P}_j} Q_j^{(s)}(q) \right) \tag{4-8}$$

这样，Stage j 的初始统一分配位置可以按加权差值最小的原则确定为

$$x_j = \arg\min_{p \in \mathcal{P}_j} R_j(p) \tag{4-9}$$

式中，$R_j(p)$ 表示把全局 Stage j 分配到处理单元 p 时，相比各场景中最合适的局部选择所增加的额外影响；这个量同时考虑了场景概率 π_s 和场景时间紧张程度 $\chi^{(s)}$ 的作用。

需要说明的是，由式 (4-9) 得到的统一任务分配结果，本质上只是一个初始结果，所以在部分场景下仍可能无法满足截止时间约束。基于这一点，当当前统一任务分配结果在一些场景中仍然导致超时时，就需要在候选 Stage 上逐个检查可以替换的处理单元。记某个重新分配方案 $\delta = (j : p \to q)$ 的综合效果为

$$\Omega(\delta) = G_{\text{time}}(\delta) - \lambda G_{\text{cost}}(\delta) \tag{4-10}$$

式中，$G_{\text{time}}(\delta)$ 表示该重新分配对超时场景执行可行性的改善程度；$G_{\text{cost}}(\delta)$ 表示它对其他场景局部代价带来的额外影响；λ 为平衡参数；要是存在 $\Omega(\delta) > 0$ 的方案，就采用综合效果最好的那个方案，并更新统一任务分配结果。

当统一任务分配结果确定后，各场景之间真正需要共同保持一致的，主要是处理单元层面的 Stage 分配结果，而频率配置和执行顺序则可以在各自场景内单独调整。对场景 s，记当前的运行频率向量为

$$f^{(s)} = \left(f_1^{(s)}, f_2^{(s)}, \ldots, f_{|\mathcal{P}|}^{(s)} \right) \qquad f_p^{(s)} \in \mathcal{F}_p \tag{4-11}$$

在给定统一任务分配结果和运行频率向量后，场景 s 的总能耗写为

$$E^{(s)}\left(f^{(s)} \right) = E_{\text{idl}}^{(s)} + E_{\text{act}}^{(s)} \tag{4-12}$$

式中，$E_{\text{idl}}^{(s)}$ 表示场景时间范围内各处理单元在对应运行频率下的静态能耗；$E_{\text{act}}^{(s)}$ 表示各个 Stage 执行时产生的动态能耗；由于统一任务分配结果确定后，各场景的执行顺序和运行频率都可以分别单独调整，所以后面的优化重点就变成了，在不破坏场景截止时间约束的前提下，尽量利用场景内还可以使用的时间来降低能耗。

对任一还没有降到最低频率的处理单元 p，记它下一档可选频率为 $f_p^{\downarrow}$，并用

$f_{p\downarrow}^{(s)}$ 表示只把处理单元 p 的频率降低一档后得到的临时频率向量。要是这次降频满足

$$M_a^{(s)}\left(f_{p\downarrow}^{(s)}\right) \le D_a^{(s)} \qquad \forall a \in A^{(s)} \tag{4-13}$$

那么这次降频带来的节能量定义为

$$\Delta E_p^{(s)} = E^{(s)}\left(f^{(s)}\right) - E^{(s)}\left(f_{p\downarrow}^{(s)}\right) \tag{4-14}$$

在每一轮场景内降频过程中，都从满足式 (4-13) 的处理单元中，选出 $\Delta E_p^{(s)}$ 最大的那个进行降频，并重新计算整张任务图的执行顺序，直到不能再继续降频为止。这样，场景内优化阶段实际上就是在统一任务分配结果已经确定的基础上，通过可行性检查、单步降频和顺序更新这几个步骤，继续挖掘各场景内部还能降低的能耗。

据此，本章近似方法的总体目标可以写为

$$\min \sum_{s\in\mathcal{S}} \pi_s E^{(s)}\left(f^{(s)}\right) \tag{4-15}$$

式 (4-15) 和式 (4-4) 在目标含义上是一致的，只是这里更直接对应了 DHA 在算法实现中的输出形式。由此可以看出，在不改变输出形式和约束条件的前提下，原来的联合优化问题可以通过统一任务分配和场景内优化这两部分来做近似求解。前一部分负责在多个场景之间建立统一且尽量可行的 Stage 分配结果，后一部分负责在各个场景内部继续利用剩余时间来降低能耗，这也构成了后面算法设计和流程展开的直接基础。

（1）为每个 Stage 确定一个跨场景共享且唯一的处理单元分配位置，形成统一任务分配结果；

（2）在统一任务分配结果给定后，为每个场景中的各个处理单元确定唯一的运行频率；

（3）在统一任务分配结果给定后，为各个场景安排满足 DAG 依赖关系和处理单元并行处理能力约束的可行执行顺序；

（4）当部分场景在初始统一任务分配结果下仍不能满足截止时间约束时，对这些超时场景中的部分 Stage 重新分配；

（5）在所有场景都满足截止时间约束后，继续利用场景内还可以使用的时间，进一步降低长期期望总能耗。

基于上述问题形式，本章针对联合优化模型在中大规模场景集合和任务规模下求解复杂度高、计算时间长的问题，提出分解式启发算法 DHA。这个算法主要解决的是，在多场景异构处理平台上，怎样更高效地确定跨场景共享的任务分配结

果，并在这个基础上结合场景内运行频率调整和执行顺序优化，在满足实时性约束的前提下尽量降低系统长期期望总能耗。

4.4 分解式启发算法流程

针对前面对联合优化问题的分析，本文提出一种面向多场景异构处理平台的分解式启发算法 DHA。这个算法不再直接在统一模型下求严格意义上的全局最优结果，而是根据跨场景共享决策和场景内局部决策之间的差别，把原来的联合优化问题拆成几个步骤来求解。

DHA 的整体流程可以分成输入、三步处理和输出三个部分。算法输入包括多场景任务图、处理单元集合和离散频率集合。DHA 的整体流程可归纳为算法 4-1 所示。

4-1 分解式启发算法 DHA

输入：场景参数元组 $\{\Theta^s\}_{s\in\mathcal{S}}$；处理单元集合 $\mathcal{P}$；离散频率集合 $\{\mathcal{F}_p\}_{p\in\mathcal{P}}$；最大重新分配轮数 $I_{\max}$。

输出：统一任务分配结果 x；各场景运行频率配置 $\{z^s\}$；各场景执行顺序 $\{S^s, F^s\}$。

1: 初始跨场景统一任务分配决策；//算法 4-2

2: 超时场景任务再分配；//算法 4-3

3: 场景内降频与执行顺序优化；//算法 4-4

由此可以看出，DHA 的核心做法是在不改变问题输出形式的前提下，把原来的联合优化过程拆成先做跨场景统一任务分配，再做场景内局部优化这两个层次。

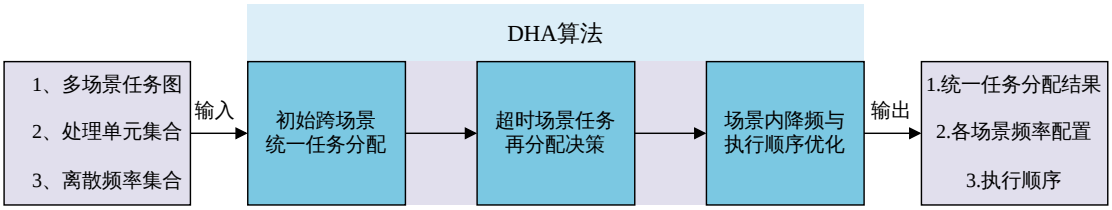


图 4-1 DHA 算法框图

Fig. 4-1 Block diagram of DHA algorithm

如图 4-1 所示，DHA 算法的输入包括多场景任务图、处理单元集合和离散频率集合。基于这些输入，算法首先执行跨场景统一任务分配，得到统一的任务分配结果；随后，针对当前分配下可能出现截止时间违例的场景，进一步进行任务再分配，以提高系统可调度性；最后，在各场景内部完成降频与执行顺序优化，以进一

步挖掘节能空间。经过上述三个阶段后，算法输出统一任务分配结果、各场景频率配置和执行顺序，从而在保持问题输出形式一致的前提下，实现由全局联合求解向跨场景分解与场景内局部优化相结合的求解方式转化。

4.4.1 初始跨场景统一任务分配

在前文已经定义局部时间代价、局部能耗代价、场景时间紧张程度以及加权差值之后，DHA 已具备生成跨场景统一任务分配结果所需的基础信息。基于这些量，本节进一步给出初始跨场景统一任务分配的具体流程。该过程并不是直接在原联合优化模型上求解全局最优解，而是先综合各场景下的局部调度结果，构造一个能够兼顾场景概率、时间紧张程度以及时间—能耗平衡关系的统一初始分配结果，为后续可行性检查、重新分配和场景内优化提供起点。

初始跨场景统一任务分配可以分为以下几个步骤：

（1）对于每个场景 $s \in \mathcal{S}$，DHA 首先在各处理单元均工作于最高频率的条件下执行一次场景级预调度。这样做的目的，是在暂不考虑频率调整的前提下，先估计当前场景中各个 Stage 分配到不同候选处理单元时的局部执行表现。基于预调度结果，可以得到 Stage j 在场景 s 下分配到处理单元 p 时的局部时间代价 $T_j^{(s)}(p)$ 和局部能耗代价 $E_j^{(s)}(p)$。进一步根据式 (4-6)，计算局部综合代价

$$Q_j^{(s)}(p) = \alpha T_j^{(s)}(p) + (1-\alpha) E_j^{(s)}(p).$$

式中，参数 $\alpha \in [0,1]$ 用于调节时间代价和能耗代价在初始统一分配阶段的相对权重；当 α 较大时，算法更偏向选择时延表现较好的处理单元；当 α 较小时，则更加关注能耗表现；通过引入 $Q_j^{(s)}(p)$，DHA 可以在单个场景内部形成统一的候选分配代价标准。

（2）仅依靠局部综合代价 $Q_j^{(s)}(p)$，还不足以直接确定跨场景共享的统一任务分配结果。这是因为，不同场景对统一分配方案的敏感程度并不相同：有些场景在最高频率下仍有较大的时间余量，而有些场景的完成时间已经接近截止约束。如果对所有场景的要求一致，就容易使初始分配结果偏向平均意义下的局部较优，而忽略对截止时间要求更严格的场景。因此，在完成场景级预调度后，DHA 还需要根据该场景的整体完成时间 $\hat{M}^{(s)}$，结合式 (4-7) 计算场景时间紧张程度 $\chi^{(s)}$。该指标反映了当前场景调度结果与截止时间约束之间的接近程度。$\chi^{(s)}$ 越大，说明该场景可调整空间越小，对统一任务分配结果越敏感，因此在后续决策中应给予更高关注。

（3）在得到局部综合代价 $Q_j^{(s)}(p)$ 和场景时间紧张程度 $\chi^{(s)}$ 后，DHA 对每个全局 Stage $j \in \mathcal{J}$ 及其候选处理单元 $p \in \mathcal{P}_j$，按照式 (4-8) 计算加权差值 $R_j(p)$。从含

义上看，$R_j(p)$ 衡量的是：如果要求全局 Stage j 在所有场景中统一分配到处理单元 p，相较于各场景分别选择自身最优处理单元，会额外带来多大的综合代价。该量同时考虑了场景出现概率和场景时间紧张程度，因此能够反映某个候选处理单元作为跨场景统一分配位置时的整体适合程度。$R_j(p)$ 越小，说明该候选位置在不同场景之间引入的综合损失越小，也就越适合作为 Stage j 的初始统一分配结果。

（4）在上述计算基础上，DHA 按照式 (4-9)，为每个全局 Stage 选择对应的处理单元，从而生成跨场景共享的初始任务分配结果 x。需要说明的是，该初始结果并不保证所有场景在一开始就满足截止时间约束。但由于它在生成过程中已经综合考虑了局部时间代价、局部能耗代价、场景概率以及场景时间紧张程度，因此通常能够作为质量较高的统一分配起点，并有助于减少后续可行性调整阶段所需调整的范围。该阶段的处理流程如图 4-2 所示，对应伪代码见算法 4-2。

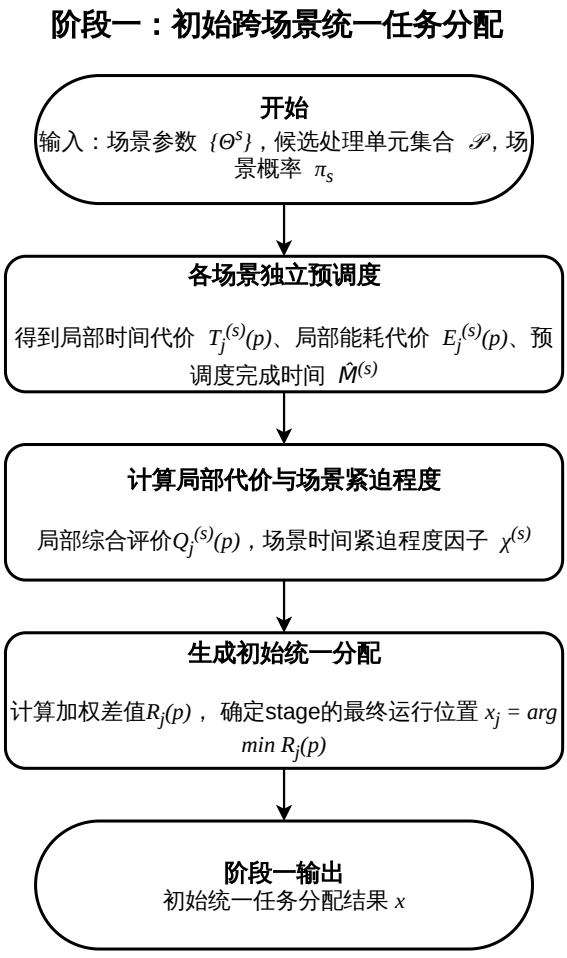


图 4-2 初始跨场景统一任务分配流程图

Fig. 4-2 Flowchart of initial cross-scenario unified task allocation

综上，初始跨场景统一任务分配阶段先利用各场景的局部预调度结果，得到统

一任务分配的初始结果。这个结果在考虑时间要求和能耗表现的同时，也为后面的超时场景再分配和场景内频率优化提供了统一的任务分配基础。

算法 4-2　初始跨场景统一任务分配算法

输入：各场景任务图，候选处理单元集合 $\mathcal{P}$，场景权重 π_s。

输出：初始统一任务分配结果 x。

1: **for** 每个场景 $s \in \mathcal{S}$ **do**

2:　在最高频率下执行场景级预调度；

3:　按式(4-6)计算各 Stage 在各处理单元上的局部综合代价 $Q_j^{(s)}(p)$；

4:　根据预调度得到的整体完成时间 $\hat{M}^{(s)}$，按式(4-7)计算场景时间紧张程度 $\chi^{(s)}$；

5: **end for**

6: **for** 每个全局 Stage $j \in \mathcal{J}$ **do**

7:　按式(4-8)计算各候选处理单元对应的加权差值 $R_j(p)$；

8:　按式(4-9)确定 Stage j 的初始统一分配位置；

9: **end for**

10: 输出初始统一任务分配结果 x。

4.4.2 超时场景任务再分配

经过初始跨场景统一任务分配后，DHA 已经得到一个跨场景共享的初始任务分配结果 x。该结果给出了所有全局 Stage 在不同场景中统一采用的处理单元分配位置。不过，这一结果主要依据前一阶段定义来生成，并不直接保证所有场景都满足全局截止时间约束。因此，本小节以上一阶段得到的统一任务分配结果 x 为输入，进一步检查各场景在最高频率下的可行性，并对仍然超时的场景执行局部重新分配。

超时场景任务再分配可以分为以下几个步骤：

（1）在得到初始统一任务分配结果 x 后，DHA 首先在各处理单元最高频率下，对所有场景在当前映射 x 下的执行结果进行重新检查，以判断其是否满足截止时间约束。之所以需要这一步，是因为上一阶段得到的 x 本质上只是一个跨场景共享的初始结果。虽然该结果在形成时已经综合考虑了局部时间代价、局部能耗代价、场景概率和场景时间紧张程度，但并没有将所有场景的全局截止时间可行性直接作为硬约束纳入统一分配过程。因此，对于部分时间约束较紧的场景，即使它们在局部上表现较好，也仍有可能在当前统一分配结果下出现整体完成时间超过截止约束的情况。

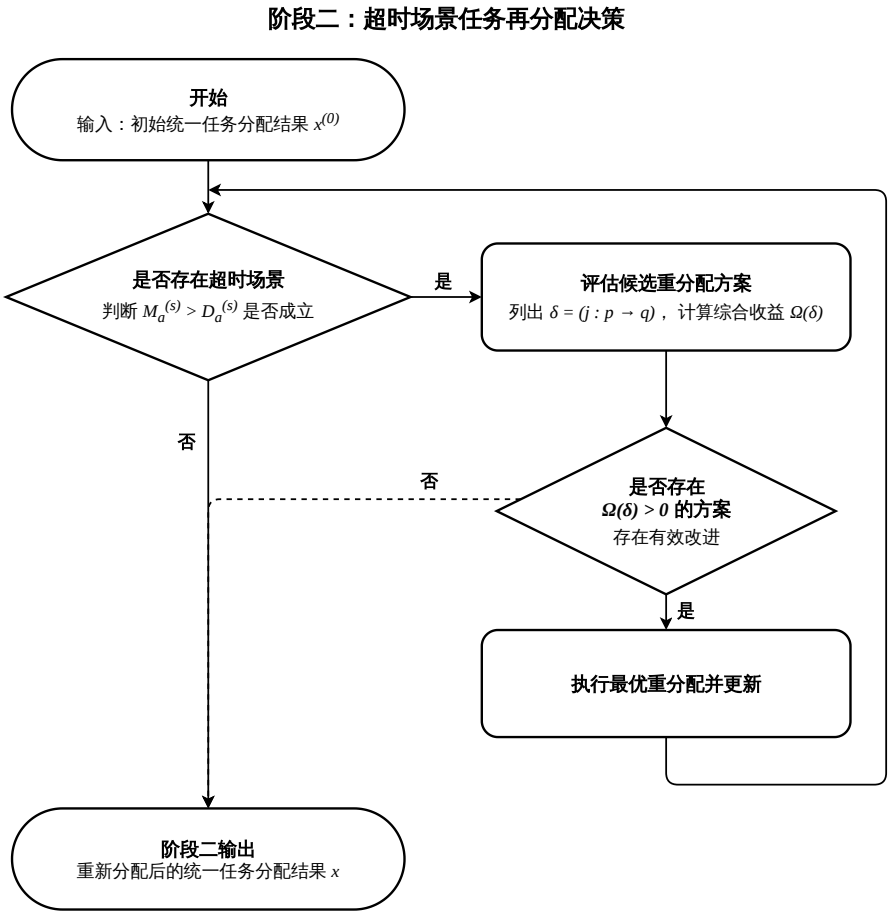


图 4-3 超时场景任务再分配流程图

Fig. 4-3 Flowchart of task reallocation for overtime scenarios

（2）若检查结果表明所有场景均满足截止时间约束，则说明当前统一任务分配结果已经可以作为后续场景内优化的任务分配基础，此时无需继续调整。若仍存在超时场景，则进入局部再分配阶段。与初始统一任务分配阶段不同，这一步的目标不再是构造整体上更优的初始结果，而是针对已经超时的场景，对当前分配结果进行有针对性的调整，尽量使统一分配结果满足所有场景的实时性要求。

（3）在再分配过程中，DHA 先根据超时场景在当前映射下的执行结果，识别出对整体超时更敏感的候选 Stage，然后检查这些 Stage 从当前处理单元 p 改分到其他候选处理单元 q 的可能性。对任意候选重新分配方案

$$\delta = (j : p \to q),$$

DHA 按照式 (4-10) 计算其综合效果

$$\Omega(\delta) = G_{\text{time}}(\delta) - \lambda G_{\text{cost}}(\delta).$$

式中，$G_{\mathrm{time}}(\delta)$ 表示该次重新分配对超时场景截止时间可行性的改善程度；$G_{\mathrm{cost}}(\delta)$ 表示该次修改对其他场景现有分配结果带来的额外影响；λ 为平衡这两部分作用的参数；通过这一指标，DHA 可以在优先缓解超时场景问题和尽量保持当前统一任务分配结果稳定之间取得平衡，避免为了改善少数场景而对整体分配结果造成过大扰动。

（4）在完成候选方案评估后，DHA 根据 $\Omega(\delta)$ 的大小判断当前局部修改是否值得执行。这里，$G_{\mathrm{time}}(\delta)$ 用于衡量该候选方案对超时场景完成时间的改善程度，取值越大，说明该修改越有利于缓解当前场景的超时问题；而 $G_{\mathrm{cost}}(\delta)$ 则用于衡量该修改对其他场景现有任务分配结果所带来的额外影响，取值越小，说明该方案在改善超时场景的同时，对其他场景的扰动越小。当存在 $\Omega(\delta) > 0$ 的候选方案时，说明该次重新分配生成的截止时间满足情况改善，因此可将其视为一次有效改进。

（5）在每一轮局部调整中，DHA 都会从当前候选修改集合中选取综合效果最优的方案，并据此更新统一任务分配结果 x。更新完成后，算法会重新检查当前系统中是否仍存在超时场景。若仍有场景未满足截止时间约束，则继续进入下一轮局部调整；若所有场景均已满足截止时间要求，则该阶段终止，说明当前统一任务分配结果已经具备整体可行性。另一种终止情况是，在某一轮中已不存在满足 $\Omega(\delta) > 0$ 的候选重新分配方案，这表明当前统一任务分配结果已经无法通过单步局部修改继续获得有效改善，此时该阶段同样结束。该阶段的处理流程如图 4-3 所示，对应伪代码见算法 4-3。

综上，超时场景任务再分配阶段是在初始跨场景统一任务分配结果基础上进行的局部调整过程。通过反复执行那些能够带来整体改善的单步重新分配，算法在尽量减少对其他场景影响的同时，逐步提高统一任务分配结果对超时场景的适应能力，也为后面场景内降频优化提供了更稳妥的统一任务分配基础。

算法 4-3　超时场景任务再分配算法

输入：初始统一任务分配结果 x；各场景任务图；最大重新分配轮数 $I_{\max}$。

输出：调整后的统一任务分配结果 x。

1: **for** $iter$ = 1 到 $I_{\max}$ **do**

2:　在当前统一任务分配结果 x 下，判断各场景是否满足截止时间约束；

3:　**if** 所有场景均无超时 **then** 跳出循环；

4:　根据超时情况选出候选 Stage，并逐一判断可以替换的处理单元；

5:　按式(4-10)计算各重新分配方案 δ 的综合效果 $\Omega(\delta)$；

6:　**if** 存在 $\Omega(\delta) > 0$ 的重新分配方案 **then**

7:　　选择效果最好的重新分配方案，并更新统一任务分配结果 x；

算法 4-3　超时场景任务再分配算法

8:　**else** 停止重新分配，跳出循环；

9: **end for**

10: 输出调整后的统一任务分配结果 x。

4.4.3 场景内降频与执行顺序优化流程

经过超时场景任务再分配后，DHA 已经得到一个经过可行性调整的统一任务分配结果 x。该结果在尽量保持跨场景共享分配结构稳定的同时，对部分可能导致场景超时的 Stage 分配进行了调整，从而提高了各场景满足截止时间约束的可能性。至此，跨场景层面的任务分配优化已经基本完成，后续优化不再重点关注 Stage 在不同处理单元之间的重新映射，而是转向各场景内部仍具有调整空间的频率配置和任务执行顺序。

与前两个阶段相比，场景内降频与执行顺序优化阶段的决策范围相对局部。该阶段不再改变全局共享的 Stage 分配位置，而是在统一任务分配结果 x 已经固定的前提下，分别对每个场景中的处理单元运行频率和任务调度顺序进行优化。由于不同场景在该阶段共享的仅是统一任务分配结果 x，而频率配置 $f^{(s)}$、任务开始时间、完成时间以及处理单元上的执行顺序均只作用于各自场景内部，因此各场景之间不存在进一步的耦合关系。基于这一特点，DHA 可以将多场景联合问题进一步分解为若干个场景内优化子问题，并在每个场景中独立完成降频与调度调整。

场景内降频与执行顺序优化可以分为以下几个步骤：

（1）对于任一场景 s，DHA 首先将所有处理单元的运行频率初始化为各自离散频率集合 $\mathcal{F}_p$ 中的最高频率，从而得到该场景的初始频率向量 $f^{(s)}$。采用最高频率作为初始配置的原因在于，高频率通常能够提供更短的 Stage 执行时间，有利于在优化初始阶段快速构造一个满足截止时间约束的可行调度结果。在此基础上，结合已经确定的统一任务分配结果 x，算法为场景中各处理单元上的 Stage 安排初始执行顺序，并计算当前频率配置下的场景总能耗。

按照前文定义，该场景总能耗由两部分组成：一部分是处理单元在场景运行期间产生的静态能耗 $E_{\mathrm{idl}}^{(s)}$，另一部分是各个 Stage 在执行过程中产生的动态能耗 $E_{\mathrm{act}}^{(s)}$。其中，静态能耗主要与处理单元在当前频率下的空闲功耗及场景运行窗口长度有关，动态能耗则与 Stage 的执行时间、处理单元工作功耗以及任务负载特征有关。因此，降低处理单元频率虽然可能减少功耗，但同时也会延长 Stage 执行时间，进而影响任务调度结果和截止时间可行性。

（2）在统一任务分配结果固定之后，场景内优化的核心问题可以概括为：如何在始终满足该场景截止时间约束的前提下，逐步降低部分处理单元的运行频率，从而减少整体能耗。为此，DHA 采用逐步降频的贪心策略。在每一轮迭代中，算法依次检查所有尚未降到最低频率的处理单元 p，尝试将其当前运行频率降低一档，并将降频后得到的临时频率向量记为 $f_{p\downarrow}^{(s)}$。

（3）由于频率变化会直接影响分配到该处理单元上的 Stage 执行时间，并进一步改变任务之间的等待关系和处理单元上的执行顺序，因此对每个候选降频方案，都需要在新的频率配置下重新进行调度。具体而言，算法根据当前统一任务分配结果 x 和临时频率向量 $f_{p\downarrow}^{(s)}$，重新计算各 Stage 的执行时间，并在满足 DAG 依赖关系和处理单元执行约束的前提下，更新该场景内的任务执行顺序、开始时间和完成时间。

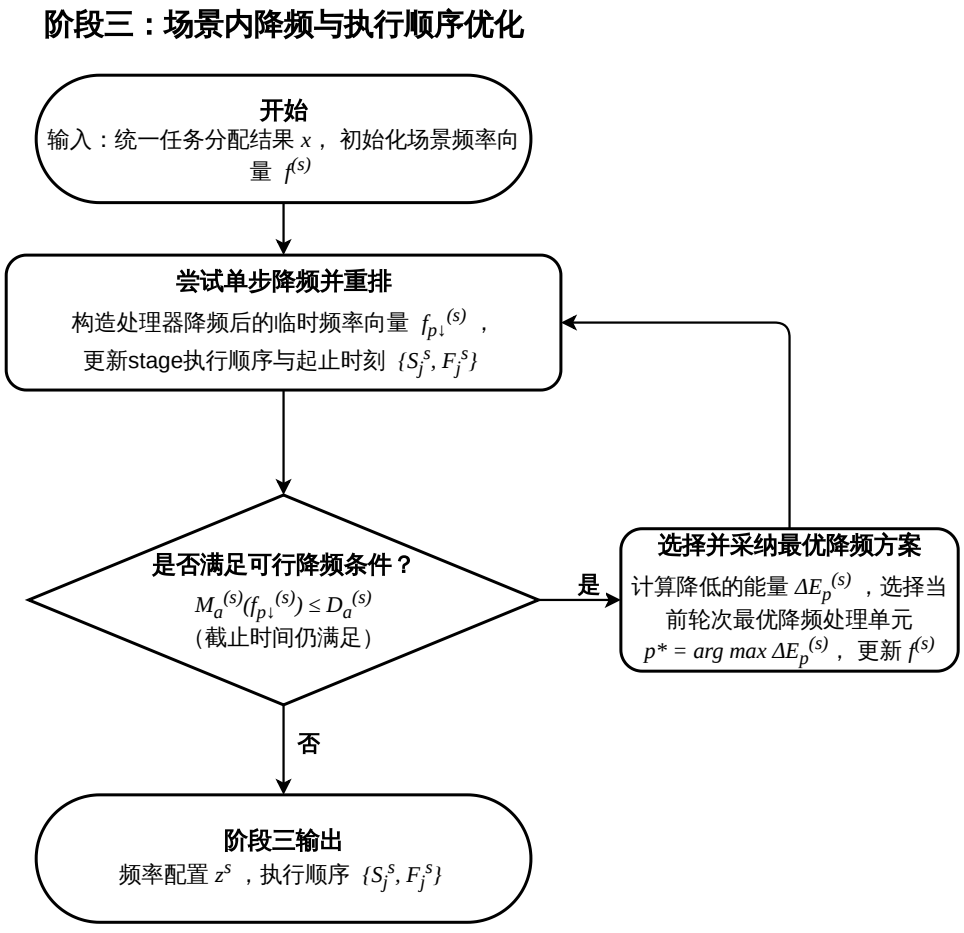


图 4-4 场景内降频与执行顺序优化流程图

Fig. 4-4 Flowchart of intra-scenario frequency scaling and execution order optimization

（4）对于每个满足截止时间约束的可行一步降频方案，DHA 进一步依据式 (4-14) 计算其对应的节能量 $\Delta E_p^{(s)}$。该节能量用于衡量处理单元 p 降低一档频率后，

对当前场景总能耗所带来的改善程度。一般而言，$\Delta E_p^{(s)}$ 越大，说明该降频操作能够带来的能耗下降越明显，也越有利于降低该场景下的系统总能耗。为避免选择节能效果较弱但同样可行的降频操作，算法在每一轮迭代中采用贪心选择策略，优先执行当前可行候选中节能收益最大的降频方案。因此，在完成一轮候选处理单元检查后，DHA 将所有可行降频方案的节能量进行比较，并从中选取节能量最大的处理单元 p^* 作为本轮实际执行降频的对象。随后，算法将处理单元 p^* 的频率降低一档，并同步更新当前场景的频率向量 $f^{(s)}$、各 Stage 的执行时间、任务开始与完成时刻以及对应的任务执行顺序。

（5）完成一轮降频更新后，当前场景的频率配置、任务执行时间以及调度结果都会发生变化，因此算法进入下一轮迭代，继续在新的频率配置基础上检查是否仍存在可行的一步降频方案。在每一轮迭代中，算法仍按照上述过程依次尝试各处理单元的降频操作，并根据更新后的任务完成时间、处理单元占用情况和应用截止时间约束，重新判断候选降频方案的可行性和节能收益。若某一轮检查结束后，不再存在同时满足截止时间约束且能够带来正向节能收益的候选降频方案，则说明在当前任务分配和执行顺序条件下，该场景中各处理单元的可利用时间余量已基本耗尽，继续降低频率将导致截止时间违约或无法获得有效能耗收益。此时，算法终止该场景的降频与执行顺序优化过程，并输出最终频率配置 $f^{(s)}$ 及对应的任务执行顺序。

该阶段的处理流程如图 4-4 所示，对应的伪代码见算法 4-4。

算法 4-4　场景内降频与能耗优化算法

输入：场景 s 的任务与参数；统一任务分配结果 x；离散频率集合 $\mathcal{F}_p$。

输出：场景 s 的运行频率向量 $f^{(s)}$ 及最终执行顺序。

1: 初始化所有处理单元的运行频率为最高频率，得到初始频率向量 $f^{(s)}$；

2: 根据 x 和 $f^{(s)}$，安排各并行处理能力单元上的初始执行顺序；

3: **repeat**

4:　初始化当前最大节能量 $\Delta E_{\max} = 0$，以及本轮选中的处理单元 $p^* = \text{null}$；

5:　**for** 每个还没有降到最低频率的处理单元 p **do**

6:　　尝试将处理单元 p 的频率降低一档，得到临时频率向量 $f_{p\downarrow}^{(s)}$；

7:　　重新计算执行顺序，并检查式(4-13)中的截止时间约束是否仍然满足；

8:　　**if** 满足所有截止时间约束 **then**

9:　　　按式(4-14)计算这次降频对应的节能量 $\Delta E_p^{(s)}$；

10:　　　**if** $\Delta E_p^{(s)} > \Delta E_{\max}$ **then**

11:　　　　$\Delta E_{\max} = \Delta E_p^{(s)}$，$p^* = p$；

算法 4-4　场景内降频与能耗优化算法

12:　　**end if**

13:　**end if**

14:　**end for**

15:　**if** $p^{*} \neq \text{null}$ **then**

16:　　正式将处理单元 p^{*} 的频率降低一档，并更新 $f^{(s)}$ 和对应的执行顺序；

17:　**end if**

18: **until** $p^{*} == \text{null}$

19: 输出最终的频率配置 $f^{(s)}$ 及对应执行顺序。

4.5 仿真结果与分析

4.5.1 仿真参数设置与实验设计

本节仍以 CPU–GPU–NPU 异构平台上的多场景 Stage 级 DAG 推理任务为研究对象。为了与第 3 章保持一致，任务生成方式、执行时间模型和频率相关功耗模型继续沿用第 2 章和第 3 章中的设定。任务规模仍用场景内所有激活应用在 Stage 融合后的总 Stage 数 N_{stage} 表示。除非特别说明，不同算法均使用同一组随机实例。每个实验点独立生成 50 组样本，并取平均结果。场景权重统一设为 $\pi_s = 1/|\mathcal{S}|$。

在平台设置方面，各处理单元均支持离散 DVFS，候选频率集合表示为

$$\mathcal{F}_p = \{0.6, 0.7, 0.8, 0.9, 1.0\} f_p^{\max}.$$

其中，异构处理单元数量记为

$$R = (|\mathcal{P}^{\text{cpu}}|,\ |\mathcal{P}^{\text{gpu}}|,\ |\mathcal{P}^{\text{npu}}|)\,.$$

此外，为分析截止时间约束松紧程度对算法性能的影响，沿用第三章的截止时间系数 β。β 越大表示应用相对截止时间越宽松。

表 4-3 给出了本节采用的主要仿真参数与控制变量设置。

DHA 的内部参数在全部正式实验中保持一致。时间权重取 $\alpha = 0.6$，调整平衡参数取 $\lambda = 0.3$，最大重新分配轮数取 $I_{\max} = 5$，候选 Stage 的数量设为

$$K = \max\left\{3, \left\lceil 0.1 N_{\text{stage}} \right\rceil\right\}.$$

表 4-3 第 4 章仿真参数与控制变量设置

Table 4-3 Simulation parameters and control variable settings in Chapter 4

参数名	参数取值范围或设定
任务规模 N_{stage}	实验一、二：$\{5, 10, 15, \ldots, 50\}$； 实验三：固定为 50； 实验四、实验五：$\{5, 10, 15, \ldots, 100\}$
截止时间系数 β	实验三：$\{1.0, 1.1, 1.2, 1.3, 1.4\}$；
场景–异构处理单元数量	$\mathrm{C}_1 : (\lvert\mathcal{S}\rvert, R) = (1, (2, 1, 1))$；$\mathrm{C}_2 : (2, (4, 1, 1))$； $\mathrm{C}_3 : (4, (4, 2, 1))$；$\mathrm{C}_4 : (8, (4, 2, 2))$
离散频点集合 $\mathcal{F}_p$	$\{0.6, 0.7, 0.8, 0.9, 1.0\} f_p^{\max}$
统计方式	每个实验点独立生成 50 组实例； ETE 与 ST 取平均，DMR 按失约比例统计

这些参数只作为算法实现时的固定设置，本文不再单独分析其变化影响。

本节采用系统期望总能耗 ETE、截止时间超时率 DMR 和求解时间 ST 作为评价指标。涉及不同算法之间的能耗比较时，统一展示归一化后的 ETE。归一化能耗定义为当前算法所得系统期望总能耗与系统整体最大运行能耗之比。该指标用于消除不同实验条件下能耗绝对值差异带来的影响，数值越小表示算法能耗越低。若某种算法未得到可行调度结果，则只统计其 DMR 和 ST，而 ETE 仅在可行实例上取平均。

对比算法包括 DHA、LCG-MILP、EA-HEFT、EA-GA 和 Mapping-Only。其中，DHA 为本文提出的分解式启发调度算法，LCG-MILP 用于提供精确基准，EA-HEFT 和 EA-GA 分别代表列表调度类方法和元启发式方法，Mapping-Only 用于考察仅进行任务映射而不进行频率联合优化时的性能表现。

实验一：在四种场景与异构处理单元数量 C_1 到 C_4 下进行，比较 DHA 和 LCG-MILP 在任务规模 $N_{\text{stage}} \in \{5, 10, \ldots, 50\}$ 变化时的归一化能耗。该实验用于评估 DHA 与精确算法之间的能耗差距。

实验二：在较复杂的 C_3 和 C_4 配置下进行，比较 DHA、EA-HEFT、EA-GA 和 Mapping-Only 在任务规模 $N_{\text{stage}} \in \{5, 10, \ldots, 50\}$ 变化时的归一化能耗。该实验用于比较不同算法的节能能力，并检验 DHA 在复杂多场景配置下的相对优势。

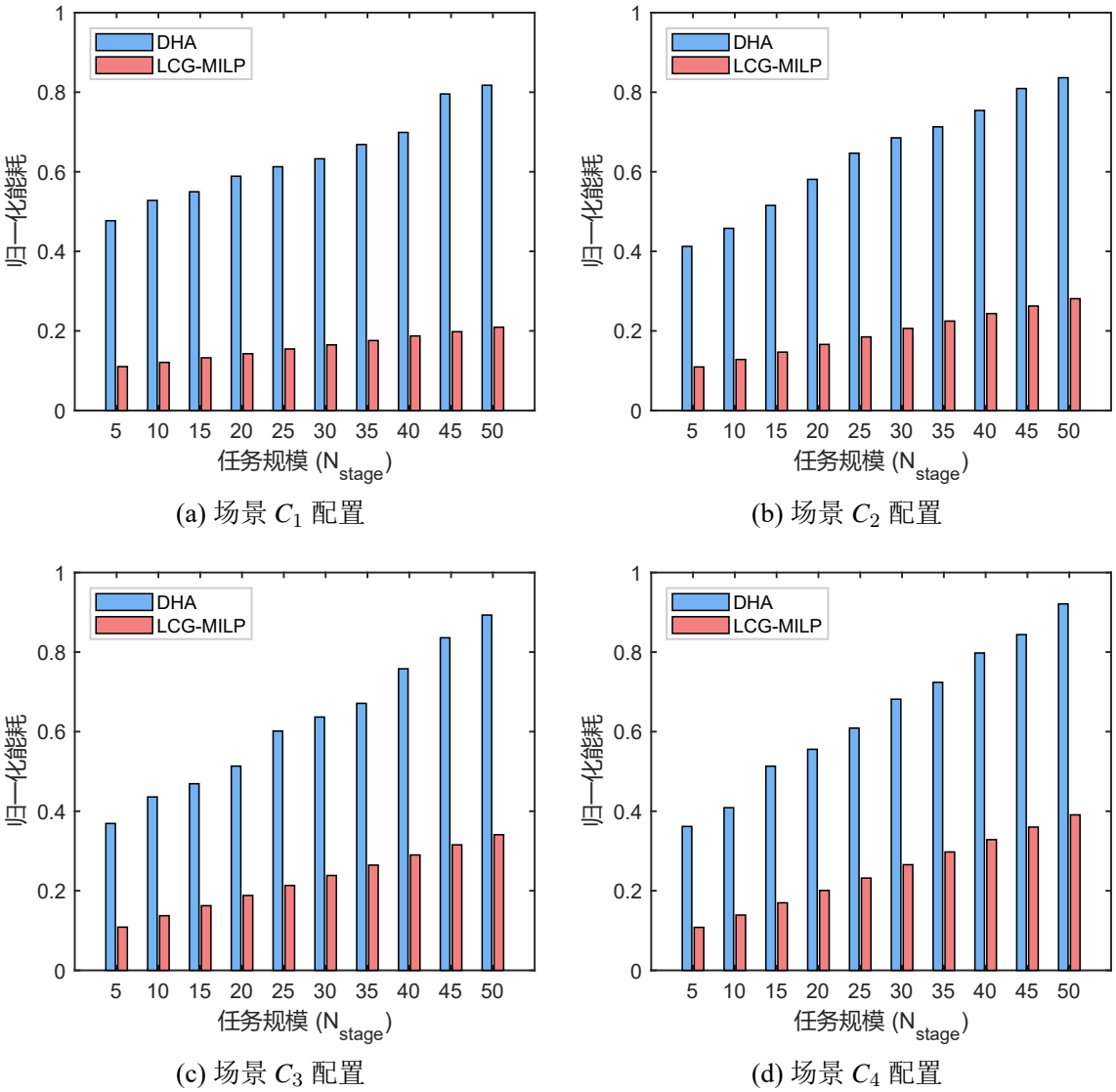
实验三：在固定任务规模 $N_{\text{stage}} = 50$ 和固定配置 C_3 下进行，改变截止时间系数 $\beta \in \{1.0, 1.1, 1.2, 1.3, 1.4\}$，比较 DHA、LCG-MILP、EA-HEFT、EA-GA 和 Mapping-Only 的归一化能耗和可调度率。该实验用于评估截止时间约束松紧程度对各算法节能能力和可行调度能力的影响。

实验四：在 C_1 到 C_4 下进行，比较 DHA、LCG-MILP、EA-HEFT、EA-GA 和

Mapping-Only 在任务规模 $N_{\text{stage}} \in \{5, 10, \ldots, 100\}$ 变化时的可调度率表现。该实验用于分析不同算法在任务规模增加时的可行调度能力。

实验五：在 C_1 到 C_4 下进行，比较 DHA、LCG-MILP、EA-HEFT、EA-GA 和 Mapping-Only 在任务规模 $N_{\text{stage}} \in \{5, 10, \ldots, 100\}$ 变化时的 ST。该实验用于评估各算法的计算效率和规模扩展能力。

4.5.2 实验结果分析



(a) 场景 C_1 配置　　(b) 场景 C_2 配置

(c) 场景 C_3 配置　　(d) 场景 C_4 配置

图 4-5 不同场景配置下 DHA 与 LCG-MILP 算法的能耗对比

Fig. 4-5 Energy consumption comparison between DHA and LCG-MILP under different scenario configurations

（1）实验一结果见图 4-5。该实验比较了 DHA 与第三章精确方法 LCG-MILP 在不同场景与异构处理单元数量、不同任务规模下的期望总能耗。整体来看，在四

组配置 C_1、C_2、C_3 和 C_4 下，随着任务规模 N_{stage} 由 5 增加到 50，两种算法的期望总能耗都持续上升。配置由 C_1 增加到 C_4 后，两种方法的整体能耗水平也同步提高。这说明场景数量、异构处理单元数量和任务规模增加后，统一任务分配、场景内执行顺序和频率选择之间的关系会更复杂，累计能耗也会随之增加。这一结果也说明，本章所考虑的问题具有明显的规模敏感性和配置敏感性，实验设置能够较好反映多场景异构系统中的实际优化难点。

从两种算法的结果来看，LCG-MILP 在所有配置和任务规模下都得到更低的能耗。DHA 的结果虽然更高，但随规模变化的趋势与 LCG-MILP 基本一致，说明 DHA 在较低计算代价下仍能较好接近精确方法的结果。在配置 C_1 下，DHA 相比 LCG-MILP 的额外能耗约为 16% ~ 28%。当 $N_{\text{stage}} = 5$ 时，额外能耗约为 16% ~ 17%。任务规模增加后，这一差距扩大到约 20% ~ 24%。当 $N_{\text{stage}} = 50$ 时，额外能耗约为 27% ~ 28%。这说明即使在单场景和较少异构处理单元数量下，DHA 由于没有进行全局联合搜索，仍会带来一定能耗损失，但整体变化趋势与精确算法保持一致。

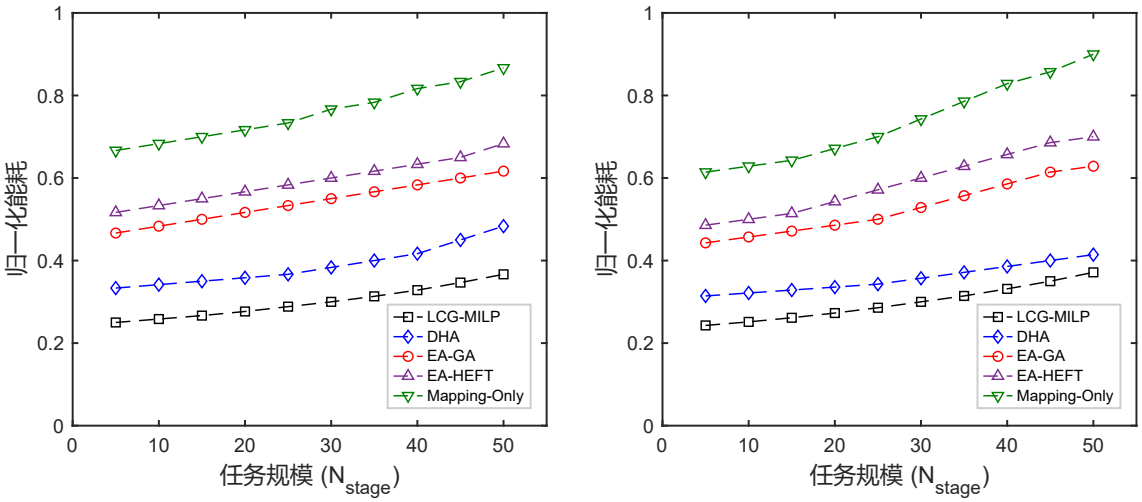
在配置 C_2 下，DHA 与 LCG-MILP 的差距进一步扩大。图中结果表明，DHA 的额外能耗仍约为 16% ~ 28%。在较小任务规模下，额外能耗约为 16% ~ 18%。在中大规模区间，通常达到约 20% ~ 27%。当 $N_{\text{stage}} = 50$ 附近时，额外能耗接近 27% ~ 28%。这说明场景数量增加后，跨场景统一任务分配与局部调度之间的关系更复杂，DHA 相比精确方法的近似误差也更明显，但整体变化仍较稳定。

在更复杂的配置 C_3 下，两种算法的差距继续扩大。根据图中结果，DHA 的额外能耗约为 15% ~ 29%。当 $N_{\text{stage}} = 5$ 时，额外能耗约为 15%。当任务规模增加到 30 ~ 50 时，这一差距扩大到约 23% ~ 29%。这表明在多场景共享优化问题中，随着资源竞争和执行顺序之间的影响进一步增强，DHA 虽然达不到 LCG-MILP 的全局最优能耗水平，但仍能较稳定地保留主要节能效果。

在四组配置中最复杂的 C_4 下，两种算法之间的差距最大。图中结果表明，DHA 的额外能耗约为 15% ~ 29%。在较小规模下，额外能耗约为 15% ~ 17%。当 N_{stage} 增加到 35 ~ 50 时，这一差距进一步扩大到约 24% ~ 29%。在最大任务规模附近，DHA 的能耗比 LCG-MILP 高出接近 30%。将四组配置放在一起看，DHA 的性能损失图像是随着场景数量和异构处理单元数量增加而扩大；这说明该方法在更复杂配置下保持了较为稳定的结果。

（2）实验二结果如图 4-6 所示。该实验在较复杂配置下比较了 LCG-MILP、DHA、EA-GA、EA-HEFT 和 Mapping-Only 五种方法的归一化期望总能耗，用于评估 DHA 的节能效果，并进一步分析其与第三章精确方法之间的差距。根据本章实验设置，实验分别在 $C_3 : (|\mathcal{S}| = 4,\ R = (4, 2, 1))$ 和 $C_4 : (|\mathcal{S}| = 8,\ R = (4, 2, 2))$ 两种配置下进行，任务规模取 $N_{\text{stage}} \in \{5, 10, \ldots, 50\}$，并统一展示归一化后的期望总能耗。

进一步观察 C_3 配置下的结果可以发现，DHA 始终是除 LCG-MILP 之外能耗最低的方法。相对于 DHA，EA-GA 通常会多消耗约 13%～14% 的能量，EA-HEFT 多消耗约 19%～21%，Mapping-Only 则普遍高出约 31%～39%。尤其在中大规模任务区间，DHA 与其余对比算法之间的差距仍保持稳定，这说明在 C_3 对应的中高复杂度多场景系统中，DHA 虽然未达到精确方法的最优能耗水平，但已经能够较好地保留联合优化中的主要节能收益，并显著优于仅依赖局部启发或不进行频率联合优化的方法。



(a) 场景 C_3 配置　　(b) 场景 C_4 配置

图 4-6 不同算法能耗对比

Fig. 4-6 Energy Consumption Comparison of Different Algorithms

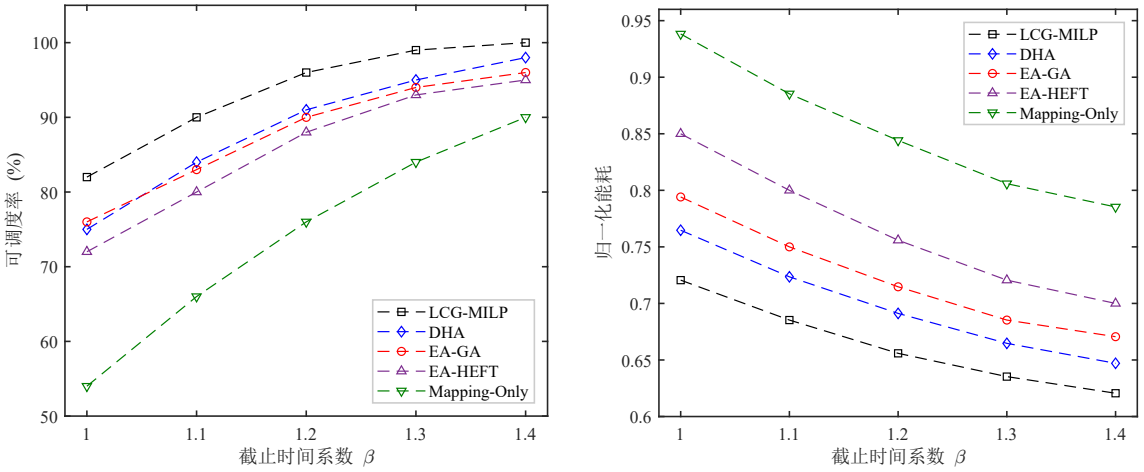
在复杂度更高的 C_4 配置下，不同算法之间的能耗差距进一步扩大，但 DHA 仍保持最优的近似表现。由图可见，与 DHA 相比，EA-GA 通常还会多消耗约 13%～16%，EA-HEFT 多消耗约 19%～22%，Mapping-Only 则高出约 35%～40%。这表明，当场景数量和异构处理单元数量同时增加后，若缺少系统性的联合优化机制，算法更难有效协调任务分配、执行顺序与频率选择之间的关系，能耗损失会进一步放大；相比之下，DHA 仍能够在复杂配置下保持接近精确方法的能耗表现。

（3）实验三的结果见图 4-7。其中，图 4-7(a) 显示了不同截止时间系数下各算法的可调度率。图 4-7(b) 显示了不同截止时间系数下各算法的归一化能耗。

从图 4-7(a) 可以看到，当 β 从 1.0 增大到 1.4 时，各算法的可调度率都在提高。β 越大，应用的相对截止时间越宽。任务图可以使用的时间余量也越多。算法在任务映射、频率选择和执行顺序调整时，就更容易找到满足截止时间约束的调度方案。

DHA 的可调度率整体接近 LCG-MILP，并且高于 EA-HEFT 和 Mapping-Only。

当 β = 1.0 时，DHA 的可调度率约为 75%。随着截止时间约束放宽，DHA 的可调度率持续上升。当 β = 1.4 时，其可调度率达到约 98%。这说明 DHA 能够改善调度可行性。其作用主要来自三个部分。先进行跨场景统一任务分配，再对超时场景进行任务再分配，最后在场景内优化降频和执行顺序。这样可以在较低求解复杂度下得到较高的可调度率。



(a) 不同截止时间系数下各算法的可调度率比较 (b) 不同截止时间系数下各算法的归一化能耗比较

图 4-7 不同截止时间系数下各算法性能对比

Fig. 4-7 Performance comparison of different algorithms under different deadline factors

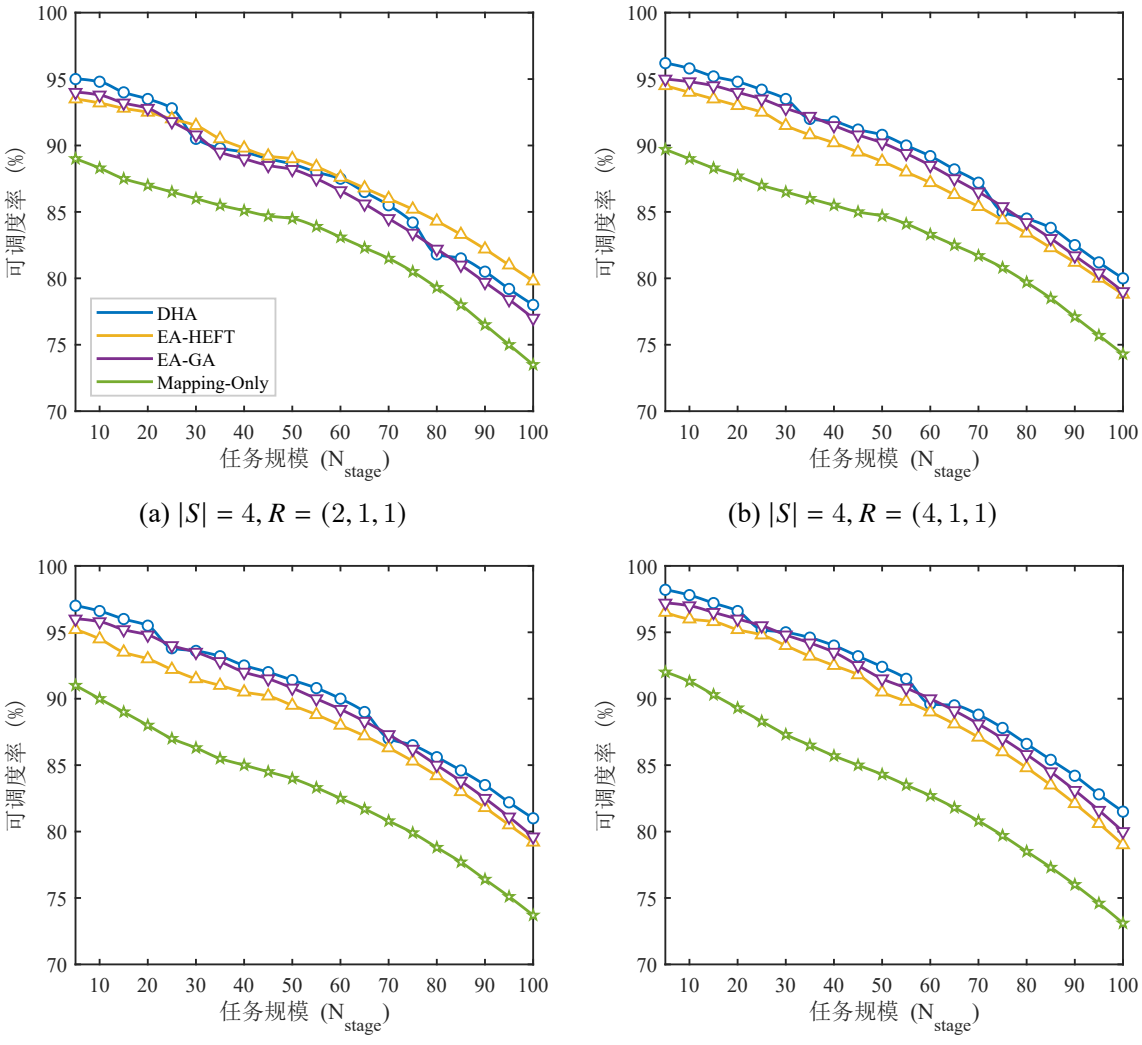
DHA 与 EA-GA 的结果比较接近。在大部分截止时间系数下，DHA 略高于 EA-GA，或者与 EA-GA 基本相当。这说明 DHA 的分解式调整策略能够较好地利用截止时间余量。EA-HEFT 的可调度率低于 DHA 和 EA-GA。原因是 EA-HEFT 更依赖局部最早完成时间来做列表调度，缺少对多场景统一映射和频率配置的整体协调。Mapping-Only 的可调度率最低。当 β = 1.0 时，它的可调度率仅约为 54%。这说明只进行任务映射，而不联合优化频率和执行顺序，很难适应严格截止时间约束下的多场景异构调度需求。

从图 4-7(b) 可以看到，随着 β 增大，各算法的归一化能耗都逐渐下降。原因是截止时间约束放宽后，任务执行可以使用更多时间余量。处理单元可以在更低频率下运行，DVFS 的节能空间也随之增加，所以系统总能耗下降。

从能耗结果看，LCG-MILP 的归一化能耗始终最低。DHA 次之。EA-GA 和 EA-HEFT 的能耗依次升高。Mapping-Only 的能耗最高。当 β = 1.0 时，LCG-MILP 的归一化能耗约为 0.72，DHA 约为 0.77，二者差距较小。当 β = 1.4 时，LCG-MILP 下降到约 0.62，DHA 下降到约 0.65。这说明截止时间约束放宽后，DHA 也能利用降频空间降低能耗。DHA 没有达到 LCG-MILP 的全局最优水平，但它的能耗曲线一直接近 LCG-MILP，并且明显低于 EA-GA、EA-HEFT 和 Mapping-Only。这个结

果说明，DHA 在节能效果和求解效率之间取得了较好的平衡。

（4）实验四结果见图 4-8。在四种异构资源配置下，随着任务规模不断增大，各算法的可调度率整体均呈现下降趋势。这说明随着场景内 Stage 数量持续增加，系统在资源容量、任务依赖关系以及截止时间约束上的耦合冲突不断增强，从而使满足全部约束的可行调度解更加难以获得。同时，当异构资源配置由较紧张的条件逐步扩展到更充足的配置时，各算法的可调度率曲线整体上移，表明增加 CPU、GPU 和 NPU 资源数量能够有效扩大系统的可行调度空间，增强平台对大规模任务运行状态的承载能力。



(a) $|S|=4, R=(2,1,1)$　(b) $|S|=4, R=(4,1,1)$

(c) $|S|=4, R=(4,2,1)$　(d) $|S|=4, R=(4,2,2)$

图 4-8 不同任务规模下各算法的可调度率比较

Fig. 4-8 Comparison of schedulability among different algorithms under different task scales

与此同时，图中也可以观察到，DHA 的可调度率并非在所有实验点上都严格优于全部对比算法；在部分资源配置和任务规模区间下，其结果会略低于个别算法。这一现象说明，DHA 在算法设计上并非要求可调度率的绝对最优，而是在解

质量与求解效率之间进行了折中。换言之，为了获得更短的求解时间和更好的规模扩展能力，DHA 在部分高运行状态条件下适当牺牲了少量可调度性表现。因此，这种局部性能让步本质上反映了启发式算法面向工程应用时对计算效率的优先考虑。

（5）实验五结果见图 4-9。从图中可以看出，随着任务规模增大，不同算法在求解时间上的差距逐渐增大。整体来看，各种启发式算法的求解时间都明显低于精确算法，而且随着任务规模增加，增长趋势相对平缓。相比之下，精确算法在中大规模区间内的求解时间上升更快，对任务规模变化也更敏感。这说明在多场景联合优化问题中，随着 Stage 数量增加，精确建模方法需要同时处理更多任务分配变量、执行顺序变量和频率相关变量，因此求解复杂度会迅速上升。启发式方法则可以用更低的计算代价构造可行解。

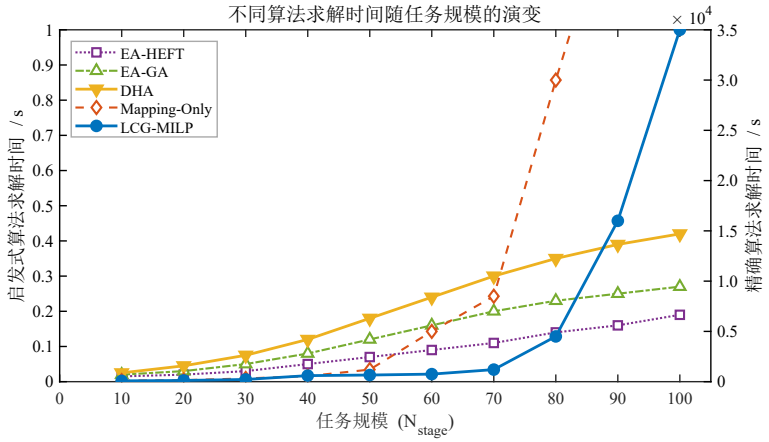


图 4-9 不同算法求解时间对比

Fig. 4-9 Comparison of Solving Time among Different Algorithms

再看几种启发式算法之间的差别，EA-HEFT 的求解时间始终最短，EA-GA 次之，DHA 略高于前两者。不过，这三种方法始终处在同一数量级，在整个任务规模范围内都表现出较好的可扩展性。这说明 DHA 虽然在流程中加入超时场景重新分配和场景内运行频率下调等步骤，使单次求解代价高于只依赖列表调度或较弱搜索方法的基线算法，但这部分代价总体仍然可控，也就是说，DHA 并不是用类似精确算法的高时间代价来换取能耗优势，而是在可接受的启发式求解时间范围内取得了更好的优化效果。

4.6 本章小结

本章针对第 3 章精确优化模型在大规模 Stage 级 DAG 任务图下求解时间过长、难以直接应用的问题，研究了端侧异构平台多场景能效优化问题的近似求解方法。目标是在满足截止时间约束的前提下，用更低的计算代价得到低能耗的调度结果。

首先，在保持第 3 章联合优化含义不变的基础上，本章重新整理了统一任务分配、场景内执行顺序和场景内运行频率配置之间的关系，并围绕大规模场景下的求解需求，构建了分解式启发算法 DHA 的整体框架。

接着，针对多场景联合优化中任务分配、调度和频率之间复杂的关系以及难以直接整体求解的问题，本章把求解过程分成三个阶段，分别是统一任务分配、超时场景重新分配和场景内运行频率下调。算法先通过场景内预调度提取局部选择信息，并生成跨场景统一的任务分配结果。然后利用超时场景反馈，对统一任务分配结果做重新分配，提高调度可行性和整体一致性。最后，在保持可行性的前提下，逐步降低场景内运行频率，把调度过程中留下的时间余量转成实际节能效果。基于这样的分步求解流程，DHA 在保持较低时间复杂度的同时，能够较好接近联合优化问题的高质量解。

最后，多组对比仿真实验验证了所提 DHA 算法的有效性。实验结果表明，在 LCG-MILP 仍可求解的规模范围内，DHA 与精确算法在整体变化趋势上保持一致，其额外能耗大致控制在 $15\% \sim 29\%$ 之间。这说明该方法能够在性能损失可接受的情况下，较好接近全局精确解。在复杂配置 C_3 下，DHA 相比 EA-GA 和 EA-HEFT 的系统期望总能耗分别降低约 $13\% \sim 14\%$ 和 $19\% \sim 21\%$。在更复杂的 C_4 配置下，DHA 相比 EA-GA 和 EA-HEFT 的降幅进一步扩大到约 $13\% \sim 16\%$ 和 $19\% \sim 22\%$。同时，在截止时间系数变化的条件下，DHA 也能较好协调截止时间满足能力与能耗控制能力之间的关系，整体表现优于其他对比方法。

第 5 章 总结与展望

5.1 主要工作与创新点

本文围绕面向深度学习应用的异构端侧平台高能效任务调度问题开展研究，重点关注在 CPU、GPU、NPU 协同环境下，如何在满足任务依赖关系、资源约束和端到端截止时间约束的前提下，降低系统长期运行能耗。

针对现有研究在统一建模、多场景联合优化以及大规模问题可扩展求解方面的不足，本文首先构建了面向 CPU、GPU、NPU 异构端侧平台的统一建模框架。该框架从平台资源、Stage 级 DAG 任务表示以及频率相关能耗模型三个层面，对多场景条件下的深度学习推理调度问题进行了统一描述，为后续算法设计提供了基础。

在此基础上，本文针对中小规模问题，提出了基于延迟约束生成的 LCG-MILP 精确求解方法。该方法将任务分配、执行顺序与频率配置纳入同一优化模型，在保证截止时间约束的前提下，以系统期望总能耗最小化为目标进行联合优化。实验结果表明，该方法能够有效降低系统能耗，并为后续启发式方法提供参考基准。

进一步地，针对精确方法在大规模任务图下求解开销较高的问题，本文提出了分解式启发调度算法 DHA。该方法通过跨场景统一任务分配、超时场景任务再分配以及场景内降频与执行顺序优化，降低了问题求解复杂度。实验结果表明，DHA 虽然在能耗上略高于精确方法，但整体变化趋势与 LCG-MILP 保持一致；同时，在复杂配置下，DHA 相较 EA-GA、EA-HEFT 和 Mapping-Only 仍具有更好的节能效果，体现出较好的综合性能。

5.2 研究展望

在 CPU–GPU–NPU 异构端侧平台上，如何在满足任务依赖关系、处理单元执行约束和端到端截止时间约束的前提下，进一步降低系统长期运行能耗，仍然是一个具有挑战性的研究问题。本文围绕多场景条件下的高能效任务调度问题开展了较为系统的研究，并提出了精确优化方法和分解式启发方法，但仍存在一定局限性。基于本文的研究内容，后续工作可从以下几个方面进一步展开：

（1）进一步研究面向动态环境的在线自适应调度机制。本文采用有限场景集合对系统运行模式进行离线建模，并在此基础上开展多场景联合优化。然而，在实际端侧系统中，输入复杂度、应用启停、分辨率切换以及运行状态变化通常具有更强的不确定性和时变性，离线预设场景难以完全覆盖真实运行状态。因此，后续可结合在线场景识别、运行时反馈控制以及离线–在线协同优化机制，进一步研究能够

适应动态环境变化的能效调度方法，以增强算法在复杂实际场景下的适用性。

（2）进一步引入更真实的系统约束与多维优化目标。本文当前建模主要考虑任务依赖关系、处理单元执行约束、截止时间约束以及频率相关能耗之间的联合优化，而对片上存储层次、共享总线带宽、数据搬移开销、NPU 编译部署约束以及热约束等实际系统因素的描述相对简化。随着端侧 SoC 架构和深度学习模型复杂度不断提升，这些因素将对调度结果产生显著影响。因此，后续研究可进一步将存储访问、通信争用、热管理、可靠性以及多目标权衡纳入统一框架，构建更贴近真实平台特征的联合优化模型。

（3）进一步加强真实平台与真实应用场景下的实验验证。本文目前主要基于仿真环境对所提方法进行了验证，虽然实验结果能够较好说明算法在理论模型下的有效性，但其在真实端侧异构平台上的部署效果仍有待进一步检验。后续工作可将算法部署到实际的 CPU–GPU–NPU 端侧设备上，结合典型深度学习模型和真实应用负载，对算法的能耗、截止时间、稳定性及运行开销进行更全面的评估，并基于真实测量结果对模型参数和求解流程进行进一步修正与优化。

参考文献

[1] SINGH R, GILL S S. Edge AI: a survey[J]. Internet of Things and Cyber-Physical Systems, 2023, 3: 71-92.

[2] ANDRIANANDRIANINA JOHANESA T V, EQUETER L, MAHMOUDI S A. Survey on AI applications for product quality control and predictive maintenance in industry 4.0[J]. Electronics, 2024, 13(5): 976.

[3] MALLIORIS P, AIVAZIDOU E, BECHTSIS D. Predictive maintenance in Industry 4.0: A systematic multi-sector mapping[J]. CIRP journal of manufacturing science and technology, 2024, 50: 80-103.

[4] TU X, MALLIK A, CHEN D, et al. Unveiling energy efficiency in deep learning: Measurement, prediction, and scoring across edge devices[C]. IEEE/ACM Symposium on Edge Computing, Wilmington, DE, USA, 2023: 80-93.

[5] WU Z, HAN L, LIU J, et al. Energy-aware mapping and scheduling strategies for real-time workflows under reliability constraints[J]. Journal of Parallel and Distributed Computing, 2023, 176: 1-16.

[6] LI X, LI Y, LI Y, et al. FlexNN: Efficient and Adaptive DNN Inference on Memory-Constrained Edge Devices[C]. The 30th Annual International Conference on Mobile Computing and Networking, Washington, D.C., USA, 2024: 709-723.

[7] HAN L, ZHOU Z, LI Z. Pantheon: Preemptible Multi-DNN Inference on Mobile Edge GPUs [C]. The 22nd Annual International Conference on Mobile Systems, Applications and Services, Minato-ku, Tokyo, Japan, 2024: 465-478.

[8] XU D, XU M, WANG Q, et al. Mandheling: Mixed-Precision On-Device DNN Training with DSP Offloading[C]. The 28th Annual International Conference on Mobile Computing and Networking, Sydney, NSW, Australia, 2022: 214-227.

[9] XU D, ZHANG H, YANG L, et al. Fast On-device LLM Inference with NPUs[C]. 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Rotterdam, The Netherlands, 2025: 445-462.

[10] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J]. IEEE journal of solid-state circuits, 2016, 52 (1): 127-138.

[11] ZHANG S, DU Z, ZHANG L, et al. Cambricon-X: An Accelerator for Sparse Neural Networks [C]. 49th Annual IEEE/ACM International Symposium on Microarchitecture, Taipei, Taiwan,

2016: 1-12.

[12] KWON H, CHATARASI P, SARKAR V, et al. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings[J]. IEEE micro, 2020, 40(3): 20-29.

[13] 陈刚, 关楠, 吕鸣松, 等. 实时多核嵌入式系统研究综述[J]. 软件学报, 2018, 29(07): 2152-2176.

[14] ESMAEILZADEH H, BLEM E, ST. AMANT R, et al. Dark silicon and the end of multicore scaling[C]. 38th International Symposium on Computer Architecture, San Jose, CA, USA, 2011: 365-376.

[15] 瞿伟, 余飞鸿. 基于多核处理器的非对称嵌入式系统研究综述[J]. 计算机科学, 2021, 48 (S1): 538-542.

[16] ALAM S, YAKOPCIC C, WU Q, et al. Survey of deep learning accelerators for edge and emerging computing[J]. Electronics, 2024, 13(15): 2988.

[17] SAMANTA A, HATAI I, MAL A K. A survey on hardware accelerator design of deep learning for edge devices[J]. Wireless Personal Communications, 2024, 137(3): 1715-1760.

[18] KIM J, HA S. Energy-aware scenario-based mapping of deep learning applications onto heterogeneous processors under real-time constraints[J]. IEEE Transactions on Computers, 2022, 72 (6): 1666-1680.

[19] XU D, LI Q, XU M, et al. Niagara: Scheduling dnn inference services on heterogeneous edge processors[C]. 21st International Conference on Service-Oriented Computing, Rome, Italy, 2023: 67-85.

[20] BOUZIDI H, ODEMA M, OUARNOUGHI H, et al. Map-and-conquer: Energy-efficient mapping of dynamic neural nets onto heterogeneous MPSoCs[C]. 60th ACM/IEEE Design Automation Conference, San Francisco, CA, USA, 2023: 1-6.

[21] SPIECK J, WILDERMANN S, TEICH J. A Scenario-Based DVFS-Aware Hybrid Application Mapping Methodology for MPSoCs[J]. ACM Transactions on Design Automation of Electronic Systems, 2024, 29(4): 1-43.

[22] LIN C, WANG K, LI Z, et al. A workload-aware DVFS robust to concurrent tasks for mobile devices[C]. The 29th Annual International Conference on Mobile Computing and Networking, Madrid, Spain, 2023: 1-16.

[23] SEO W, CHA S, KIM Y, et al. SLO-aware inference scheduler for heterogeneous processors in edge platforms[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2021, 18 (4): 1-26.

[24] QIN Y, ZENG G, KURACHI R, et al. Energy-aware task allocation for heterogeneous multiprocessor systems by using integer linear programming[J]. Journal of Information Processing, 2019,

27: 136-148.

[25] HUANG J, BUCKL C, RAABE A, et al. Energy-aware task allocation for network-on-chip based heterogeneous multiprocessor systems[C]. 19th International Euromicro Conference on Parallel, Distributed and Network-Based Processing, Ayia Napa, Cyprus, 2011: 447-454.

[26] LI Y, NIU J, ATIQUZZAMAN M, et al. Energy-aware scheduling on heterogeneous multi-core systems with guaranteed probability[J]. Journal of Parallel and Distributed Computing, 2017, 103: 64-76.

[27] CHEN J, HE Y, ZHANG Y, et al. Energy-aware scheduling for dependent tasks in heterogeneous multiprocessor systems[J]. Journal of Systems Architecture, 2022, 129: 102598.

[28] LANE N D, BHATTACHARYA S, GEORGIEV P, et al. DeepX: A software accelerator for low-power deep learning inference on mobile devices[C]. 15th ACM/IEEE International Conference on Information Processing in Sensor Networks, Vienna, Austria, 2016: 1-12.

[29] FANG B, ZENG X, ZHANG M. NestDNN: Resource-aware multi-tenant on-device deep learning for continuous mobile vision[C]. The 24th Annual International Conference on Mobile Computing and Networking, New Delhi, India, 2018: 115-127.

[30] MATHUR A, LANE N D, BHATTACHARYA S, et al. DeepEye: Resource efficient local execution of multiple deep vision models using wearable commodity hardware[C]. The 15th Annual International Conference on Mobile Systems, Applications, and Services, Niagara Falls, NY, USA, 2017: 68-81.

[31] XIANG Y, KIM H. Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference[C]. IEEE Real-Time Systems Symposium, Hong Kong, SAR, China, 2019: 392-405.

[32] KANG W, LEE K, LEE J, et al. LaLaRAND: Flexible Layer-by-Layer CPU/GPU Scheduling for Real-Time DNN Tasks[C]. 42nd IEEE Real-Time Systems Symposium, Dortmund, Germany, 2021: 329-341.

[33] RUSSO E, BLANCO F G, PALESI M, et al. Towards Fair and Firm Real-Time Scheduling in DNN Multi-Tenant Multi-Accelerator Systems via Reinforcement Learning[C]. IEEE International Symposium on Circuits and Systems, Singapore, 2024: 1-5.

[34] YU C, WANG M, CHEN S, et al. Efficient NPU–GPU scheduling for real-time deep learning inference on mobile devices[J]. Journal of Real-Time Image Processing, 2025, 22(2): 1-13.

[35] LIU C L, LAYLAND J W. Scheduling algorithms for multiprogramming in a hard-real-time environment[J]. Journal of the ACM (JACM), 1973, 20(1): 46-61.

[36] BUTTAZZO G C. Hard real-time computing systems: predictable scheduling algorithms and applications[M]. Springer, 1997.

[37] STANKOVIC J A. Misconceptions about real-time computing: A serious problem for next-

generation systems[J]. Computer, 2002, 21(10): 10-19.

[38] LIU D, HU X S, LEMMON M D, et al. Firm real-time system scheduling based on a novel QoS constraint[J]. IEEE Transactions on Computers, 2006, 55(3): 320-333.

[39] BERNAT G, BURNS A, LIAMOSI A. Weakly hard real-time systems[J]. IEEE transactions on Computers, 2001, 50(4): 308-321.

[40] HAMDAOUI M, RAMANATHAN P. A dynamic priority assignment technique for streams with (m, k)-firm deadlines[J]. IEEE transactions on Computers, 2002, 44(12): 1443-1451.

[41] 彭林. 软件事务存储并行编程环境关键技术研究[D]. 国防科学技术大学, 2010: 15-20.

[42] 陈磊. 异构多核架构下的高能效混合任务调度算法研究[D]. 重庆邮电大学, 2022: 15-20.

[43] HAMMOND L, OLUKOTUN K. Considerations in the Design of Hydra: a Multiprocessor-on-a-Chip Microarchitecture[R]. Stanford University Technical Report No. CSL-TR-98-749, 1998.

[44] HAMMOND L, HUBBERT B A, SIU M, et al. The stanford hydra cmp[J]. IEEE micro, 2000, 20(2): 71-84.

[45] 黄乐天, 别丽华. 同构与异构片上多核系统的演进过程[J]. 电子技术应用, 2017, 43(03): 6-11.

[46] BARUAH S K, MOK A K, ROSIER L E. Preemptively scheduling hard-real-time sporadic tasks on one processor[C]. 11th IEEE Real-Time Systems Symposium, Lake Buena Vista, Florida, USA, 1990: 182-190.

[47] 陈晓明. 异构分布式嵌入式系统的高能效与安全性调度研究[D]. 湖南大学, 2018: 15-30.

[48] CHENG Y, WU Z, LIU K, et al. Smart dag tasks scheduling between trusted and untrusted entities using the mcts method[J]. Sustainability, 2019, 11(7): 1826.

[49] VERUCCHI M, OLMEDO I S, BERTOGNA M. A survey on real-time DAG scheduling, revisiting the global-partitioned infinity war[J]. Real-Time Systems, 2023, 59(3): 479-530.

[50] ABADI M, BARHAM P, CHEN J, et al. TensorFlow: A System for Large-Scale Machine Learning[C]. 12th USENIX Symposium on Operating Systems Design and Implementation, Savannah, GA, USA, 2016: 265-283.

[51] CHEN T, MOREAU T, JIANG Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[C]. 13th USENIX Symposium on Operating Systems Design and Implementation, Carlsbad, CA, USA, 2018: 578-594.

[52] MA L, XIE Z, YANG Z, et al. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks[C]. 14th USENIX Symposium on Operating Systems Design and Implementation, Banff, Alberta, Canada, 2020: 881-897.

[53] KHAN A A. Developing an Energy-efficient Real-time System[M]. Missouri University of Science and Technology, 2018.

[54] POUWELSE J, LANGENDOEN K, SIPS H J. Application-directed voltage scaling[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2003, 11(5): 812-826.

[55] 杨逸. 异构多核计算系统的节能任务调度算法研究[D]. 重庆邮电大学, 2020: 10-15.

[56] MEI X, WANG Q, CHU X. A survey and measurement study of GPU DVFS on energy conservation[J]. Digital Communications and Networks, 2017, 3(2): 89-100.

[57] BHAGAVATHULA V K, GAO X, ZHOU Y, et al. Energy-Efficiency Modeling for AI Applications on Edge Computing[C]. 25th International Conference on Internet Computing and IoT and 22nd International Conference on Embedded Systems, Cyber-physical Systems, and Applications, Las Vegas, NV, USA, 2024: 164-177.

[58] CHUNG J, KIM H, SHIN K, et al. A layer-wise frequency scaling for a neural processing unit [J]. ETRI Journal, 2022, 44(5): 849-858.

[59] TOPCUOGLU H, HARIRI S, WU M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260-274. DOI: 10.1109/71.993206.

[60] MO L, KRITIKAKOU A, SENTIEYS O. Energy-quality-time optimized task mapping on DVFS-enabled multicores[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11): 2428-2439.

[61] WANG Z, ZHANG Y, WEI F, et al. Using Analytical Performance/Power Model and Fine-Grained DVFS to Enhance AI Accelerator Energy Efficiency[C]. 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Rotterdam, The Netherlands, 2025: 1118-1132.

[62] SUN Z, ZHANG B, GU C, et al. ET2FA: A Hybrid Heuristic Algorithm for Deadline-Constrained Workflow Scheduling in Cloud[J]. IEEE Transactions on Services Computing, 2022, 16(3): 1807-1821.

作者简介

1. 攻读学位期间的研究成果

1.1 申请（授权）专利

[1] **第 2 作者** (导师第 1 作者)，2026.

1.2 参与的科研项目及获奖

[1] 重庆邮电大学研究生学业奖学金，三等奖，2023.

[2] 重庆邮电大学研究生学业奖学金，三等奖，2024.

[3] 重庆邮电大学研究生学业奖学金，二等奖，2025.

致　谢